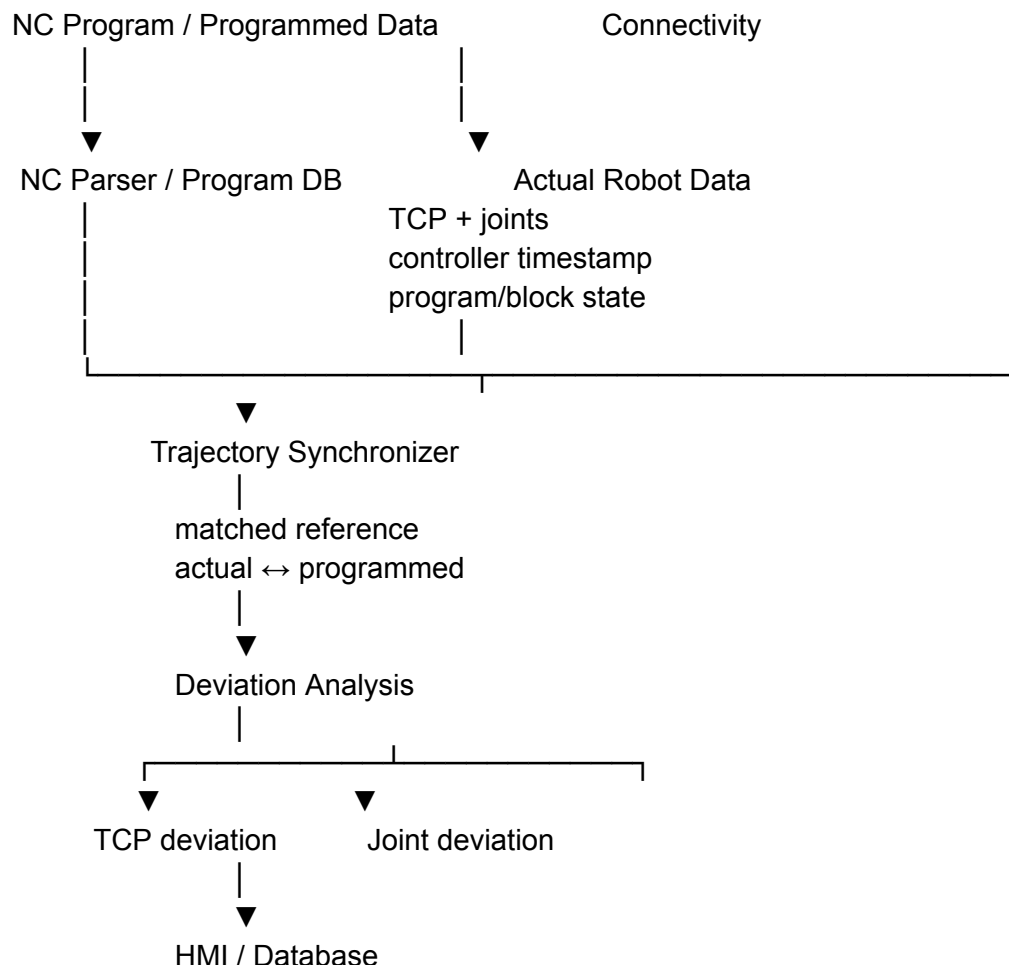


In case of calculating tcp or joint value deviation we require actual data from connectivity component and we will have the nc programmed file, which will be saved in data base
How do we do the time synchronisation and calculate?

Yes. This is an important point for your WP5.3 architecture because **you should not simply compare the nth actual TCP sample with the nth point extracted from the NC file**. The actual data and programmed data usually have different sampling rates, timestamps, execution delays, and potentially different representations.

For your case, I would introduce a dedicated **Trajectory Synchronization / Alignment step** between data ingestion and deviation calculation.

The overall flow should become:



1. First question: what does the NC file actually contain?

This matters because an NC program typically describes the **commanded path**, for example:

G01 X100 Y200 Z50 F1000
G01 X120 Y200 Z50 F1000
G01 X140 Y210 Z52 F1000
...

or in an AFP-specific system it may contain course information, machine commands, process start/stop commands and geometric waypoints.

This is **not necessarily the exact interpolated trajectory executed by the controller**.

The controller may internally generate something like:

NC waypoint

P1 ----- P2

Controller interpolation:

P1
├─ qref(t0)
├─ qref(t1)
├─ qref(t2)
├─ qref(t3)
├─ ...
└─ P2

So you should distinguish between:

Programmed NC path

and

Controller reference trajectory

For **TCP path deviation**, the NC path is often sufficient for your Process Twin analysis if you properly interpolate the programmed geometry.

For **joint deviation**, ideally you should obtain the controller's programmed/reference joint values. You cannot always derive an unambiguous programmed joint trajectory from Cartesian NC coordinates because multiple IK solutions can correspond to the same TCP pose.

2. The ideal synchronization method

The best solution is for your connectivity component to provide more than just:

actual_TCP
actual_joint_values

It should ideally give:

timestamp
actual TCP
actual joints

program ID
run ID

NC block number
course number
segment number

program execution state

For example:

```
{  
  "timestamp": 1725365112.245,  
  "run_id": "AFP_RUN_0035",  
  "program_id": "Panel_A_NC_012",  
  "block_id": 258,  
  "course_id": 7,  
  
  "tcp": {  
    "x": 1.245,  
    "y": 0.376,  
    "z": 0.827  
  },  
  
  "joints": [  
    0.25,  
    -1.15,  
    1.43,  
    -0.72,  
    1.57,  
    0.18  
  ]  
}
```

That gives WP5.3 much stronger synchronization than attempting to infer everything from timestamps.

3. What should be stored from the NC program?

When the NC file is loaded into the database, I would not store only the raw file.

Parse it into a structured representation.

For example:

Run/Program	Block	Course	Sequence	Programmed TCP	Feed	Program State
Panel_A	101	1	0	P_0	100 mm/s	approach
Panel_A	102	1	1	P_1	100 mm/s	laying
Panel_A	103	1	2	P_2	100 mm/s	laying
Panel_A	104	1	3	P_3	100 mm/s	laying

Then the NC-processing component can construct a continuous reference trajectory.

Conceptually:

Raw NC

G01 ...

G01 ...

G01 ...

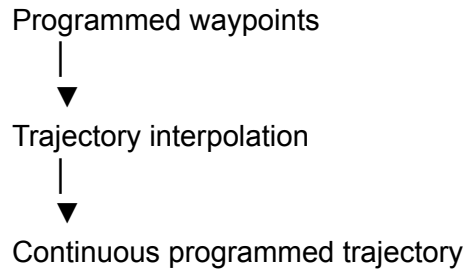
|



NC Parser

|





$P_{ref}(s)$

Here s represents progress along the programmed path.

That is actually more useful than immediately assigning timestamps.

4. Time synchronization alone is not enough

Suppose:

Program starts at 10:00:00.000

Programmed TCP:
100 Hz reference

Actual data:
50 Hz connectivity

Even if the clocks are perfectly synchronized, imagine the robot pauses for 2 seconds because of:

machine hold
AFP tow event
operator pause
controller condition

Then simple elapsed-time matching becomes wrong.

For example:

PROGRAMMED TIME

0 ---- 1 ---- 2 ---- 3 ---- 4 ---- 5
 |
 Expected P3

ACTUAL EXECUTION

0 ---- 1 ---- HOLD ---- HOLD ---- 2 ---- 3
 |
 Actual P2

If you compare purely based on wall-clock time, you may compare:

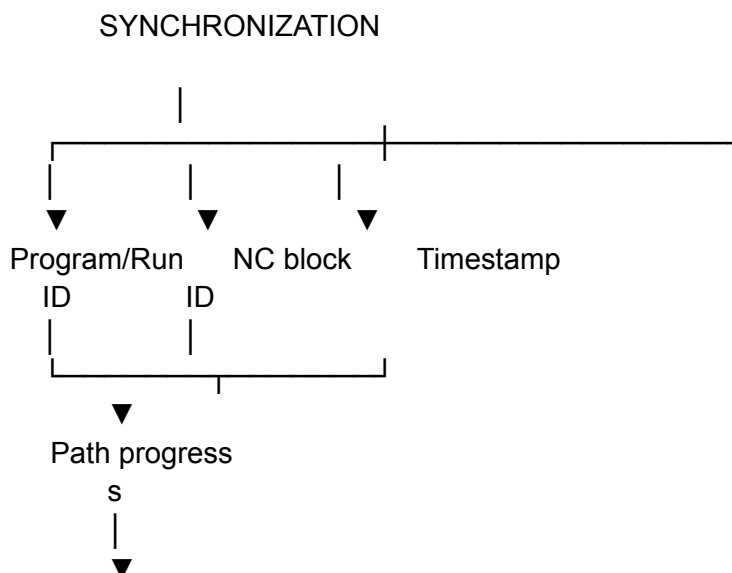
Actual P2
 against
 Programmed P4

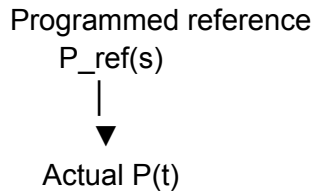
and calculate a huge false deviation.

Therefore I recommend using **execution synchronization first** and timestamps second.

5. Recommended synchronization hierarchy for WP5.3

I would use this priority:





In practical terms:

Best: `run_id + course_id + NC block/segment + timestamp`

Good: `run_id + course_id + timestamp`

Fallback: geometrical path alignment / path-progress matching.

6. For TCP deviation, I would preferably synchronize by path progress

For your visualization use case, this is particularly useful.

Consider the programmed path:

P0 — P1 — P2 — P3 — P4

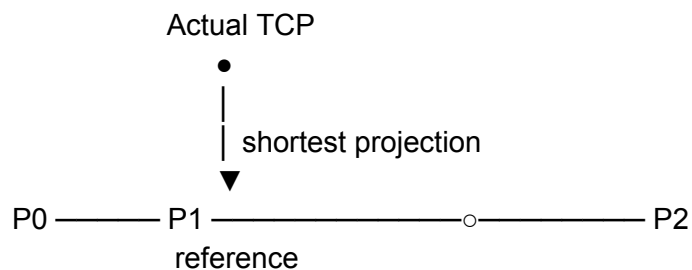
Instead of asking:

Which programmed point has exactly the same timestamp?

you can ask:

Where does this actual TCP correspond on the programmed path?

For an actual TCP point:



You calculate the closest point on the programmed curve.

Let:

P_a = actual TCP position

$P_r(s)$ = programmed trajectory

Find:

$$s^* = \operatorname{argmin}_s \|P_a - P_r(s)\| \quad s^* = \operatorname{argmin}_s \|P_a - P_r(s)\|$$

Then:

$$e = P_a - P_r(s^*) \quad e = P_a - P_r(s^*)$$

and TCP position deviation is:

$$d = \|e\| \quad d = \|e\|$$

So you obtain:

actual_TCP

programmed_TCP_at_corresponding_path_position

dx

dy

dz

deviation

This works especially well for your **CAD heatmap**.

7. But don't use unconstrained nearest point alone

There is a problem.

Suppose AFP has closely spaced neighbouring courses:

Course 1 —————>

Course 2 <—————

Course 3 —————>

A nearest-point algorithm could accidentally associate an actual point from Course 2 with Course 1.

Therefore the search should be constrained by:

run_id
+
course_id
+
segment/block
+
previous path progress

The correspondence should also be **monotonic**.

If the previous matching point was:

$s = 10.52 \text{ m}$

the next point should normally be around:

$s \geq 10.52 \text{ m}$

rather than suddenly matching:

$s = 4.2 \text{ m}$

8. Combining time and geometry is the strongest approach

For your system I recommend a hybrid synchronizer.

Suppose actual data arrives at:

$t_{\text{actual}} = 13.245 \text{ s}$

The synchronizer first uses execution information to establish a candidate window:

Course 7
NC blocks 250–255

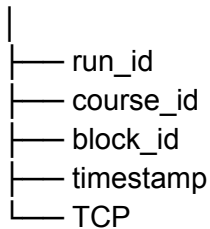
Then it uses timestamp/progress to narrow the search:

expected progress ≈ 4.21 m

Then geometry finds the best reference point within that small region.

So:

Actual sample



Candidate programmed segment



Time/progress estimate



Local geometric projection



Synchronized programmed TCP

This is considerably more robust than only timestamp matching.

9. What if the NC file already provides timing?

If the NC information gives:

position
speed/feed
acceleration rules

you can generate a nominal time-parameterized programmed trajectory:

$P_{ref}(t)$

Then actual timestamps can be interpolated against it.

Suppose programmed data is:

t0	P0
t1	P1
t2	P2

Actual sample occurs at:

t_a

where:

$t_1 < t_a < t_2$

Then interpolate:

$$\alpha = \frac{t_a - t_1}{t_2 - t_1} \quad \alpha = \frac{t_a - t_1}{t_2 - t_1}$$

and:

$$P_r(t_a) = P_1 + \alpha(P_2 - P_1) \quad P_r(t_a) = P_1 + \alpha(P_2 - P_1)$$

Then deviation is:

$$e(t_a) = P_a(t_a) - P_r(t_a) \quad e(t_a) = P_a(t_a) - P_r(t_a)$$

This is straightforward, but again it assumes that program execution follows the nominal timing. Pauses and overrides require additional handling.

10. Clock synchronization is another separate issue

There are actually **two synchronization problems**:

1. CLOCK SYNCHRONIZATION

Connectivity timestamp $\leftrightarrow$ controller timestamp $\leftrightarrow$ server timestamp

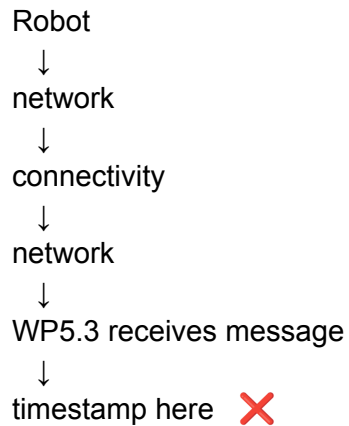
2. TRAJECTORY SYNCHRONIZATION

Actual TCP $\leftrightarrow$ correct position in NC trajectory

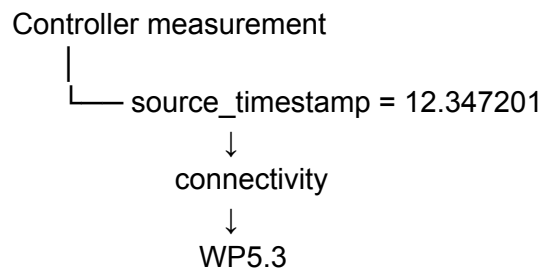
Don't combine them.

For the production system, actual measurements should preferably be timestamped **at the source**, not when WP5.3 receives them.

Bad:



Better:



Network latency then does not distort the measurement time.

For multiple industrial machines/computers, use a synchronized clock infrastructure such as **PTP** where high precision is required, or NTP where millisecond-level synchronization is sufficient for the analysis.

11. Joint deviation needs slightly different handling

If connectivity provides:

$q_{\text{actual}}(t)$

you need:

$q_{\text{reference}}(t)$

before calculating joint deviation.

Then for joint i :

$e_{q_i} = q_{\text{actual},i} - q_{\text{reference},i}$

For example:

Programmed joints

J1 = 32.00°

J2 = -52.30°

J3 = 68.10°

...

Actual joints

J1 = 32.15°

J2 = -52.22°

J3 = 68.41°

...

Then:

J1 error = +0.15°

J2 error = +0.08°

J3 error = +0.31°

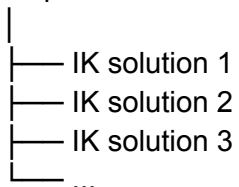
But if your NC database contains only:

X Y Z Rx Ry Rz

you cannot simply claim that an IK-generated joint vector is the **programmed joint value**.

For a redundant or multi-solution robot:

TCP pose



The controller chose one particular configuration.

Therefore for reliable **joint tracking deviation**, my preference is:

Controller reference joints

+

Actual joints

from connectivity.

If those reference joints are unavailable, you should describe an IK-derived comparison as an **estimated/reference joint trajectory**, not the controller's exact commanded joint trajectory.

12. Coordinate-frame synchronization is equally critical for TCP

Before calculating TCP deviation:

Programmed TCP



NC coordinate frame

Actual TCP



Robot base / machine frame

must be transformed into the same frame, preferably your:

PART / PANEL FRAME

So:

Programmed NC

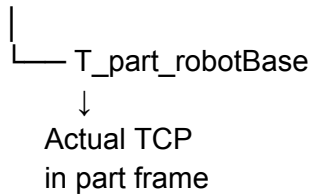


└─ T_part_NC



Programmed TCP
in part frame

Actual robot TCP



Only then calculate deviation.

Otherwise the heatmap would be measuring coordinate-system misalignment as though it were robot tracking error.

13. What I would make the combined synchronized data look like

After synchronization, your **Data Analysis input** could become:

```
{
  "run_id": "AFP_RUN_0035",
  "program_id": "Panel_A_NC_012",
  "course_id": 7,
  "block_id": 258,

  "source_timestamp": 1725365112.245,

  "path_progress_m": 4.326,

  "programmed_tcp": {
    "x": 1.2451,
    "y": 0.3760,
    "z": 0.8274
  },

  "actual_tcp": {
    "x": 1.2460,
    "y": 0.3756,
    "z": 0.8292
  },

  "programmed_joint": [...],
```

```
"actual_joint": [...]  
}
```

Data Analysis adds:

```
{  
  "tcp_deviation": {  
    "dx_mm": 0.9,  
    "dy_mm": -0.4,  
    "dz_mm": 1.8,  
    "magnitude_mm": 2.05  
  },  
  
  "joint_deviation_deg": [  
    0.12,  
    -0.08,  
    0.22,  
    0.04,  
    -0.11,  
    0.06  
  ]  
}
```

That same synchronized record later becomes extremely useful when you add AFP process parameters:

timestamp
path progress
actual TCP
programmed TCP
TCP deviation
actual joints
joint deviation
tow tension
compaction force
temperature
speed
pressure
anomaly score

Everything becomes associated with the **same physical position on the aircraft panel**.

For your WP5.3 architecture, I would therefore slightly change the modules

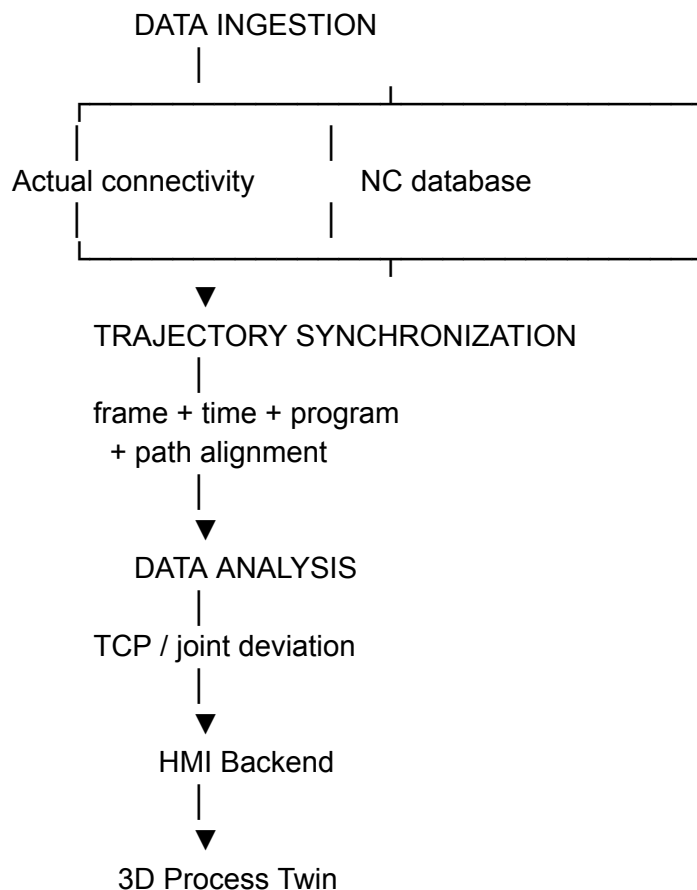
Instead of:

Data Ingestion



Data Analysis

I recommend:



For your particular AFP use case, I would make **run_id + course_id/block_id + source timestamp + monotonic path-progress projection** the main synchronization strategy. That gives you something much more robust than timestamp-only matching and will also scale naturally when you later add process parameters and anomaly visualization.

We are planning to use quest db

We use a platform called RXU which is similar like ros 2 , but it supports different middleware such as dds, fast dds, ros 2 etc

So the communication medium between any rxu components will. Be through pub sub or service client

But we use protobuf structure to transmit messages

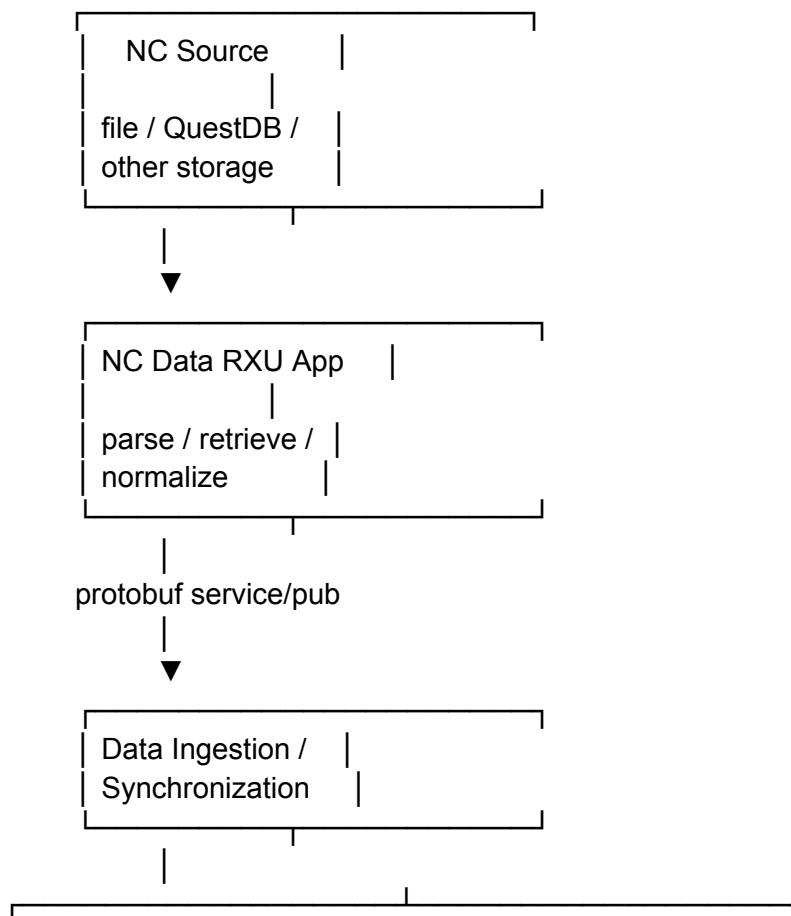
Here data base also an rxu component and other rxu apps doesn't need to know how to query data from quest db and its is Abstracted

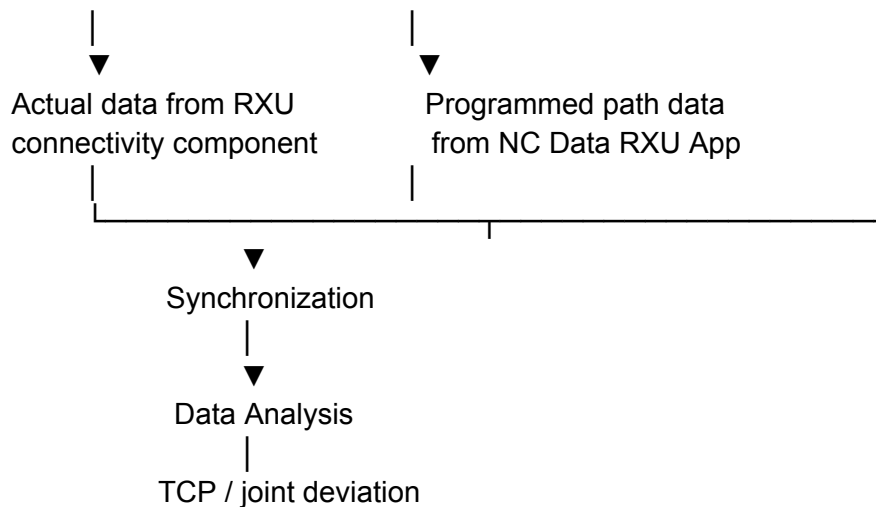
Now I am not sure that NC data will be available as file or as table in the data base

That actually makes the architecture cleaner, because with RXU the other WP5.3 components should **not care whether the NC source is a file, a QuestDB table, or something else**. The database/storage details stay behind an RXU component boundary, and the rest of the system consumes protobuf messages through publish/subscribe or service/client interfaces.

For your case, I would define the NC/programmed-path access as an **abstract RXU service**, not as “query QuestDB directly”.

The architecture should look like this:





The important point is that the **Data Analysis component should never know** whether the programmed trajectory came from:

- .nc file
- CSV
- QuestDB
- PostgreSQL
- filesystem
- PLC
- external service

It should only know:

“I can request the programmed trajectory for run X / program Y / course Z, and I receive a protobuf structure.”

That is the right abstraction.

I would separate “NC storage” from “NC representation”

At the moment, you are uncertain whether the NC program will be stored as a file or as rows in QuestDB. You do not need to resolve that yet at the architecture level.

Define a logical model such as:

```

message ProgrammedTrajectoryRequest {
  string program_id = 1;
  string run_id = 2;
  optional string course_id = 3;
}
  
```

and return:

```

message ProgrammedTrajectory {
  string program_id = 1;
  string run_id = 2;

  repeated ProgrammedTrajectoryPoint points = 3;
}

```

with something like:

```

message ProgrammedTrajectoryPoint {
  uint64 sequence = 1;

  double path_position = 2;

  Pose tcp = 3;

  repeated double joint_position = 4;

  optional uint32 nc_block = 5;
  optional string course_id = 6;

  optional double nominal_speed = 7;
  optional double nominal_time = 8;
}

```

The caller does not need to know how that message was obtained.

If the NC is stored as a file

The NC Data RXU component could internally do:

```

request(program_id)
  ↓
QuestDB/file metadata
  ↓
locate .nc file
  ↓
NC parser
  ↓
interpolated programmed trajectory
  ↓
protobuf ProgrammedTrajectory

```

QuestDB may then contain only metadata:

program_id
filename
file_path / object reference
revision
created_at
part_id
machine_id

while the actual NC file remains as a file.

If NC data is stored as structured records

Then internally:

request(program_id)
↓
RXU DB component
↓
QuestDB
↓
rows:
block / X / Y / Z / orientation / feed / etc.
↓
ProgrammedTrajectory protobuf

Again, the Data Analysis side sees exactly the same protobuf structure.

This is why I would avoid designing your WP5.3 interfaces around QuestDB SQL.

Where QuestDB fits

QuestDB is particularly well suited to your **actual execution/process data**, because that data is naturally time-series:

timestamp
run_id
course_id
actual_tcp_x
actual_tcp_y
actual_tcp_z
joint_1
...
joint_n

pressure
temperature
speed
tension
...

That is a natural QuestDB representation.

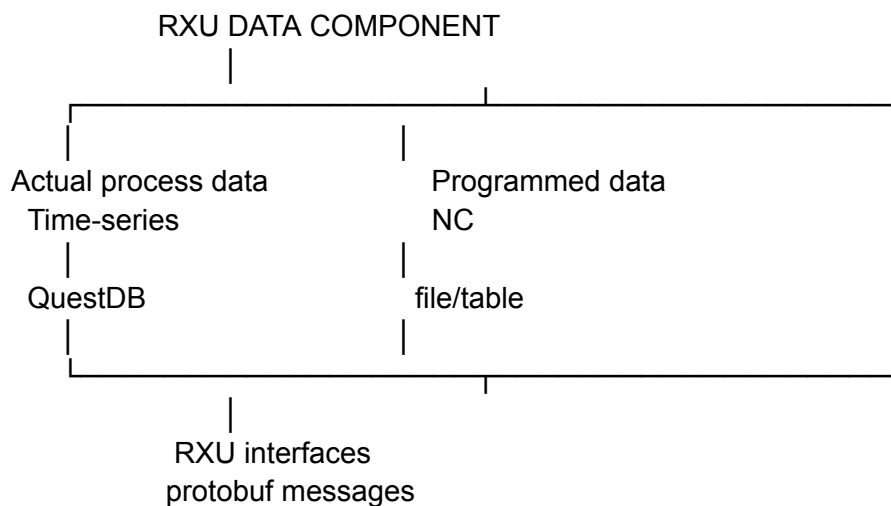
The NC/programmed path is different. It is fundamentally more like:

program → course → segment/block → trajectory

rather than purely a time-series.

You *can* store it in QuestDB, but I would not let the database choice dictate your logical architecture.

A useful model is:



I would also distinguish two RXU communication patterns

For **actual TCP**, use pub/sub because it is streaming:

Connectivity RXU App

publishes:

ActualMachineState

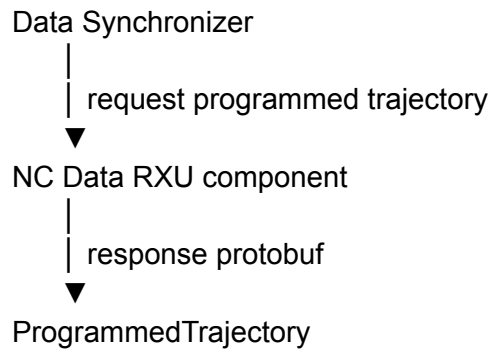
ActualTCP
ActualJoints
ProcessParameters

For example:

```
message ActualMachineState {  
    int64 timestamp_ns = 1;  
  
    string run_id = 2;  
    string program_id = 3;  
    string course_id = 4;  
  
    Pose actual_tcp = 5;  
    repeated double actual_joint = 6;  
  
    optional uint32 nc_block = 7;  
}
```

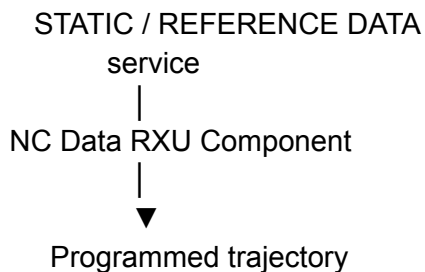
Your ingestion/synchronization component subscribes to this continuously.

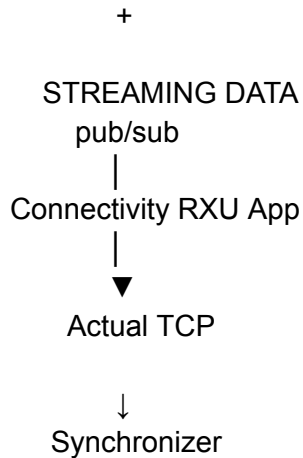
For **programmed NC data**, I would normally use a **service/client** interface because it is reference data:



You don't need to publish the entire NC trajectory continuously.

This becomes:





That is a strong architectural fit for RXU.

Then how should synchronization work?

Once the synchronization component gets both sources, it can maintain the programmed trajectory in memory.

For example, at the beginning of a run:

Actual state arrives:

```
run_id    = RUN_025
program_id = AFP_PANEL_A_17
course_id = C04
```

The synchronizer sees that it does not yet have the corresponding programmed trajectory.

It calls:

```
GetProgrammedTrajectory(
    program_id = AFP_PANEL_A_17
)
```

The NC Data component returns:

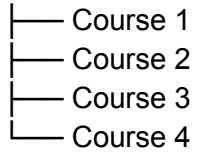
```
P0
P1
P2
...
```

Pn

The synchronizer caches it:

Program cache

AFP_PANEL_A_17



Actual messages then arrive continuously:

t0 actual P

t1 actual P

t2 actual P

...

and synchronization is done against the cached reference.

That avoids querying the database for every actual TCP sample.

This is important.

Do **not** design:

actual sample



QuestDB query



programmed sample



calculate

for every incoming point.

Instead:

START OF RUN

Retrieve programmed path once



Cache reference trajectory

RUNTIME

actual sample



synchronize against cache



calculate deviation

This will be much more efficient and deterministic.

What about time?

Your programmed NC information may not initially contain a timestamp, and that is okay.

I would represent it using **path progress**, sequence, block, and course:

sequence
course_id
block_id
arc_length_s
programmed TCP
feedrate

Example:

Course 4

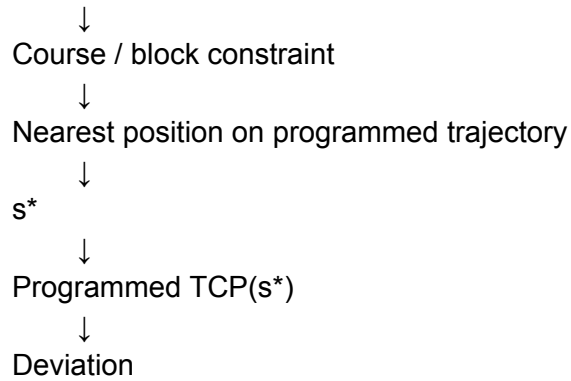
seq	s[m]	TCP
0	0.000	P0
1	0.005	P1
2	0.010	P2
...		

The actual data has:

timestamp
actual TCP
course / block information if available

Then your synchronizer can calculate:

Actual TCP(t)



So you are not dependent on the NC file providing absolute timestamps.

That is probably the most important architectural point for your situation.

If the machine gives NC block information, use it

If the connectivity component can publish:

program_id
course_id
active NC block

then synchronization becomes substantially easier.

For example:

Actual sample

timestamp = 17:05:33.154
program = P17
course = C4
block = 287
TCP = Pa

The synchronizer uses:

P17
↓
C4

↓
block 287
↓
reference segment

and then projects the actual TCP only onto that region.

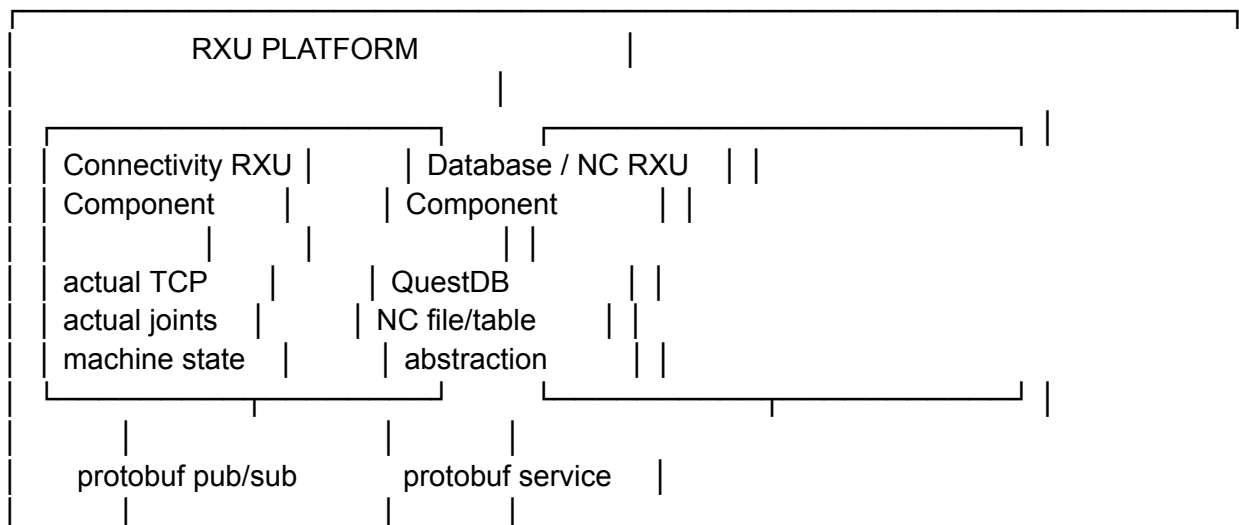
This is far safer than searching the whole programmed path.

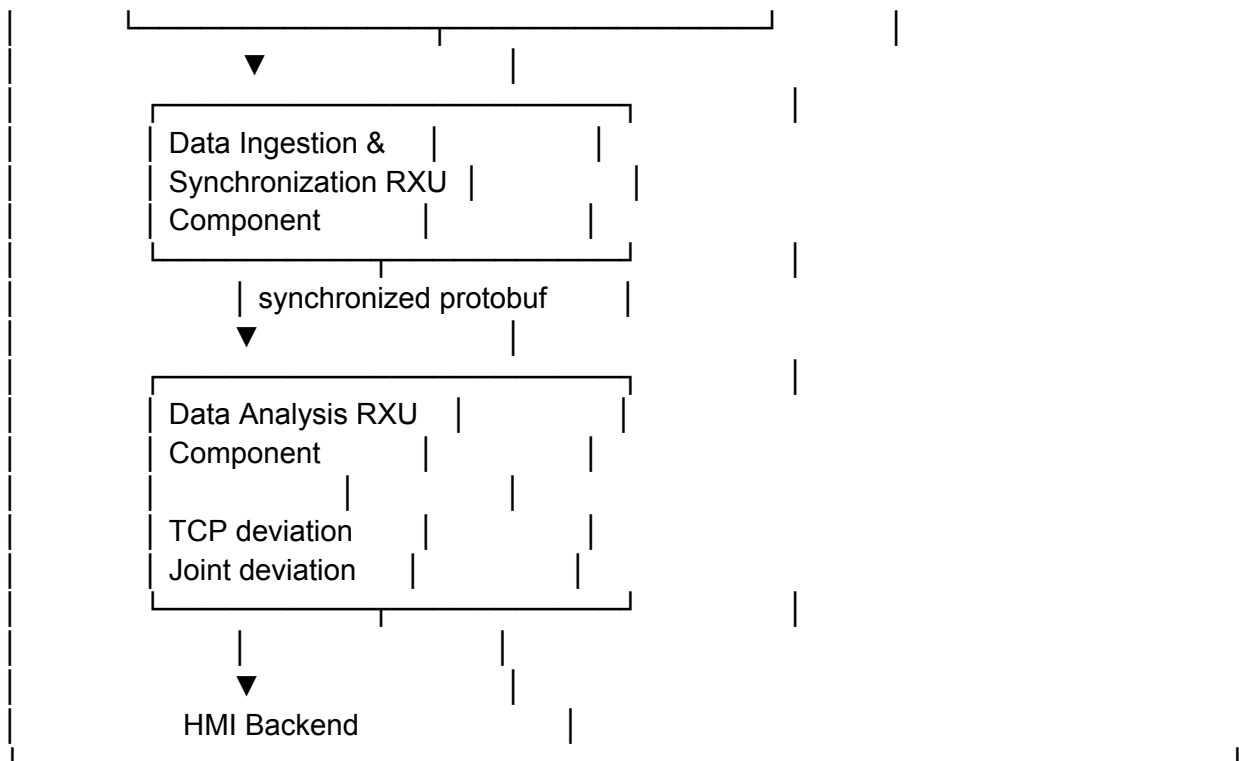
So if you have influence over the **Connectivity RXU protobuf**, I would strongly recommend requesting these fields:

source timestamp
run ID
program ID
active course
active NC block / segment
actual TCP
actual joints
machine state

Not every field needs to exist on day one, but having `program_id` and some form of `block/course progress` will make deviation analysis much more robust.

I would put the components like this in your WP5.3 architecture





And importantly, I would name the interface from the storage side something like **Programmed Data Service** or **NC Reference Data Service**, not “QuestDB Interface.”

That keeps the architecture technology-independent.

For your current uncertainty about whether NC will be a **file** or a **QuestDB table**, I would explicitly document it as an implementation decision still open, while freezing the protobuf interface now. That way the rest of WP5.3 can proceed without waiting for the storage decision.

I still didn't understand how do. We calculate the deviation after time synchronisation
Can you explain in both cases where
1. Nc program available as file
2. Nc program available as the

Yes. I think the missing point is this:

Time synchronization does not itself calculate the deviation. Its job is to determine **which programmed/reference TCP point should be compared with each actual TCP measurement.** Once that correspondence is established, deviation calculation is simple.

I assume your second case means **NC/programmed data is available as structured data/table through the database RXU component.**

Core idea

Suppose connectivity gives:

Actual:

timestamp = 10.135 s

TCPactual = [101.2, 200.4, 50.3]

You need to find:

What was the PROGRAMMED TCP corresponding to this same execution instant/progress?

After synchronization gives you:

TCPprogrammed = [100.0, 200.0, 50.0]

TCPactual = [101.2, 200.4, 50.3]

then:

$dx = 101.2 - 100.0 = 1.2 \text{ mm}$

$dy = 200.4 - 200.0 = 0.4 \text{ mm}$

$dz = 50.3 - 50.0 = 0.3 \text{ mm}$

and total TCP deviation is:

$d = \sqrt{dx^2 + dy^2 + dz^2}$

Here that is about **1.30 mm**.

The difficult part is therefore obtaining the correct **TCPprogrammed** for that actual measurement.

Case 1 — NC program is available as a file

Imagine the NC file contains:

```
N100 X=0 Y=0 Z=100 F=100
N110 X=100 Y=0 Z=100 F=100
N120 X=200 Y=0 Z=100 F=100
N130 X=300 Y=0 Z=100 F=100
```

These are **waypoints**, not necessarily samples corresponding one-to-one with the actual data.

Actual connectivity may instead give:

timestamp	actual X
10.000	0.1
10.010	1.0
10.020	2.1
10.030	3.0
...	
10.500	50.4
...	
11.000	100.2

Therefore you cannot compare:

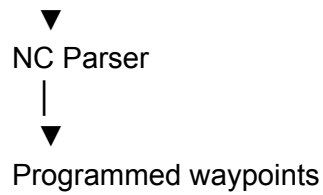
NC point 1 ↔ Actual point 1
NC point 2 ↔ Actual point 2

because they represent completely different sampling.

Step 1 — Parse the NC file

Your NC Data RXU component reads the file and converts it into a structured trajectory representation:

NC FILE
|



P0
P1
P2
P3
...

For example:

Block	X	Y	Z	Feed
N100	0	0	100	
N110	100	0	100	
N120	200	0	100	
N130	300	0	100	

Then construct/interpolate the programmed path.

Step 2 — Identify execution start

Suppose connectivity tells you:

Program started:

10:00:00.000

This becomes your execution reference:

$t_{\text{execution}} = t_{\text{actual}} - t_{\text{start}}$

So an actual sample received at:

10:00:00.350

has:

execution time = 0.350 s

Step 3 — Obtain the corresponding programmed point

There are several possibilities depending on what information the machine provides.

If the NC program and machine information allow you to construct the nominal time trajectory, you could have:

Programmed trajectory

t = 0.00	X=0
t = 0.10	X=10
t = 0.20	X=20
t = 0.30	X=30
t = 0.40	X=40

Actual sample:

t = 0.35

There is no exact programmed sample at 0.35.

So interpolate between:

t=0.30	→ X=30
t=0.40	→ X=40

giving:

Programmed X at t=0.35 = 35

Now suppose:

Actual X at t=0.35 = 35.8

Deviation in X:

$35.8 - 35.0 = +0.8 \text{ mm}$

This is **time-based interpolation**.

But there is a significant problem with that approach

Imagine the machine stops for 2 seconds.

PROGRAMMED

0s ----- 1s ----- 2s ----- 3s ----- 4s
P0 P1 P2 P3 P4

ACTUAL

0s ----- 1s ----[MACHINE HOLD]---- 3s ---- 4s
P0 P1 P1 P1.5 P2

At wall-clock time 3 seconds:

Nominal programmed trajectory says → P3

Actual machine is still around → P1

You would calculate an enormous "deviation", even though this is mainly a **timing/progress difference**, not necessarily a spatial tracking error.

Therefore I would **not rely only on elapsed time** for your AFP TCP heatmap.

Better: synchronize using execution state + path position

Suppose connectivity also provides:

timestamp
program_id
course_id
NC block
actual TCP

Actual message:

timestamp = 10:00:12.352
program = Panel_123
course = 7
NC block = 450
actual TCP = Pa

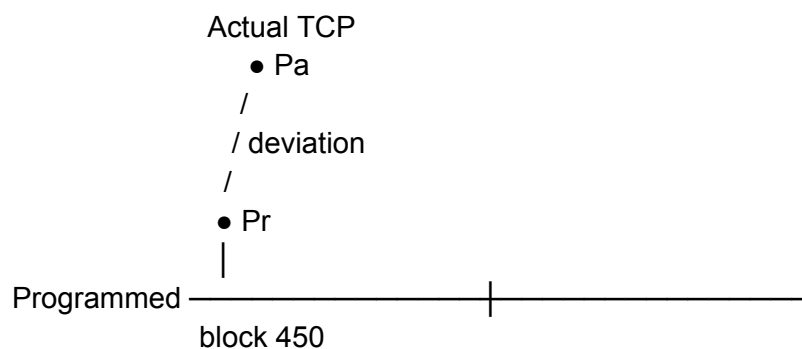
The synchronization component knows from the parsed NC file:

Course 7

Block 440 ——— Block 450 ——— Block 460
 ↑
 machine currently here

So instead of searching the entire NC file, it considers only the programmed path around block 450.

Then it projects the actual point onto that programmed segment:



Pr is now the corresponding programmed TCP.

Then:

Deviation vector = $Pa - Pr$

This is the value you put on your heatmap.

Case 2 — NC/programmed data is available as a database table

Suppose the database RXU component internally has structured data like:

Program	Cours e	Block	Se q	X	Y	Z	Fee d
P123	7	440	0	100	200	50	100

P123	7	450	1	110	200	50	100
P123	7	460	2	120	200	50	100
P123	7	470	3	130	200	50	100

The major difference from Case 1 is that **you don't need the NC parsing step at runtime.**

Your synchronization RXU component requests:

GetProgrammedTrajectory

program_id = P123

course_id = 7

Database RXU component internally accesses QuestDB and returns protobuf:

ProgrammedTrajectory

P0

P1

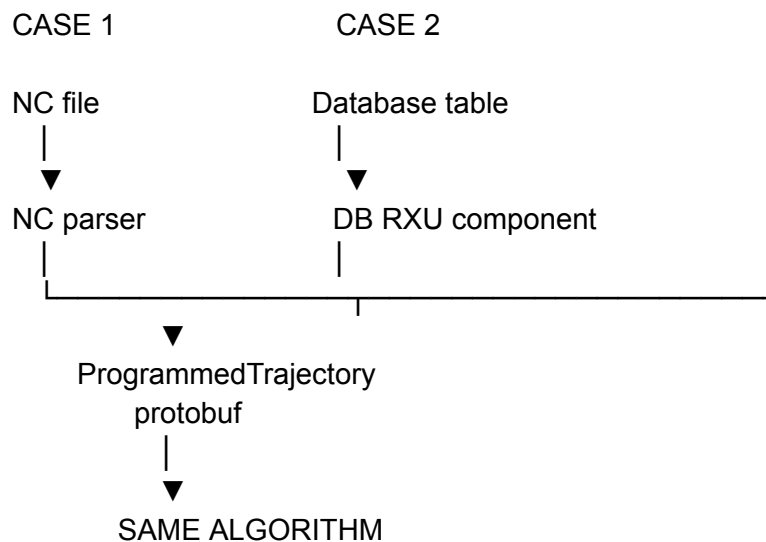
P2

P3

...

From that point onward, **the synchronization and deviation algorithm is exactly the same as Case 1.**

That's an important point:



Storage changes. **Deviation calculation does not.**

Let's use a realistic example

Suppose the database/programmed trajectory contains:

Course 5

Point	X	Y	Z
P0	100	200	50
P1	110	200	50
P2	120	200	50
P3	130	200	50

Actual connectivity message:

timestamp = 12.354

course = 5

block = corresponding to $P1 \rightarrow P2$

Actual TCP:

$X = 115.4$

$Y = 200.7$

$Z = 51.2$

The actual point lies somewhere between programmed P1 and P2.

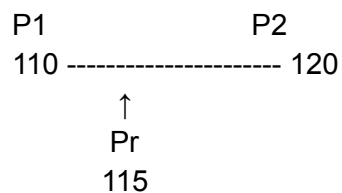
So don't compare against P1:

[110,200,50] **×**

or P2:

[120,200,50] **×**

Find the corresponding interpolated point on the segment:



Programmed reference becomes:

Pr = [115.0, 200.0, 50.0]

Actual:

Pa = [115.4, 200.7, 51.2]

Therefore:

dx = +0.4 mm

dy = +0.7 mm

dz = +1.2 mm

and:

total deviation $\approx$ 1.45 mm

That 1.45 mm is what your HMI can map to the corresponding location on the 3D panel.

Where does timestamp synchronization enter then?

This is probably the part causing the confusion.

Timestamp synchronization is primarily used to establish **execution correspondence**, not to directly subtract positions.

Think of three stages:

ACTUAL DATA

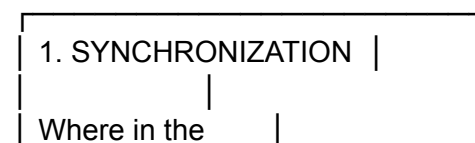
timestamp

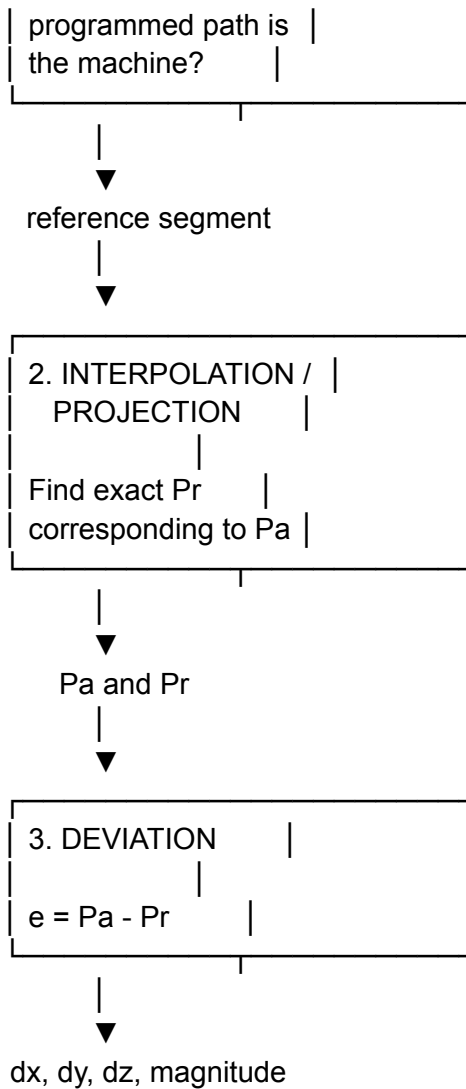
program ID

course ID

block ID

actual TCP



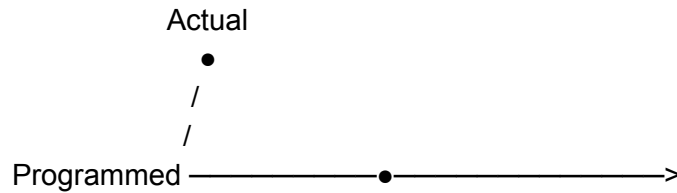


So I would actually call your component **Trajectory Alignment & Synchronization**, rather than simply "Time Synchronization."

One more important distinction for your HMI

There are actually **two different deviations** you might want to show, and you should decide which your client means.

Suppose:



You can calculate:

Spatial/path deviation: shortest distance from actual TCP to the programmed path.

This answers:

"How far did the TCP move away from the programmed geometric path?"

This is what I would use for your **3D panel deviation heatmap**.

But you could also calculate:

Time-synchronized tracking error: difference between actual TCP and where the programmed trajectory says the TCP should be at the same execution time.

This answers:

"At this particular time, how far is the machine from its commanded reference?"

These are **not necessarily the same quantity**.

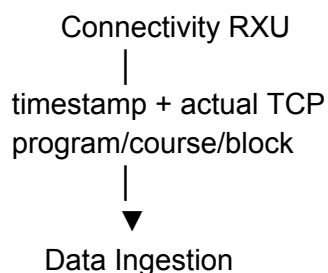
For WP5.3, I would actually preserve both if the available machine data supports them:

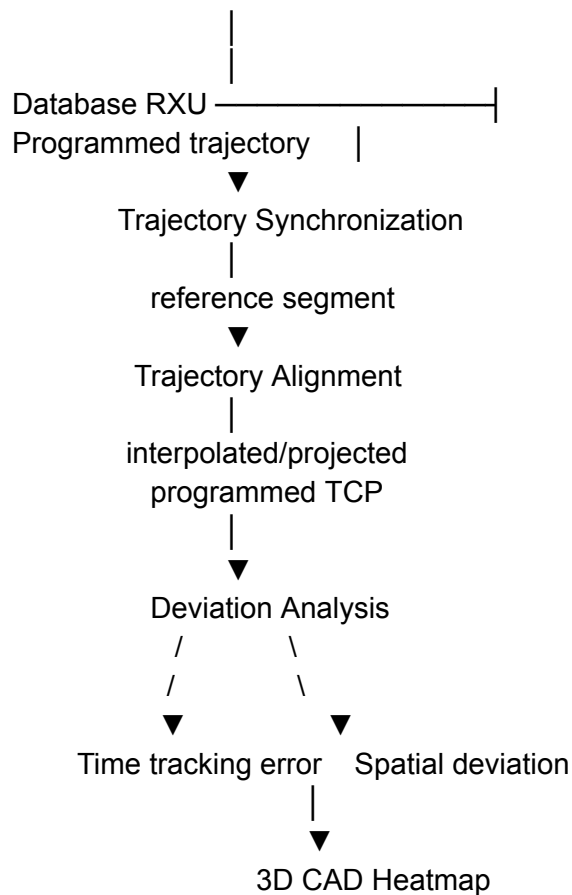
TCP Spatial Deviation
+
TCP Tracking Error

The first is very meaningful for geometry/process quality visualization. The second is meaningful for machine/controller performance.

Recommended implementation for your RXU system

I would therefore design the analysis flow as:





And this design is **identical whether the NC originates as a file or a database table**. Only the RXU component responsible for producing **ProgrammedTrajectory** changes.

For your client-facing **actual vs programmed TCP + deviation heatmap**, I would make **spatial path deviation constrained by `program_id/course_id/block_id`** the primary displayed metric, while keeping time-synchronized tracking error as a separate metric if the controller/connectivity data is sufficient.

We have an orchestrator component which is Behaviour tree
Except hmi front end all are rxu components
Here I am not understanding that how overall sequence works because we are not
controlling any afp states
We are just focusing only on monitoring and analysis
So who can initiate the things and how

Yes. In your case, the key is to stop thinking of the Behaviour Tree orchestrator as an **AFP machine controller**. WP5.3 is a **monitoring/analysis system**, so the orchestrator coordinates the *analysis lifecycle*, while the AFP machine remains controlled externally.

The missing concept is an **external trigger that tells WP5.3 an AFP execution/run has started**.

Separate the two control domains

Your system effectively has:

AFP / MANUFACTURING SYSTEM

Controls:

- machine start/stop
- AFP course execution
- robot/gantry motion
- process execution
- NC execution

|
| status/data only
▼

WP5.3 / RXU

DOES NOT CONTROL AFP

It only:

- observes
- retrieves reference data
- synchronizes
- analyzes
- stores
- visualizes

Therefore your BT should never contain actions such as:

Start AFP
Move machine
Execute NC
Start laying
Stop machine

Instead it contains monitoring-side actions such as:

WaitForRun
LoadProgramReference
InitializeAnalysis
MonitorData
FinalizeRun
GenerateResults

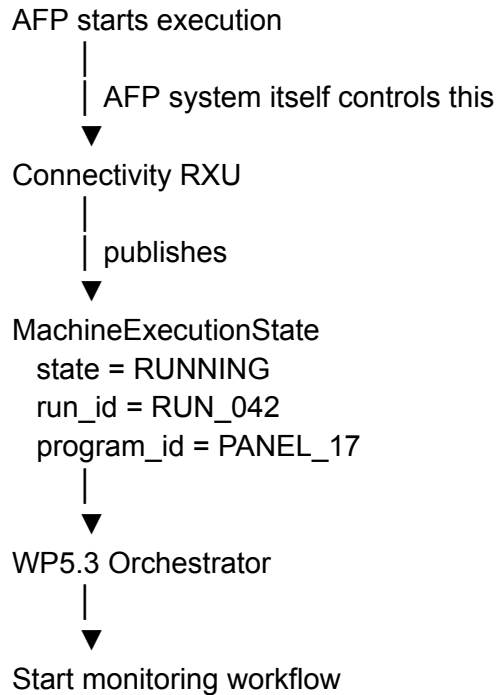
So who initiates WP5.3?

Ideally, **the manufacturing/AFP side produces an execution-state event**, which your Connectivity RXU component exposes to the rest of RXU.

For example, connectivity could publish:

```
message MachineExecutionState {  
    int64 timestamp_ns = 1;  
  
    string machine_id = 2;  
    string run_id = 3;  
    string program_id = 4;  
  
    enum State {  
        IDLE = 0;  
        PROGRAM_LOADED = 1;  
        RUNNING = 2;  
        PAUSED = 3;  
        COMPLETED = 4;  
        ABORTED = 5;  
    }  
  
    State state = 5;  
}
```

Then your WP5.3 system is **event-driven**.



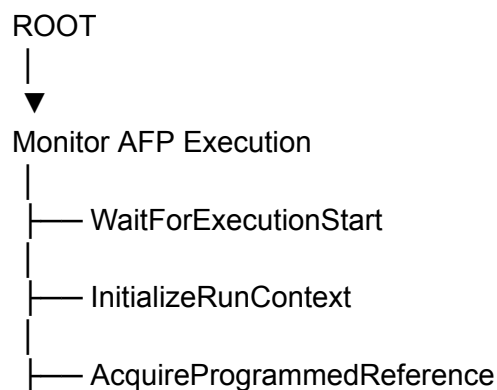
The orchestrator didn't start AFP.

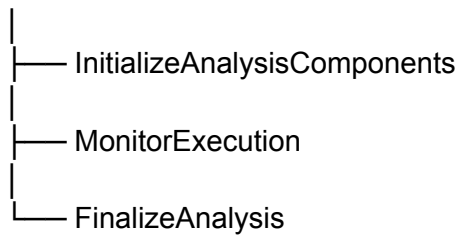
AFP starting caused the WP5.3 monitoring workflow to start.

That distinction is important.

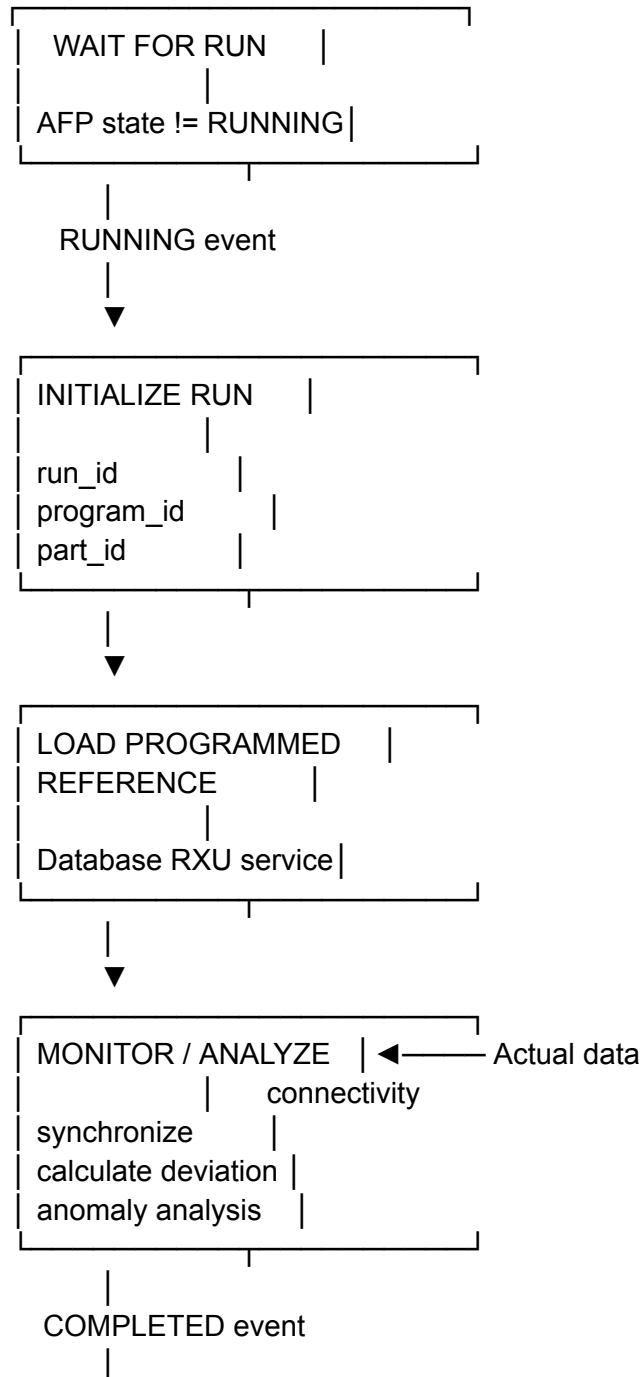
What would the Behaviour Tree actually do?

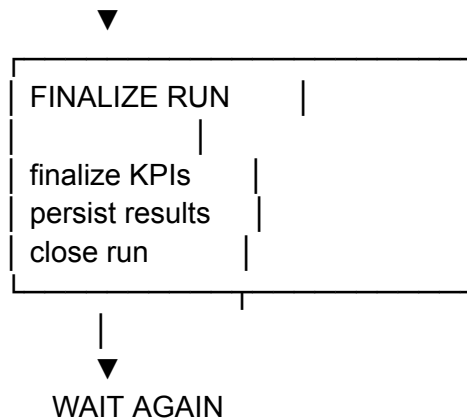
I would make your top-level BT something approximately like:





But because a BT runs continuously, conceptually it behaves like:





That's a perfectly valid use of a Behaviour Tree even though you control **nothing on the AFP machine**.

But there is another important architectural question

Not every RXU component needs to be explicitly started/stopped by the BT.

This is where I think your architecture can become unnecessarily complicated.

Your components can be **long-running reactive RXU applications**.

For example:

Data Ingestion

ALWAYS RUNNING
subscribed to connectivity

Database

ALWAYS RUNNING
waiting for service requests

Data Analysis

ALWAYS RUNNING
waiting for synchronized messages

HMI Backend

ALWAYS RUNNING
waiting for analyzed messages

Orchestrator
ALWAYS RUNNING
observing execution lifecycle

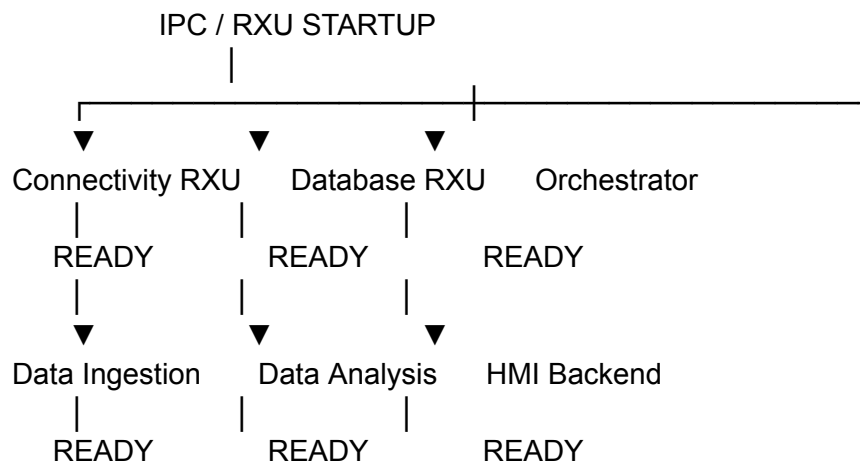
The orchestrator doesn't have to launch processes.

Instead, it manages **logical session/run state**.

This is a much better architecture for your monitoring system.

Example from machine power-up

Imagine everything starts when the industrial IPC starts.



At this point:

Nothing is being manufactured.

That's completely okay.

All components are simply waiting.

Then operator starts AFP externally

Suppose the AFP operator loads:

Panel_A_Program_25.nc

and presses **Cycle Start** on the AFP machine.

Your WP5.3 does not participate in this command.

The AFP system transitions:

```
IDLE
↓
PROGRAM_LOADED
↓
RUNNING
```

Connectivity detects this and publishes:

ExecutionState

```
state    = RUNNING
run_id   = RUN_025
program_id = PANEL_A_25
timestamp = ...
```

The orchestrator reacts

The orchestrator BT receives that information.

Conceptually:

Condition:
IsExecutionRunning?

```
TRUE
|
▼
InitializeMonitoringSession
```

The BT creates/propagates a logical context:

RunContext

```
run_id   = RUN_025
program_id = PANEL_A_25
```

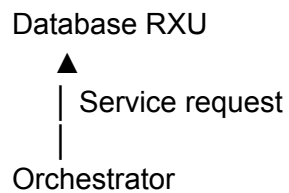
```
part_id = PANEL_A
start_time = ...
```

That `run_id` becomes extremely important.

Everything belonging to this manufacturing execution should carry it.

Then programmed data needs to be loaded

The orchestrator can request:



```
GetProgrammedData(
    program_id = PANEL_A_25
)
```

Now the database RXU implementation decides internally:

Is NC stored as file?

↓

read + parse file

OR

Is NC structured in QuestDB?

↓

query QuestDB

The orchestrator doesn't know or care.

It receives something logically equivalent to:

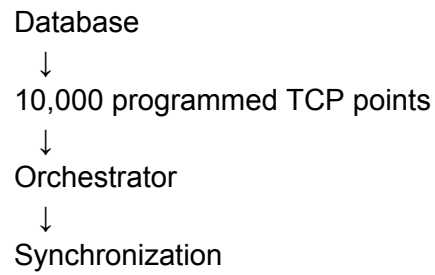
```
ProgrammedTrajectory
    program_id
    courses[]
```

blocks[]
TCP[]
joints[]
feed[]

But should the orchestrator itself receive the huge trajectory?

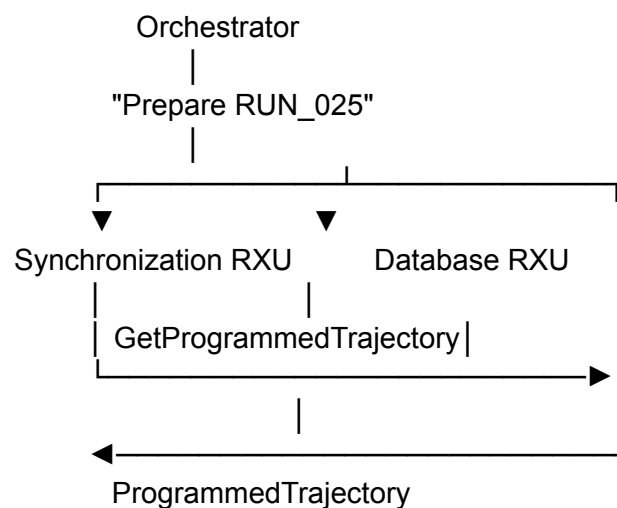
Probably **no**.

I would avoid:



The BT doesn't need to become a data pipeline.

Instead:



The orchestrator coordinates.

The data components exchange data directly.

That's an important architectural principle.

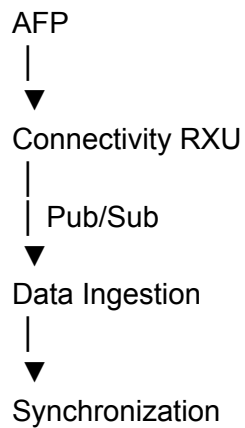
Then actual data starts flowing independently

Connectivity continuously publishes:

ActualMachineData

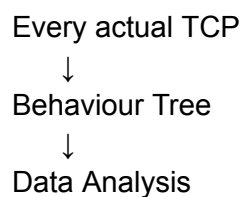
timestamp
run_id
program_id
course_id
block_id
actual_TCP
actual_joints
...

Your ingestion component subscribes:



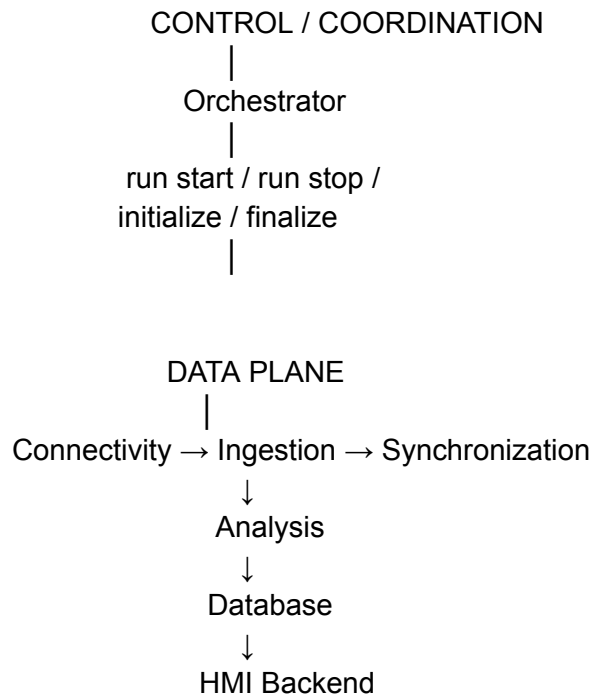
The BT does **not** need to process every message.

It would be a mistake to do:



Don't put the BT in the high-frequency data path.

Instead:

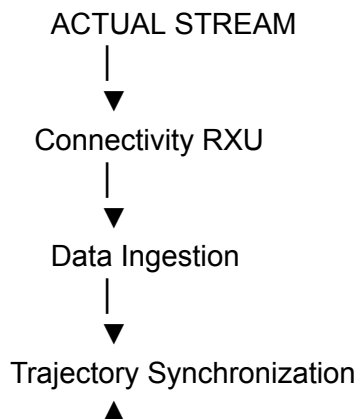


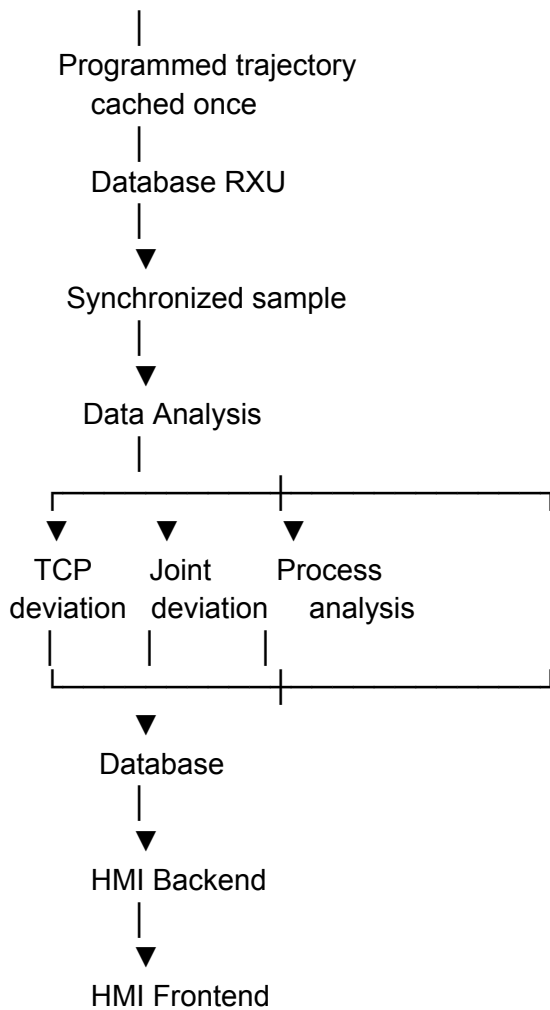
This separation is very useful in your architecture.

During RUNNING

The orchestrator is mostly monitoring lifecycle/state.

Meanwhile your data pipeline works continuously:





No BT interaction is required for each sample.

What happens if AFP pauses?

Connectivity reports:

RUNNING → PAUSED

The orchestrator receives it.

Now the BT could change the monitoring state:

MachineExecutionState == PAUSED



MonitoringSession
PAUSED

But that doesn't necessarily mean shutting down components.

The components stay alive.

It might simply mean:

don't advance trajectory progress
mark samples as paused
don't interpret stationary data as tracking deviation

Then:

PAUSED → RUNNING

continues the same `run_id`.

This also helps solve the time-synchronization problem we discussed earlier.

When AFP completes

Connectivity reports:

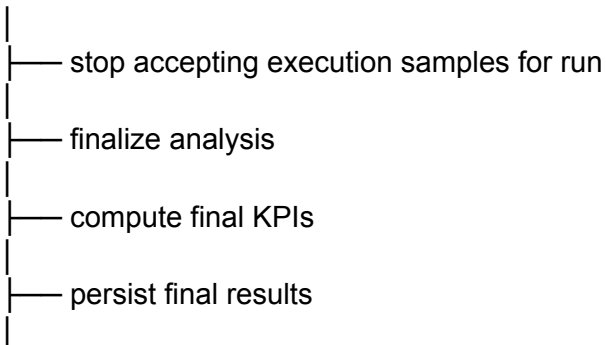
state = COMPLETED
run_id = RUN_025

The orchestrator reacts:

COMPLETED



Finalize RUN_025



└─ mark run COMPLETE

Then:

RUN_025



COMPLETE



Orchestrator returns to
WAIT_FOR_RUN

The HMI can continue displaying RUN_025 from the database.

Where does HMI fit?

You said everything except HMI frontend is RXU. That's fine.

I would keep HMI out of orchestration completely.

RXU

Analysis RXU



HMI Backend RXU



REST/WebSocket



HMI Frontend

The frontend should not initiate the manufacturing analysis workflow.

It can request things such as:

Show RUN_025

Show TCP deviation

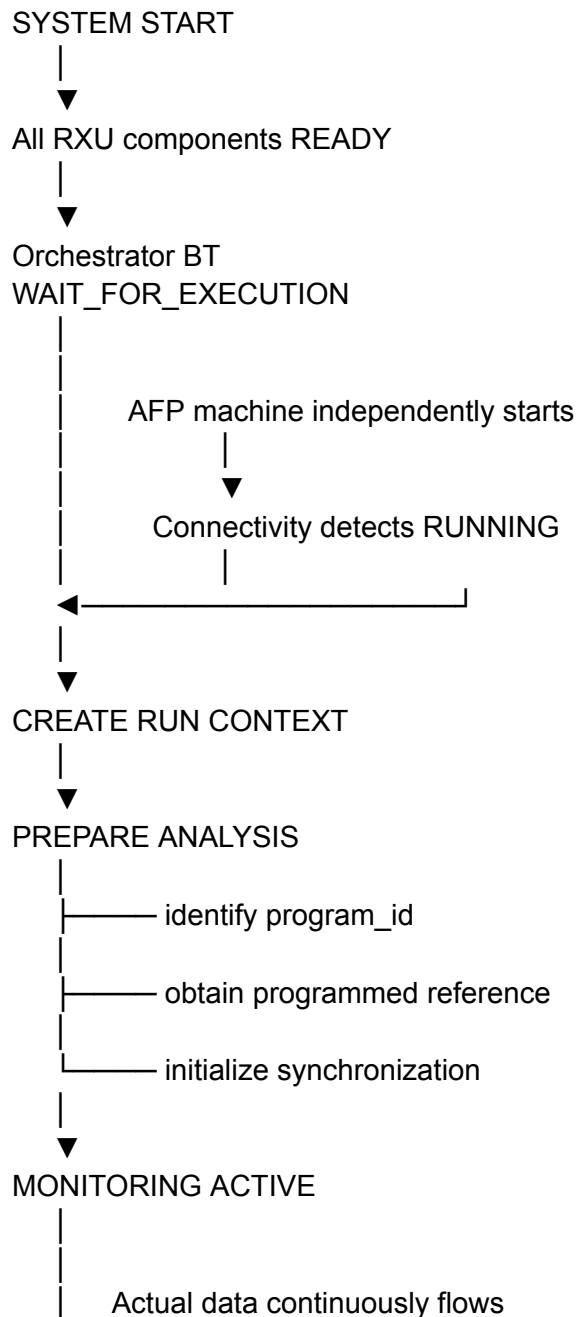
Show Course 7

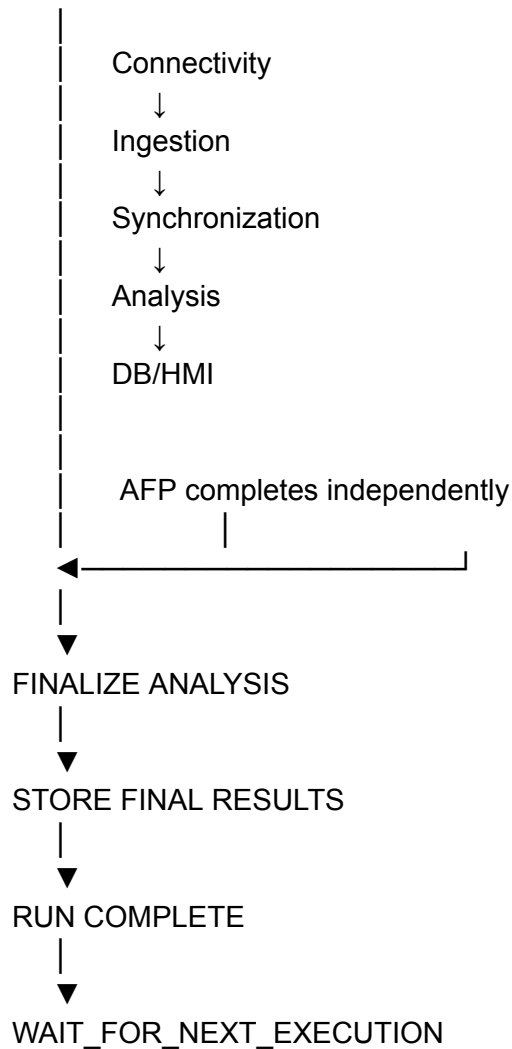
Show programmed vs actual

Show historical run

but those are **visualization queries**, not manufacturing control.

Your complete lifecycle then becomes





So, who initiates everything?

There are actually **three different initiators**, depending on what you mean by "initiate."

The **deployment/system manager** starts the RXU applications when your system starts. The **AFP system/operator** initiates manufacturing; WP5.3 merely observes the resulting execution-state event. The **Behaviour Tree orchestrator** reacts to that event and initiates the *WP5.3 monitoring session*—loading the appropriate reference context, initializing synchronization/analysis, tracking run lifecycle, and finalizing results.

That distinction resolves the architectural confusion:

The BT orchestrates the monitoring workflow, not the manufacturing process.

And I would strongly recommend keeping the BT entirely out of the high-frequency TCP/process-data path. It should coordinate **states and lifecycle**, while RXU pub/sub carries the actual streaming data.

As soon as we start the ipc and user logged into the hmi , do we need to start showing the analysis and spatial deviation etc or some user intervention required? What do you think?

I would not require the user to manually press “Start Analysis” every time. For a monitoring-oriented WP5.3 system, the better design is:

Start the HMI immediately, but start live analysis only when a valid manufacturing run is detected.

So after IPC startup and login, the HMI should first show the system in a **Ready / Waiting for run** state, together with the last available historical information if useful. It should not fabricate or continuously update spatial deviation when there is no active AFP execution.

A good sequence is:

IPC starts



RXU components start



HMI available



User logs in



HMI shows:

System status: READY

Machine state: IDLE

Current run: None

Previous run / historical data can still be viewed

↓

AFP program starts externally

↓

Connectivity reports RUNNING

↓

Orchestrator creates monitoring session

↓

Programmed reference is loaded

↓

Actual data starts arriving

↓

Synchronization becomes valid

↓

Analysis becomes ACTIVE

↓

HMI automatically switches to LIVE view

↓

TCP deviation / process data / spatial heatmap update

So the trigger for live analysis should be the **AFP execution state**, not the HMI login.

The HMI login just gives the user access to the monitoring application.

There is one more subtle point: I would not display spatial deviation immediately at the exact instant **RUNNING** is received. There should be a short internal preparation state because the system first needs enough context to make the calculation meaningful:

RUNNING detected



INITIALIZING



program_id identified



programmed trajectory loaded



coordinate frame/reference validated



actual stream received



trajectory synchronization established



ANALYSIS ACTIVE

During that time the HMI could show something like:

Run: RUN_025

Program: PANEL_A_17

Status:

Preparing analysis...

- ✓ Actual data connected
- ✓ Program reference loaded
- ✓ Frame alignment available

- Synchronization establishing

Then when the synchronizer says it is valid:

Analysis status: LIVE

and the heatmap begins updating.

I would therefore define perhaps four HMI states:

Monitoring state	What HMI shows
IDLE	System ready, no active AFP run
INITIALIZING	Run detected, loading reference/synchronizing
LIVE	Actual vs programmed, deviation, process trends updating
COMPLETED	Finalized results for completed run

PAUSED and ERROR/DEGRADED could additionally be included.

Should the operator have any intervention at all?

Yes, but mostly for **viewing and analysis selection**, not for starting the analysis pipeline.

For example, the user can select:

Live / Historical

TCP deviation

Joint deviation

Compaction force

Tow tension

Temperature

Speed

Anomaly score

Course 1 / Course 2 / ...

Time range

Deviation threshold

But the user should generally **not** have to do:

Select NC file

Press "Start Synchronization"

Press "Start Data Analysis"

Press "Start TCP Deviation"

Those are system responsibilities once an AFP run is recognized.

This is especially important because otherwise your monitoring becomes operator-dependent. If someone forgets to press Start Analysis, you lose the monitoring data for that run.

There is one useful exception

For development, commissioning, or uncertain integration, you can keep a **manual analysis mode**.

For example:

Monitoring Mode

- Automatic
- Manual

Automatic would be the production mode:

AFP RUNNING → automatically start analysis

Manual could allow an engineer to select:

run_id

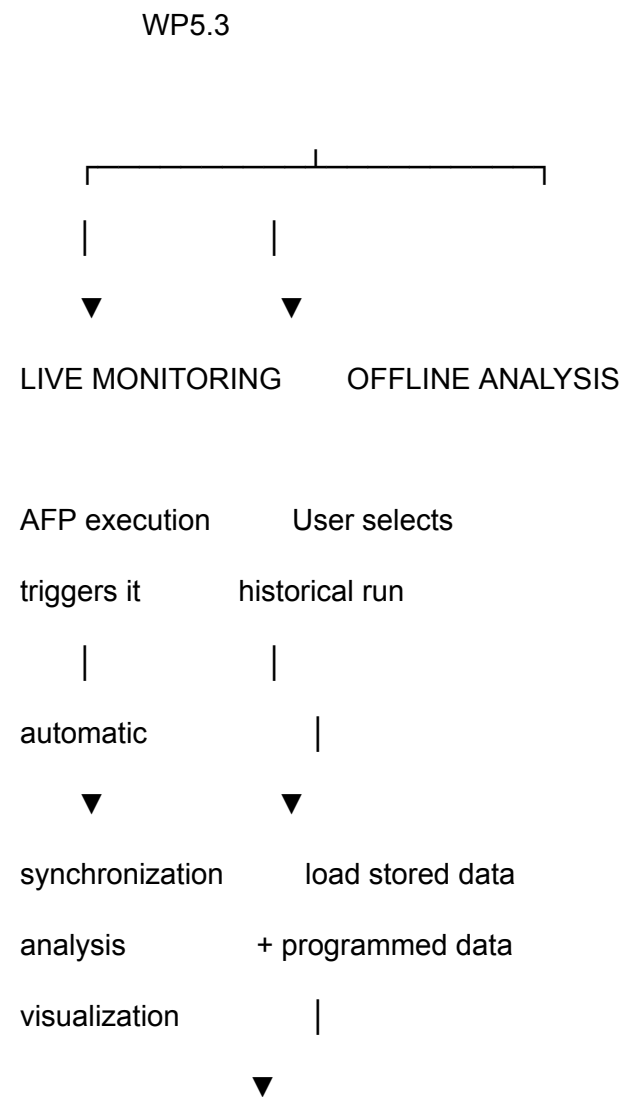
NC program

historical actual data

and run the analysis offline.

That will be very useful for WP5.3 development and validation.

So you actually have two workflows:



analysis

For your architecture, I would make **Automatic Live Monitoring** the normal production behavior and **Manual/Historical Analysis** a secondary HMI feature.

That also gives the Behaviour Tree a clearer purpose: it owns the automatic run lifecycle, while the HMI is mainly a consumer and exploration interface rather than the component that starts the manufacturing analysis.

May be kpp analysis, Pareto dashboard, deviation etc will not same page of hmi i could be different page based on the click

This is not sure

,but this is important in sequence diagram and I am working on it

So how do I do?

You do **not** need to decide the final HMI page layout before making the sequence diagram. The sequence diagram should capture **system interactions and lifecycle**, not whether KPI analysis, Pareto, and TCP deviation end up on one screen or three screens.

For your architecture, I would model the frontend as a single **HMI Frontend** participant in the main sequence diagram, and show the user selecting a view after the data is available. That keeps the sequence stable even if the UI design changes later.

A good main sequence is:

User

|

| Login

▼

HMI Frontend

|

| authenticate / connect

▼

HMI Backend RXU

|

| current system/run status

▼

HMI Frontend

|



AFP starts externally



Connectivity RXU



execution state = RUNNING



Orchestrator BT



initialize run context



Programmed Data RXU



programmed trajectory



Synchronization RXU

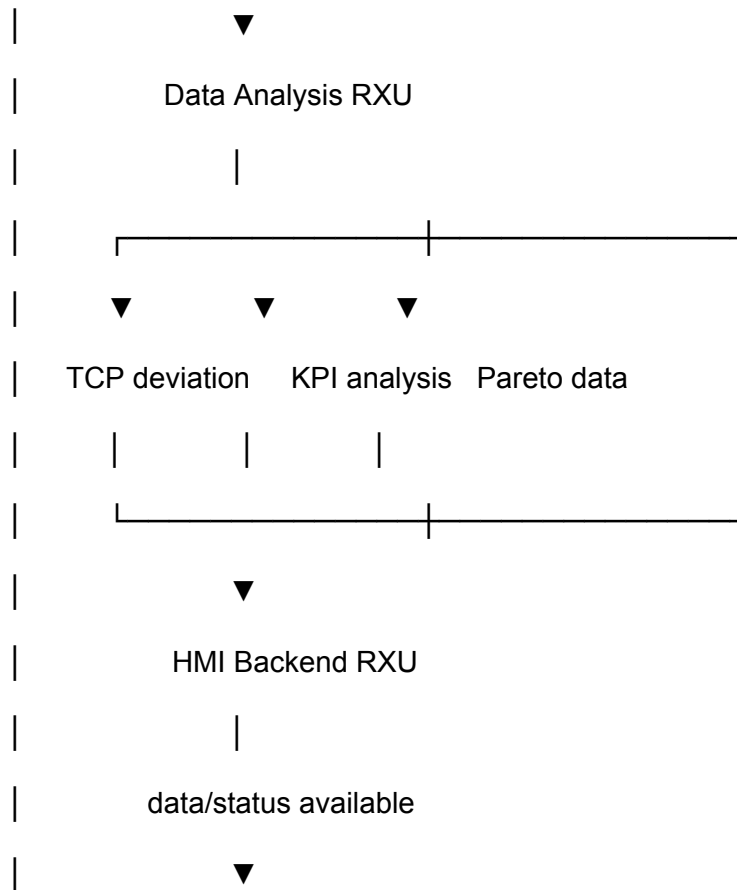


actual process data

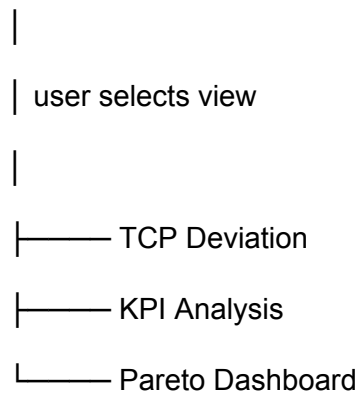


Connectivity RXU





HMI Frontend



The important point is that **analysis does not need to start because the user clicked a page.**

For example, when the run is active, the backend can continue receiving and preparing:

TCP deviation

KPI values

Pareto input/statistics

time-series indicators

even if the user is currently looking only at the TCP page.

Then the click only controls **what is requested/rendered**.

So:

User click $\neq$ Start analysis

User click = Select visualization

That distinction should appear clearly in your sequence diagram.

I would actually use two sequence diagrams

Trying to put everything into one diagram may become crowded.

The first should be the **System Monitoring Lifecycle Sequence**, showing:

AFP run detection

→ orchestrator initialization

→ programmed-data retrieval

→ actual-data ingestion

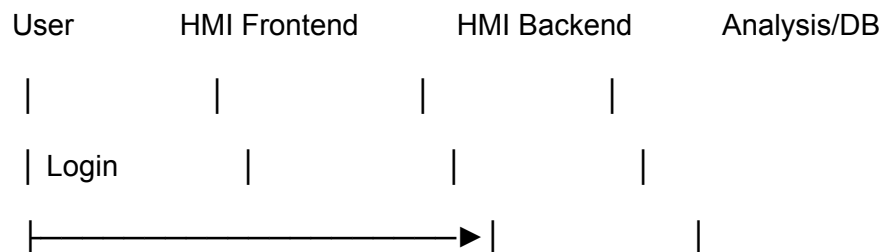
→ synchronization

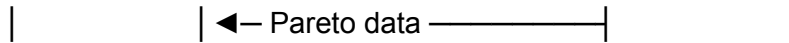
→ analysis

→ storage/HMI backend

This diagram should not care whether the frontend has one page or ten pages.

Then create a second diagram called something like **HMI User Interaction Sequence**:





This allows your UI to later become:

Dashboard

- └─ Overview
- └─ TCP Deviation
- └─ KPI Analysis
- └─ Pareto
- └─ Time-Series
- └─ Historical Run

or:

Single dashboard

with multiple tabs/panels

without changing the system architecture.

One thing I would avoid in the sequence diagram

Don't model it like this:

User clicks "TCP deviation"



Orchestrator



Start synchronization

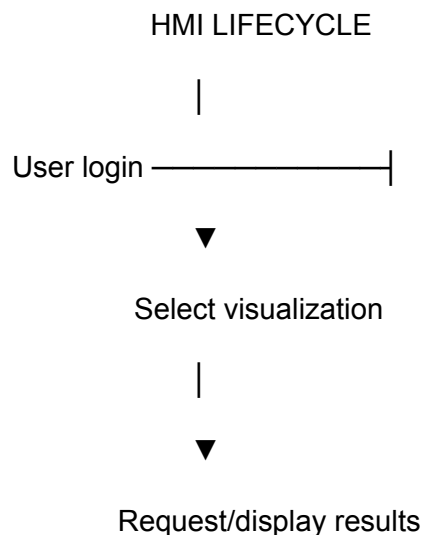
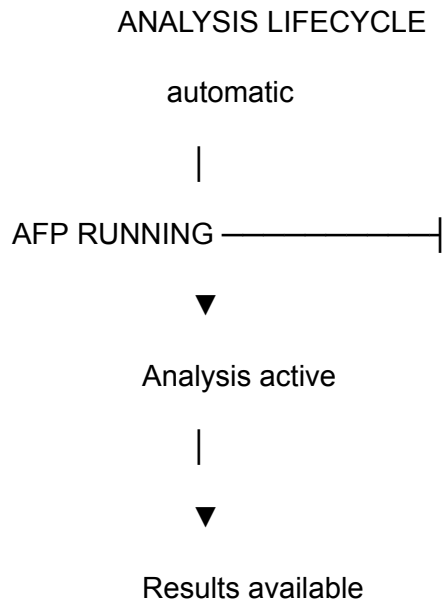


Start analysis

unless your product specifically wants on-demand analysis.

For your live monitoring use case, that would couple the backend processing to the user's navigation. If the user never opens the Pareto page, you could end up not computing or storing Pareto-related information.

A cleaner separation is:



There is also a choice about **when particular analyses are computed**. You do not have to calculate everything continuously. For example, TCP deviation may be a live calculation, KPI aggregation can update periodically, and Pareto analysis may only make sense after enough

data exists or after a course/run completes. You can show that using **alt** or **opt** fragments in the sequence diagram:

alt Live analysis

TCP deviation → continuously during RUNNING

end

opt Periodic KPI update

KPI aggregation → every analysis interval

end

opt Run/course completed

Pareto analysis → calculate/update

end

That would actually make your diagram more accurate than implying all analyses behave identically.

For your current design stage, I would keep **HMI Frontend as one participant**, use **user selection as a visualization action only**, and make the uncertainty about “separate pages vs same dashboard” an HMI design decision outside the main monitoring sequence. This lets you finish the sequence diagram now without waiting for the UI layout to be finalized.

But based on user selection there will be communication between hmi front end, back end and then BT and the corresponding corresponding component

Example user selects kpp analysis or deviation tab

So was thinking corresponding component starts calculating the deviation or analysis only after user clicks and it gets initiated by bt

What do you think

That approach can make sense, but I would not make **every analysis component start only when the user opens a tab**. A better architecture is a hybrid one: the BT can initiate **on-demand analysis** for expensive or user-driven functions, while core monitoring calculations that are needed for traceability should continue independently of the HMI.

For example, if the user clicks **TCP Deviation**, this sequence is reasonable:

User

|

| Select "TCP Deviation"

▼

HMI Frontend

|

| request deviation view

▼

HMI Backend RXU

|

| request / command

▼

Orchestrator BT

|

| "Activate TCP deviation analysis"



Deviation Analysis RXU



| obtain synchronized actual + programmed data

| calculate deviation



HMI Backend



HMI Frontend

Similarly for KPP analysis:

User



| Select "KPP Analysis"



HMI Frontend



HMI Backend



Orchestrator BT



| Activate KPP analysis



KPP Analysis RXU



| retrieve required process data

| calculate KPIs/KPP analysis



HMI Backend



HMI Frontend

So your idea is valid, especially if these analyses are **not required continuously**.

However, I would distinguish between two categories.

Core monitoring calculations

Things such as synchronization, run identification, data ingestion, basic TCP deviation storage, and perhaps basic anomaly flags are often better kept running during an active AFP run.

Why? Because otherwise, if nobody opens the TCP Deviation tab during the run, you may never compute or store deviation for that run.

For example:

AFP RUNNING



Actual data



Synchronization



Basic deviation calculation



Store result

This happens regardless of HMI activity.

Then when the user opens the tab:

User selects TCP Deviation



HMI Backend



retrieve current/history deviation



visualize

This is more robust for traceability.

On-demand analysis

For analysis that is computationally heavier or mainly exploratory, your BT-driven model is a very good fit.

Examples could be:

Pareto analysis

Root-cause investigation

Correlation analysis

Sensitivity analysis

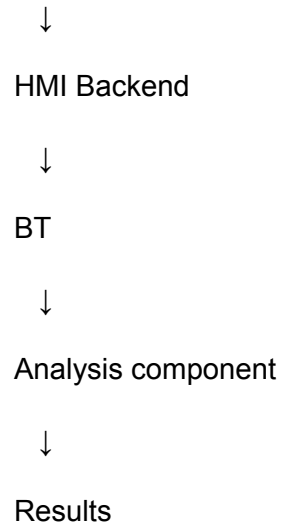
Historical comparison

Detailed KPP analysis

These do not necessarily need to execute continuously.

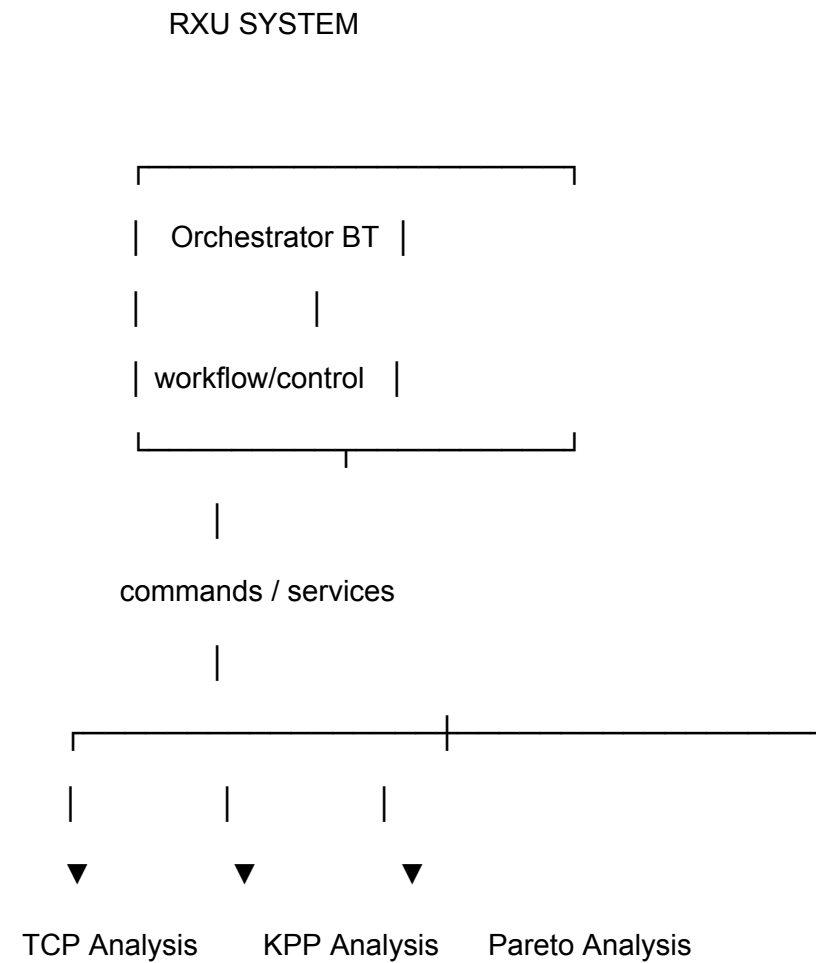
The user can initiate them:

User click

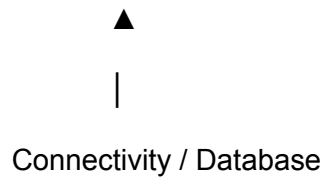
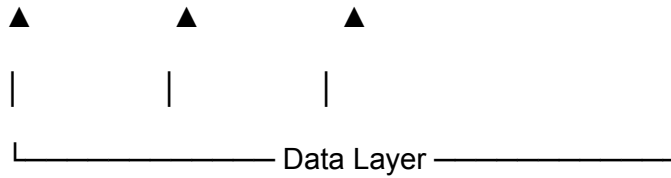


That gives the BT an important role without putting it in the real-time data pipeline.

So I would structure WP5.3 like this:



RXU component RXU component RXU component



Then HMI interaction:

User



HMI Frontend



HMI Backend RXU



| user intent



Orchestrator BT

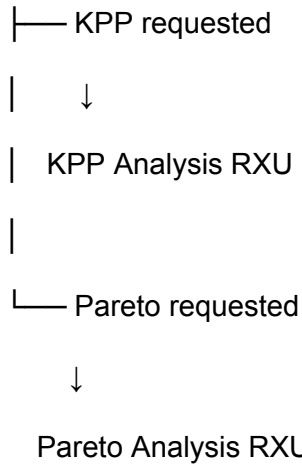


| TCP deviation requested



| TCP Analysis RXU





That is a clean sequence for your architecture.

One important thing, though: the BT should usually **request an analysis operation**, not physically start and stop the RXU executable.

The RXU component can already be alive and waiting.

For example:

KPP Analysis RXU

state = READY

User selects KPP:

BT

|
| StartKppAnalysis(run_id)



KPP Analysis RXU

state = PROCESSING

After completion:

KPP Analysis RXU



AnalysisResult

↓

BT / HMI Backend

So "start" means:

start this analysis job

rather than:

launch the KPP process/application.

That distinction will make your sequence diagram cleaner.

I would also make the BT remember the analysis context. For example, the HMI request should not just say:

Start KPP

It should include something like:

analysis_type = KPP

run_id = RUN_025

course_id = C07

time_range = ...

The BT can then call the correct RXU component.

Conceptually:

User selects KPP

↓

HMI Frontend

↓

HMI Backend

AnalysisRequest

{

```
type    = KPP
run_id  = RUN_025
course_id = C07
}
```



Orchestrator BT



ExecuteKppAnalysis



KPP Analysis RXU

For your sequence diagram, I would therefore show **two mechanisms**:

Automatic monitoring path

AFP Run → Connectivity → Ingestion → Synchronization → basic analysis/store

User-driven analysis path

User → HMI → Backend → BT → requested analysis RXU → result → HMI

That hybrid model is, in my view, the strongest design for WP5.3.

It preserves complete monitoring data even when nobody is watching the dashboard, while still allowing the BT to orchestrate user-selected analytical workflows such as KPP, Pareto, root-cause, or detailed deviation analysis.

These are the front end requirements

Yes. These frontend requirements help a lot, and they actually clarify the question we were discussing about **whether analysis should start automatically or when the user selects something in the HMI**.

From the requirements you shared, I can see several distinct frontend capabilities:

- **Overview/dashboard:** run information, current process state, active KPIs/events.
- **Telemetry/KPI:** KPI summary cards, time-series trends, filtering, comparison, status.
- **3D spatial:** actual TCP trajectory, NC/programmed path, planned-vs-actual, geometry viewer, layer controls, KPI heatmap, KPI overlay selection, TCP point inspection.
- **Quality/anomaly:** anomaly list, timeline, geometry markers, filtering and drill-down.
- **Expected-vs-actual / root-cause:** expected vs actual behavior, deviation visualization, process envelope, root-cause investigation.
- **Historical:** run browser, replay, run comparison.
- **Maintenance:** Pareto dashboard.
- **Multi-tow:** live values for 48 laying-path tows.
- Operator feedback/quality logging.

Looking at these requirements, I would **modify slightly what I suggested earlier**.

The HMI should not start all analysis

There are really three different behaviors hiding behind your frontend requirements.

1. Continuous monitoring — automatic

Some information must exist irrespective of which page the user opens.

For example, your requirements say the overview should show:

current process state, active KPIs and latest events

and you have live telemetry, anomaly information, event logs, etc.

Therefore:

AFP execution

↓

Connectivity



Data ingestion



Basic processing / monitoring



Database

should operate independently of whether somebody has clicked a particular HMI page.

Otherwise, suppose the operator spends 20 minutes on the Overview page and then clicks Anomaly. You don't want the system to say:

"Anomaly analysis started now."

You normally want the anomalies from the previous 20 minutes to already exist.

2. View-specific data preparation — triggered by HMI

This is where your idea about the BT becomes very relevant.

Suppose the user clicks:

3D → Planned vs Actual / TCP Deviation

The sequence can be:

User



| Select "TCP Deviation"



HMI Frontend



| Request TCP deviation view



HMI Backend RXU



| AnalysisRequest



Orchestrator BT



|— Check active/historical Run ID



|— Check required data available



|— Request programmed trajectory



|— Trigger deviation workflow



Deviation Analysis RXU



Actual TCP



Programmed TCP

Connectivity/DB

DB RXU



Synchronization



Spatial deviation



Visualization data



HMI Backend



HMI Frontend



3D deviation view

This is a good use of the BT.

But I would describe this as triggering the **deviation-analysis workflow**, not necessarily starting the executable itself.

3. Heavy/investigative analysis — definitely on demand

Your requirements make this even clearer for things like:

- root-cause investigation;
- historical replay;
- run comparison;
- Pareto analysis;

- possibly expected-vs-actual/model analysis.

These are excellent candidates for:

User selection



HMI



Backend



BT



Corresponding RXU analysis component

For example, your [PT_HMI_FE-43](#) Maintenance Pareto Dashboard requirement says the frontend should display a Maintenance Pareto Dashboard with translated streaming/pollled data from a Backend API and allow filtering by time range, severity, and component.

So:

User clicks

"Maintenance Pareto"



Frontend



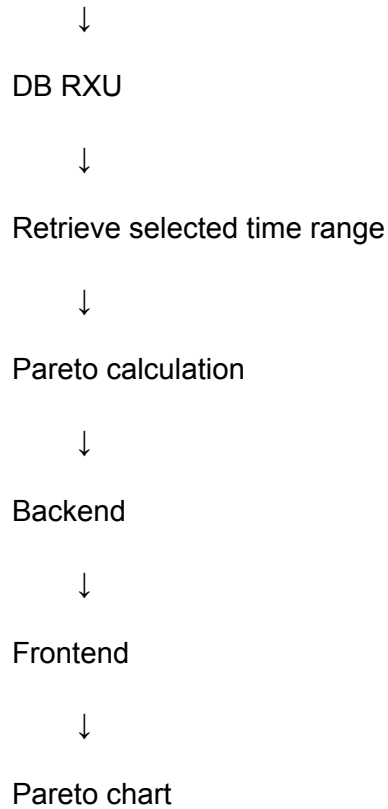
Backend



BT



Maintenance Analysis RXU



That is quite reasonable.

I would therefore classify your requirements

This will make your sequence diagrams much easier.

Frontend capability	Backend behaviour	Trigger
Overview	Current state/status	Automatic
Process state/timeline	Continuous event/state collection	Automatic

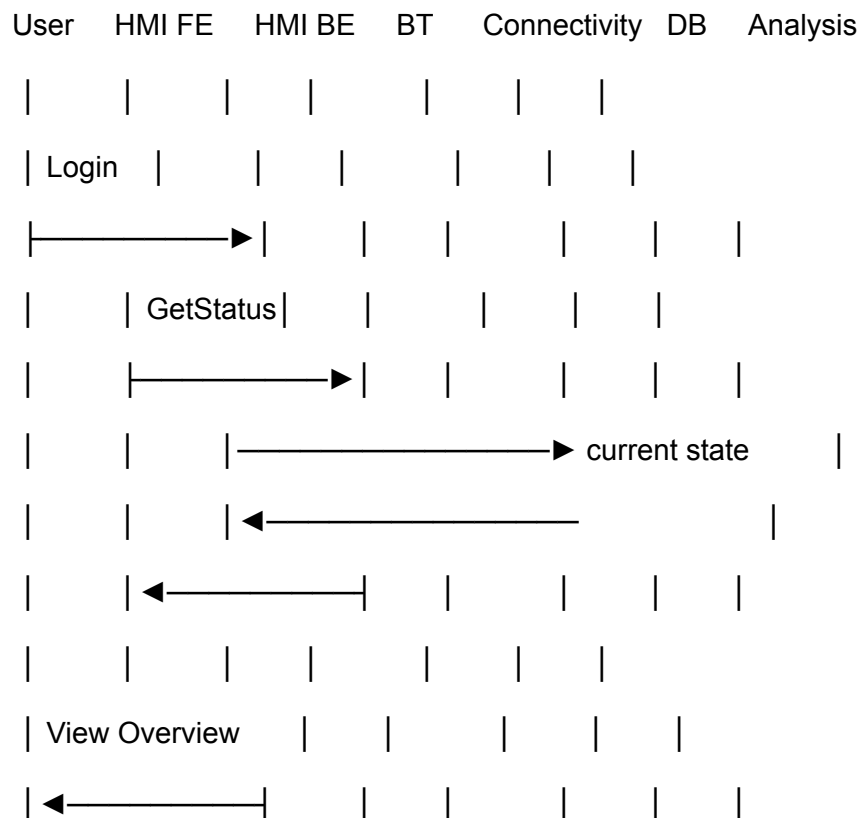
Live KPI cards	Continuous/periodic	Automatic
Live tow telemetry	Continuous	Automatic
Event/anomaly collection	Continuous	Automatic
KPI trend display	Query prepared/stored data	User view/filter
TCP actual trajectory	Query/live subscription	User view
Programmed NC path	Retrieve reference	User view
Planned vs Actual	Deviation workflow	User/BT
KPI heatmap	Spatial mapping workflow	User/BT
Expected vs Actual	Model analysis workflow	User/BT
Root Cause	Investigation workflow	User/BT
Historical replay	Historical data workflow	User/BT
Run comparison	Comparison workflow	User/BT
Pareto	Pareto analysis workflow	User/BT

This is the separation I would use.

Your main sequence diagram should start from login

Now that I have seen the frontend requirements, I think you can make a fairly concrete **HMI interaction sequence**.

Something like:



At this point:

AFP idle:

"Waiting for active run"

or

AFP running:

RUN_025

Course 07

State: LAYING

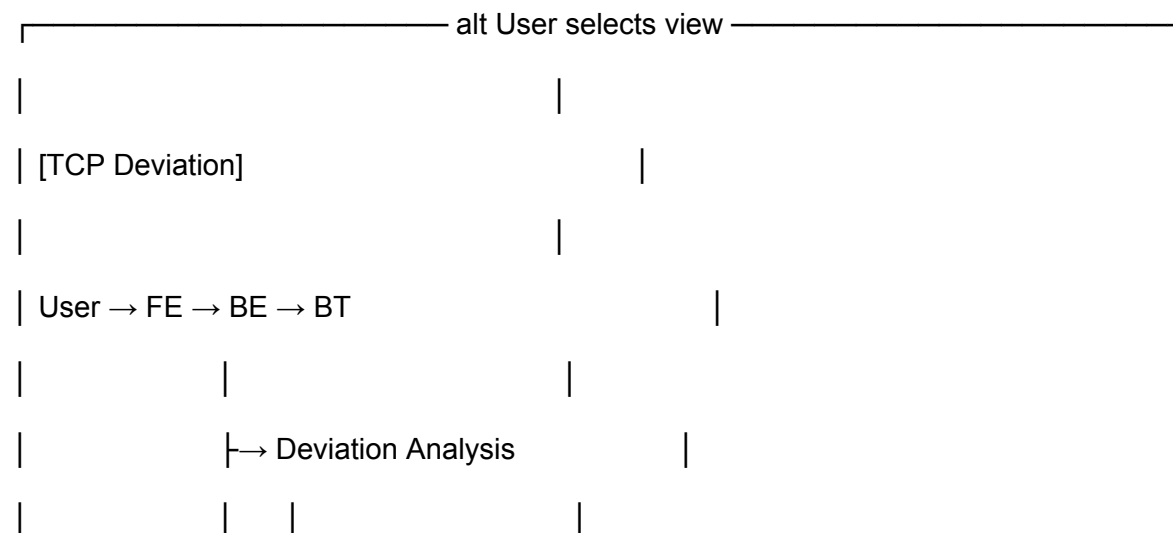
Current KPIs...

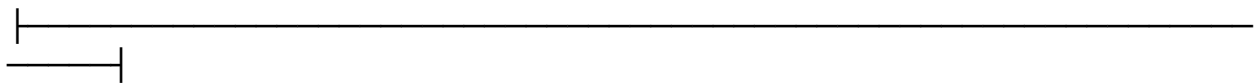
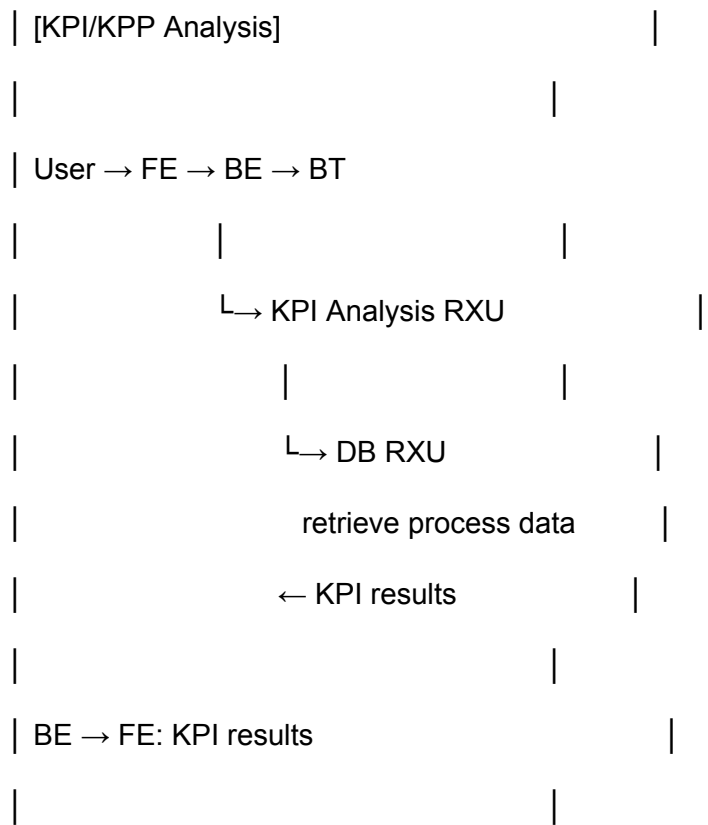
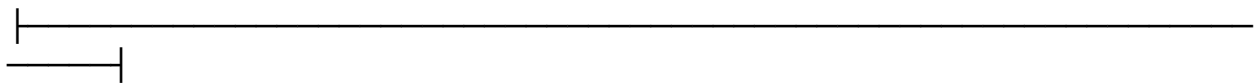
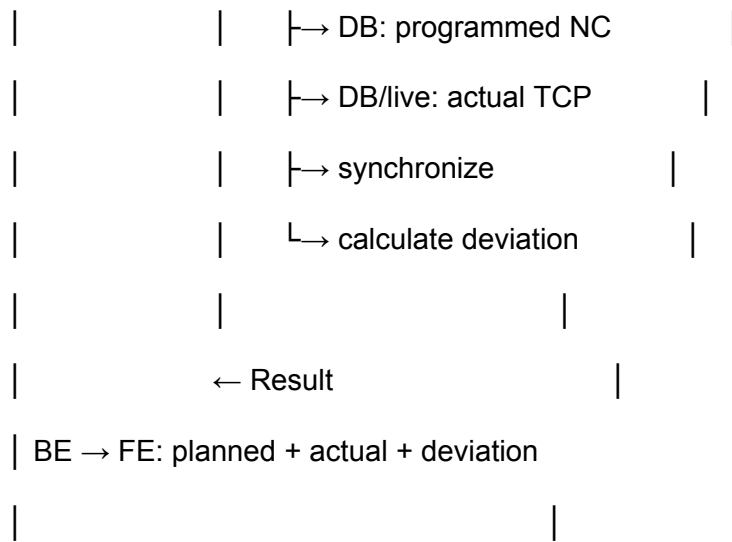
No analysis page needs to be selected yet.

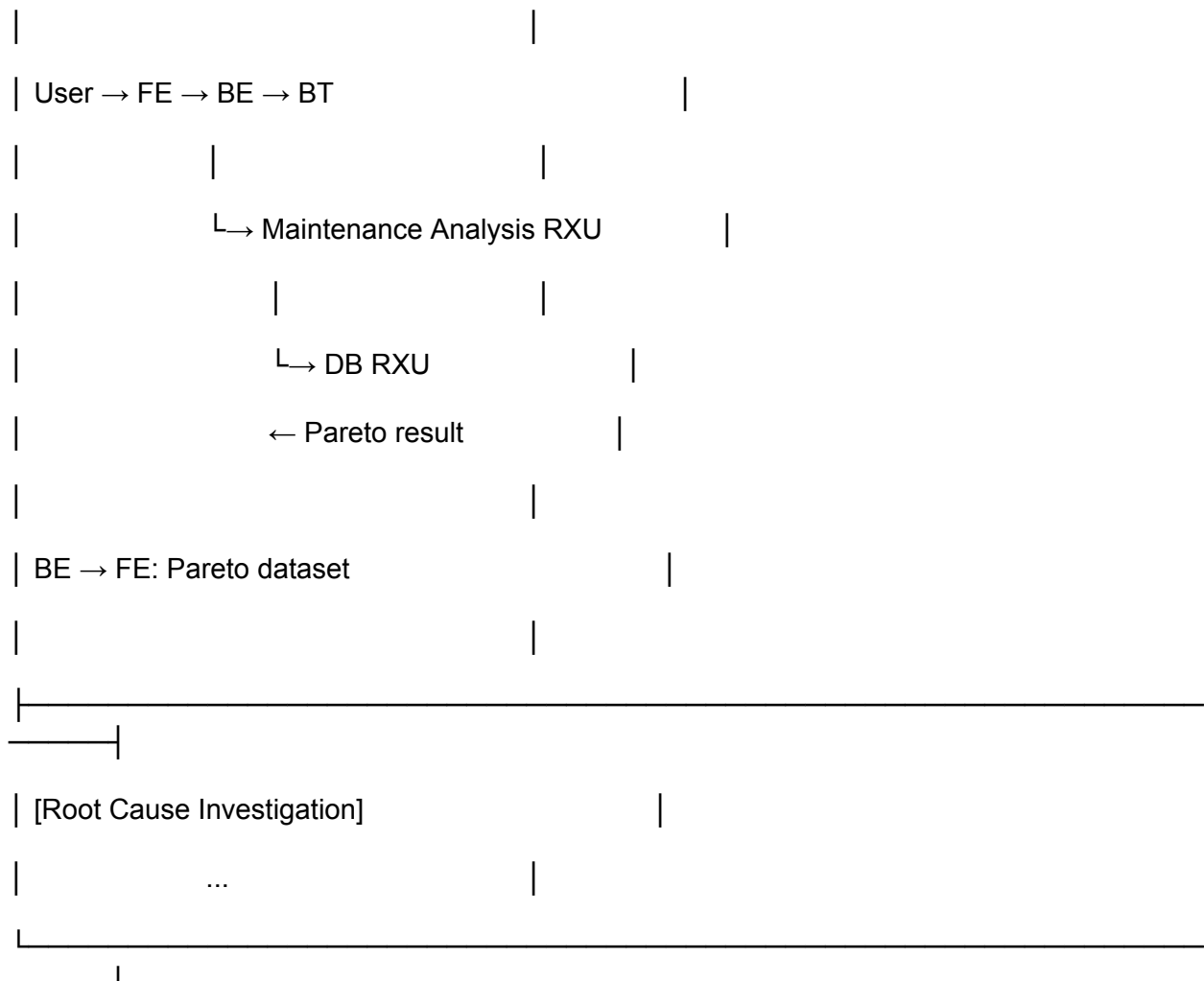
Then use **alt** fragments for the user's selection

This is probably the best way to solve your sequence-diagram problem.

After the initial dashboard sequence:







You don't need to know whether those are **tabs**, **pages**, **buttons**, or **menu items**.

Your sequence diagram simply says:

User selects analysis/view.

That's enough for architecture.

There is one thing I would change in your current thinking

I wouldn't make:

User clicks TCP deviation



Start collecting actual TCP

because that creates a problem.

Actual TCP should already have been collected/stored during the manufacturing run.

Instead:

AFP RUNNING

Connectivity —► DB



├— actual TCP

├— joints

├— process parameters

└— states/events

continuously

Then:

User clicks TCP deviation



BT triggers deviation workflow



Get actual data + programmed data



Synchronize



Calculate



Display

For a live view, once the workflow is activated, the component can additionally subscribe to the incoming stream:

Historical/current data from DB

+

new actual samples from pub/sub



Deviation Analysis



HMI

This gives you both initial history and continuous live updates.

And the BT becomes meaningful

With this architecture, your Behaviour Tree isn't artificially inserted between every component.

It handles **workflow decisions**.

For example:

TCP Deviation requested



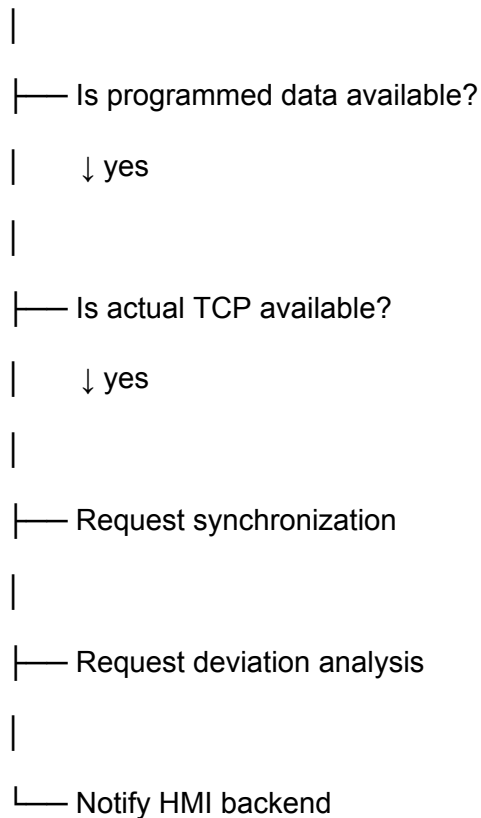
BT



Is valid run selected?



↓ yes



Whereas the high-frequency stream remains:

Connectivity —pub/sub—► Ingestion/DB

without passing every TCP point through the BT.

One more recommendation based on your frontend requirements

I would **not make one enormous sequence diagram containing FE-01 through FE-46.**

You will end up with something unreadable.

I'd create approximately four runtime sequence views:

SD-01 — System Startup & Live Monitoring

IPC → RXU startup → HMI login → status →

AFP run detected → monitoring active

SD-02 — User-Requested Analysis

This is the important one for your current question:

User → FE → BE → BT → Analysis component → DB/data → result → HMI

Use **alt** for TCP deviation / KPP / Pareto / etc.

SD-03 — Live Data & Anomaly Monitoring

Connectivity → ingestion → analysis/anomaly →

DB/backend → frontend

SD-04 — Historical Analysis / Replay

User → FE → BE → BT →

DB → analysis → historical result → FE

That structure will remain valid even if next month somebody decides:

"KPI and deviation should be tabs on one page"

or:

"No, make them completely separate pages."

The frontend navigation design changes, but your RXU architecture and sequence diagrams don't need to change.

And from the requirements you've shown, I think your earlier intuition was partly correct: **the HMI can trigger specific analytical workflows through Backend → BT → corresponding RXU component**, while the underlying acquisition/monitoring must remain independent of which HMI view happens to be open.

i was thinking that Analysis component which calculates the deviation will continuously calculate and publish over a topic irrespective user clicks 3d planned vs actual deviation

BT might pass this info to HMI backend only once the user clicks

or BT can stream it continuously and HMI can store info?

what is your opinion?

I prefer your first idea, with one refinement:

Let the deviation-analysis RXU component calculate continuously during an active run and publish continuously. Do not route the high-rate stream through the Behaviour Tree. Let the HMI Backend subscribe/store the latest data continuously, and let the user click only control whether that data is sent/rendered to the frontend.

That gives you the cleanest separation.

DATA PLANE

Connectivity RXU

|



Synchronization

|



Deviation Analysis RXU

|

| continuous pub/sub



HMI Backend RXU

|

| — keep latest/live buffer

| — optionally persist via DB RXU

CONTROL PLANE

User

|

| selects "Planned vs Actual"

▼

HMI Frontend

|

▼

HMI Backend

|

| view request

▼

BT / Orchestrator

|

| authorize/configure/select workflow if needed

▼

HMI Backend

|

| start forwarding requested representation



HMI Frontend

The key point is:

BT should not forward every deviation sample.

If deviation is coming at, say, 50 Hz or 100 Hz, doing this would be poor architecture:

Deviation Analysis



BT



HMI Backend



Frontend

for every message.

The BT is better for orchestration and decisions, not high-frequency data transport.

Why continuous deviation calculation is better

Suppose the AFP run lasts 30 minutes, but the user opens the 3D deviation page only after 20 minutes.

If deviation calculation starts only when the user clicks, you either lose the first 20 minutes or must recalculate it retrospectively.

With continuous calculation:

Run starts



Deviation continuously calculated

↓

Results continuously available

↓

User opens page at any point

↓

Immediate visualization

It also means anomaly detection, historical analysis, KPI correlation, and later root-cause analysis can use the same deviation data even if nobody opened the 3D page.

Should HMI Backend store the continuous stream?

I would distinguish **buffering** from **persistent storage**.

The HMI Backend should absolutely maintain a small live buffer/cache, for example the latest few seconds or minutes:

HMI Backend

latest deviation sample

latest actual TCP

latest programmed TCP

recent trajectory window

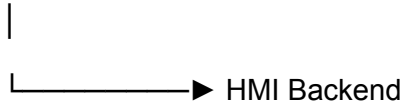
latest KPI values

But I would avoid making the HMI Backend your long-term data store.

Since you already have a Database RXU abstraction over QuestDB, preferably:

Deviation Analysis





live subscription

or potentially another storage subscriber consumes the same topic.

Then responsibilities stay clean:

Database RXU

= persistence / historical data

HMI Backend

= API, live cache, frontend delivery

HMI Frontend

= visualization

Then what happens when the user clicks the deviation tab?

I would make it very lightweight.

For example:

User clicks

"Planned vs Actual"

|



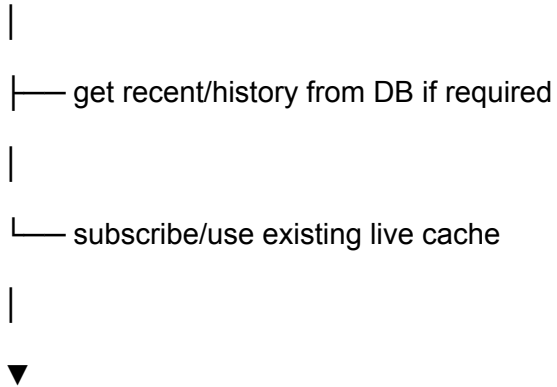
HMI Frontend

|

| OpenDeviationView(run_id)



HMI Backend



Frontend receives

actual TCP

programmed TCP

deviation



3D visualization

You may not even need the BT for the simple act of opening a visualization page.

That is worth emphasizing.

I would use BT only when there is an actual workflow to orchestrate

For example, BT makes sense for:

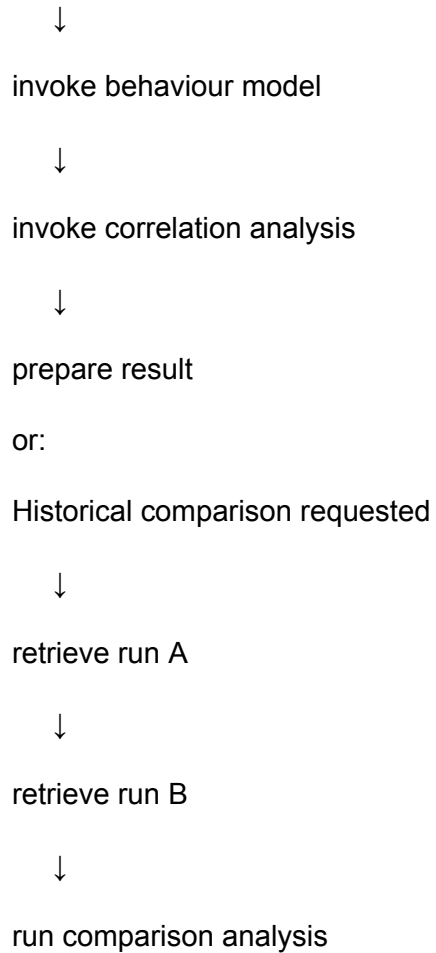
Root-cause analysis requested



validate run



retrieve datasets



For simply:

"Show me the deviation values that are already being calculated"

routing:

Frontend → Backend → BT → Backend

may be unnecessary.

The HMI Backend can directly serve that data.

So I would divide your HMI interactions into two types

User action

**BT
needed?**

Reason

Open live TCP deviation view	Usually no	Data already available
Open KPI trend	Usually no	Query/display existing data
Change time range/filter	Usually no	Backend/database query
Select heatmap variable	Usually no	Visualization/data-selection operation
Start root-cause investigation	Yes	Multi-step analytical workflow
Run historical comparison	Yes	Requires coordinated analysis
Execute KPP analysis on selected dataset	Possibly yes	Analysis job/workflow
Generate Pareto from selected conditions	Possibly yes	If computation is on demand

This is, in my opinion, cleaner than forcing every frontend request through the Behaviour Tree.

For your specific deviation case

I would implement:

ACTIVE AFP RUN

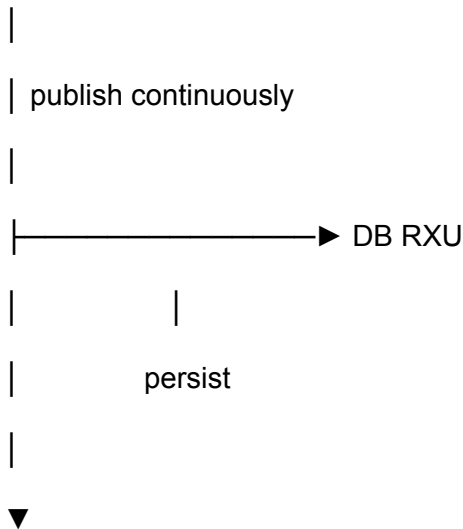
Connectivity



Synchronization



Deviation Analysis



HMI Backend

live cache

USER OPENS 3D VIEW

User



Frontend



Backend



send:

programmed path

actual path

deviation



Frontend 3D Viewer

The frontend could then receive ongoing updates through WebSocket or your chosen frontend-facing streaming mechanism while that page is active.

When the user leaves the page:

Frontend stops requesting/rendering deviation stream

but:

Deviation Analysis continues

Database continues storing

That is exactly what I would choose for your WP5.3 architecture.

So, between the two options you proposed, I would **not have BT continuously stream the deviation and I would not use BT as a message relay**. Keep continuous analysis and streaming in the RXU pub/sub data plane; use the HMI Backend as the live-data gateway/cache, the Database RXU for persistence, and reserve the BT for genuine orchestration workflows.

CONTROL PLANE

User

├

├ selects "Planned vs Actual"

└

HMI Frontend

├

└

HMI Backend

├

├ view request

└

BT / Orchestrator

├

├ authorize/configure/select workflow if needed

└

HMI Backend

├

├ start forwarding requested representation

└

can yiu explain this agian? i didn't understand

also what do you meant by this?

HMI Backend | | — get recent/history from DB if required | |
subscribe/use existing live cache

because deviation component is calculated by RXU App(analysis)

Yes — your confusion is valid because my earlier diagram mixed **control flow** and **data flow** too much.

In your architecture, the **Analysis RXU App is the component that calculates deviation**. The HMI Backend does not calculate it. The HMI Backend only receives, buffers, retrieves, and sends deviation results to the frontend.

So the cleaner architecture is:

DATA FLOW

Actual TCP + Programmed TCP

|

▼

Analysis RXU App

|

| calculates deviation continuously

|

| —————▶ Database RXU

| store deviation

|

| —————▶ HMI Backend RXU

receive live deviation

Now separately:

USER / VIEW FLOW

User clicks

"Planned vs Actual"

|



HMI Frontend

|

| request deviation view



HMI Backend

|



returns already-available deviation data

|



HMI Frontend displays 3D view

So for simply opening the deviation page, **BT may not be required at all.**

That is the main correction.

The HMI Backend can already be subscribed to the deviation topic published by the Analysis RXU App:

Analysis RXU App

|

- | publish:
- | Actual TCP
- | Programmed TCP
- | Deviation



HMI Backend RXU

The backend can keep the latest samples in memory.

For example:

HMI Backend live buffer

sample 1001

sample 1002

sample 1003

...

sample 1500

Then when the user opens the 3D page, the backend sends those samples immediately.

When I said:

“get recent/history from DB if required”

I meant this situation:

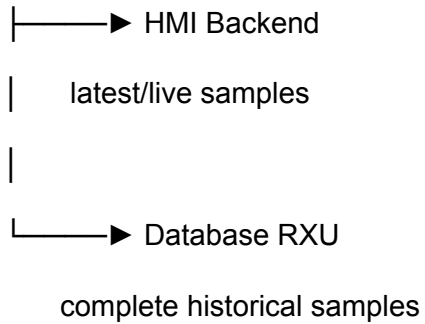
Suppose the user opens the deviation page **15 minutes after the AFP run started**.

The HMI Backend may only keep, say, the latest 30 seconds or 2 minutes in memory. It does not necessarily keep all 15 minutes.

So:

Analysis RXU App

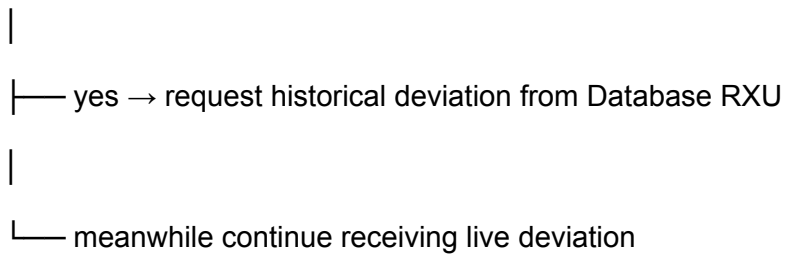
|



Then the user opens the page.

The HMI Backend can do:

Need previous 15 minutes?



So the frontend can receive:

historical deviation

+

current live deviation

and show the complete trajectory.

The HMI Backend is not calculating anything.

It is just combining access to:

live results → from Analysis RXU topic

historical results → from Database RXU

That is what I meant.

A concrete sequence may help.

Suppose AFP starts at 10:00.

From 10:00 onward:

Connectivity RXU

|



Analysis RXU

|

| calculate continuously



DeviationResult

10:00:01 → 0.8 mm

10:00:02 → 1.1 mm

10:00:03 → 0.7 mm

...

Analysis publishes every result.

Both of these can consume it:

Analysis RXU

|

DeviationResult

/ \

/ \



Database RXU HMI Backend

persist live cache

Now at 10:10 the user clicks:

"Planned vs Actual"

Then:

User

|



HMI Frontend

|

| Give me deviation data for RUN_25



HMI Backend

|

|— recent/live samples already in memory

|

|— older samples requested from Database RXU

|



Database RXU

Backend then sends:

10:00 → 10:10 historical deviation

+

10:10 onwards live deviation

to the frontend.

This gives the user a complete 3D trajectory.

Now regarding the BT.

For this simple case:

User clicks deviation tab

I would not use:

Frontend



Backend



BT



Analysis

because the Analysis RXU is already running and calculating deviation.

There is nothing for the BT to initiate.

The cleaner sequence is:

User



| selects Planned vs Actual



HMI Frontend



| request view data



HMI Backend

|

| —————▶ Database RXU

| retrieve history if needed

|

| —————▶ already receives live Analysis topic

|

▼

HMI Frontend

The BT is unnecessary here.

The BT becomes useful when the user requests something that really needs a **workflow to be executed**.

For example:

User clicks

"Root Cause Investigation"

|

▼

HMI Frontend

|

▼

HMI Backend

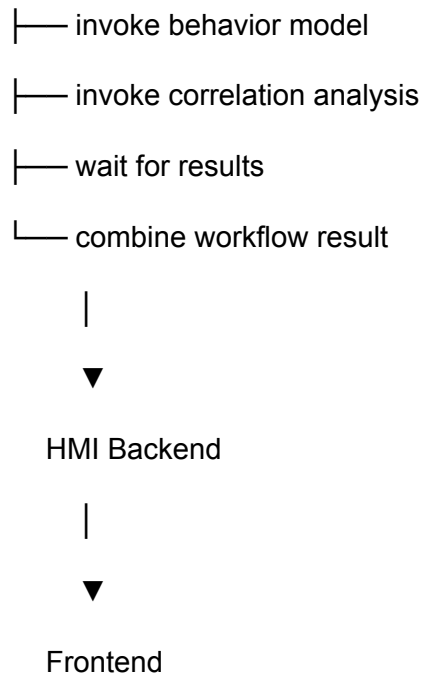
|

▼

Orchestrator BT

|

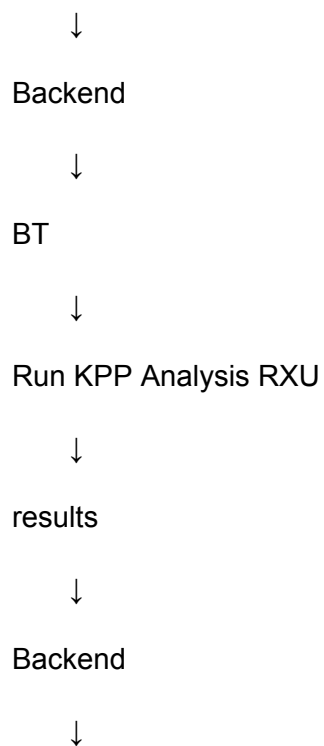
| — request required historical dataset



Here BT makes sense because there are multiple coordinated steps.

Similarly, perhaps for a KPP analysis that is not continuously computed:

User selects KPP Analysis



Frontend

But for continuously computed TCP deviation:

Analysis RXU continuously computes it

there is no reason to tell the Analysis App:

"Start deviation calculation because user opened the tab."

It is already doing it.

So I would define your system like this:

Function	Who does it
Actual data acquisition	Connectivity RXU
Programmed/reference data	Database/NC RXU
Synchronization	Synchronization/Data Analysis RXU
TCP deviation calculation	Analysis RXU
Continuous deviation publication	Analysis RXU
Long-term result storage	Database RXU
Latest/live deviation cache	HMI Backend RXU
Send deviation to browser	HMI Backend

Render 3D trajectory/heatmap HMI Frontend

Complex workflow orchestration Behaviour Tree

And one more important point: the HMI Backend does **not necessarily have to subscribe only after the user clicks**.

I would probably keep the HMI Backend subscribed continuously:

Analysis RXU

|

| continuous pub/sub



HMI Backend

It can simply decide:

No user viewing deviation page

→ receive/cache data, don't send to browser

User viewing deviation page

→ receive/cache data + forward updates to browser

That is simpler and faster.

So the final clean picture for your TCP deviation use case is:

AFP RUNNING

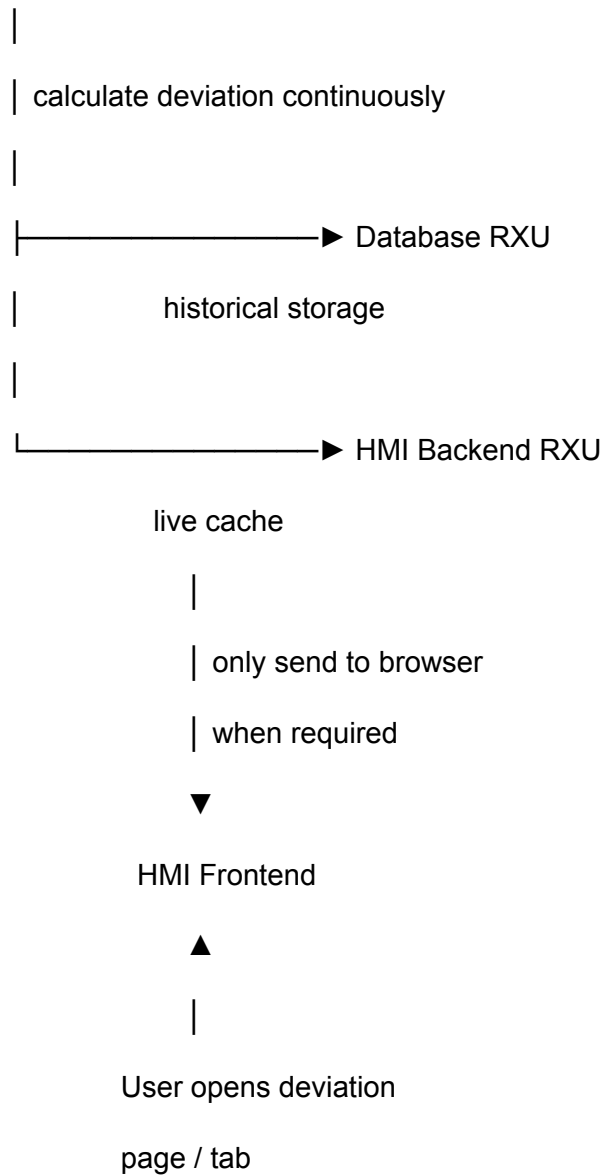
Connectivity RXU

|



Synchronization /

Analysis RXU



This is the architecture I would recommend for the requirements you showed. The **user click controls visualization, not deviation computation.**

now based on the front end requirement

can you give detailed documenttaion on how architecture looks like?

also a sequence diagram to explain how in each sequence is? whenever a user clicks or not clicks

i am looking for detailed one

also make sure that we are covering three important calculation

1. Tcp and joint deviation - Variation should be shown on 3D CAD geoemry
2. KPI trends (not anomaly) - 3D CAD geoemtry
3. Process anomaly (any variation in KPI using time series analysis) - show on grafana dashboard or any other better options

in 1st and 2nd case probably user should have an option to see both, or only one at a time (filter)

also filter based on IDs such as tow id, course id

Yes. Based on the frontend requirements you shared and the RXU/QuestDB/BT architecture we have discussed, I would structure WP5.3 as a **monitoring-and-analysis architecture with two clearly separated planes**:

1. **Continuous data/analysis plane** — acquisition, synchronization, deviation calculation, KPI preparation, time-series anomaly detection, storage.
2. **User/control plane** — HMI navigation, filtering, requesting a particular visualization or investigation, and BT orchestration only where an actual workflow must be coordinated.

The important architectural principle is:

The analysis does not depend on whether the user is currently looking at a page. The user click determines what is retrieved, filtered, streamed, and visualized.

That is particularly important for your three primary functions: TCP/joint deviation, KPI spatial trends, and process anomalies.

WP5.3 Process Twin & HMI — Proposed Detailed Architecture

1. Architectural objective

WP5.3 shall provide monitoring, analysis, spatial visualization and engineering investigation capabilities for the AFP process without controlling the AFP machine or changing the manufacturing process.

The AFP machine and its native control system remain responsible for:

NC execution

machine motion

layup execution

process start / stop

machine pause / resume

machine operating states

WP5.3 observes the process through the RXU connectivity layer.

The core flow is therefore:

AFP MACHINE

|

| machine/process information

▼

Connectivity RXU

|

▼

WP5.3 Monitoring & Analysis

|

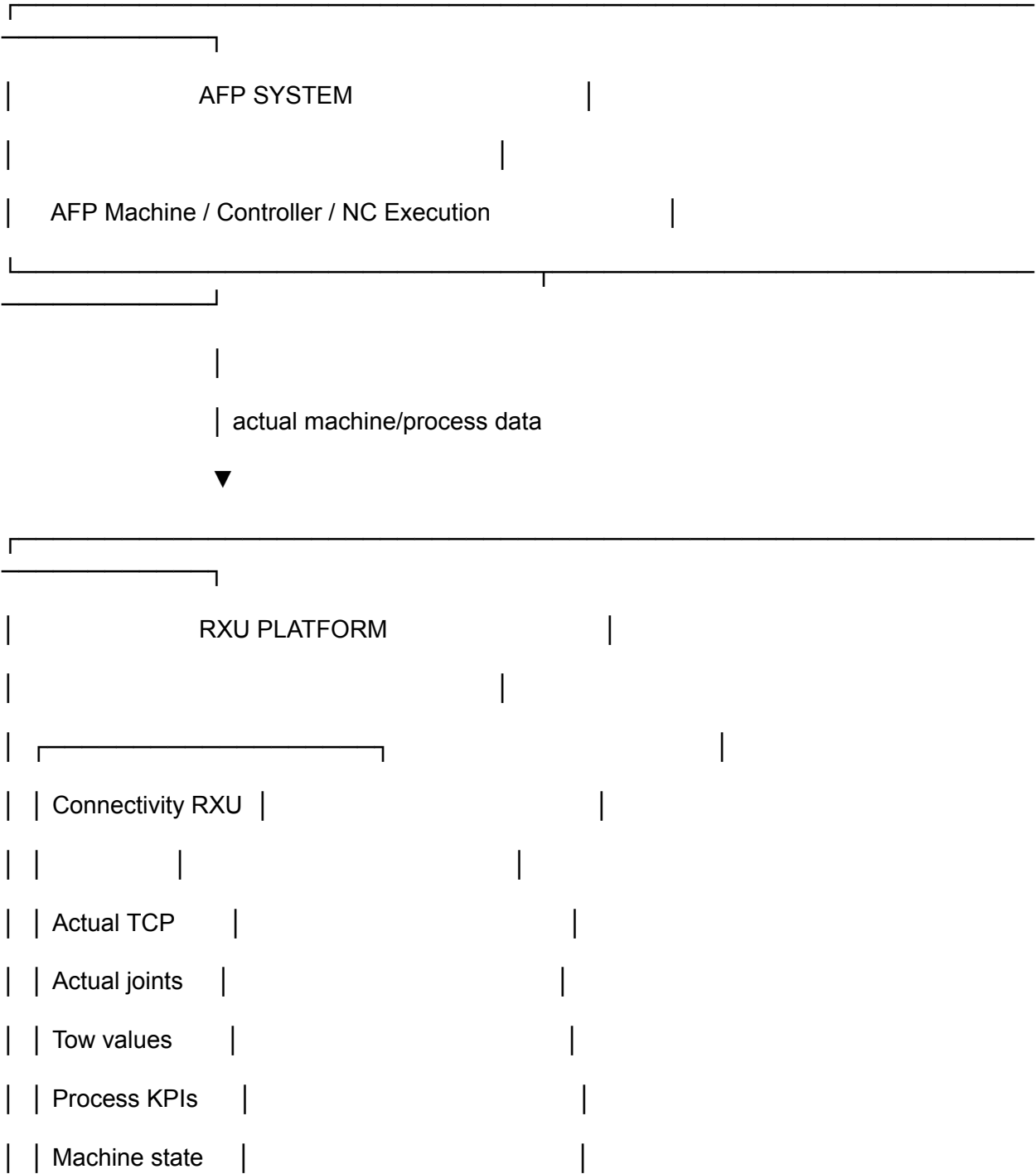
▼

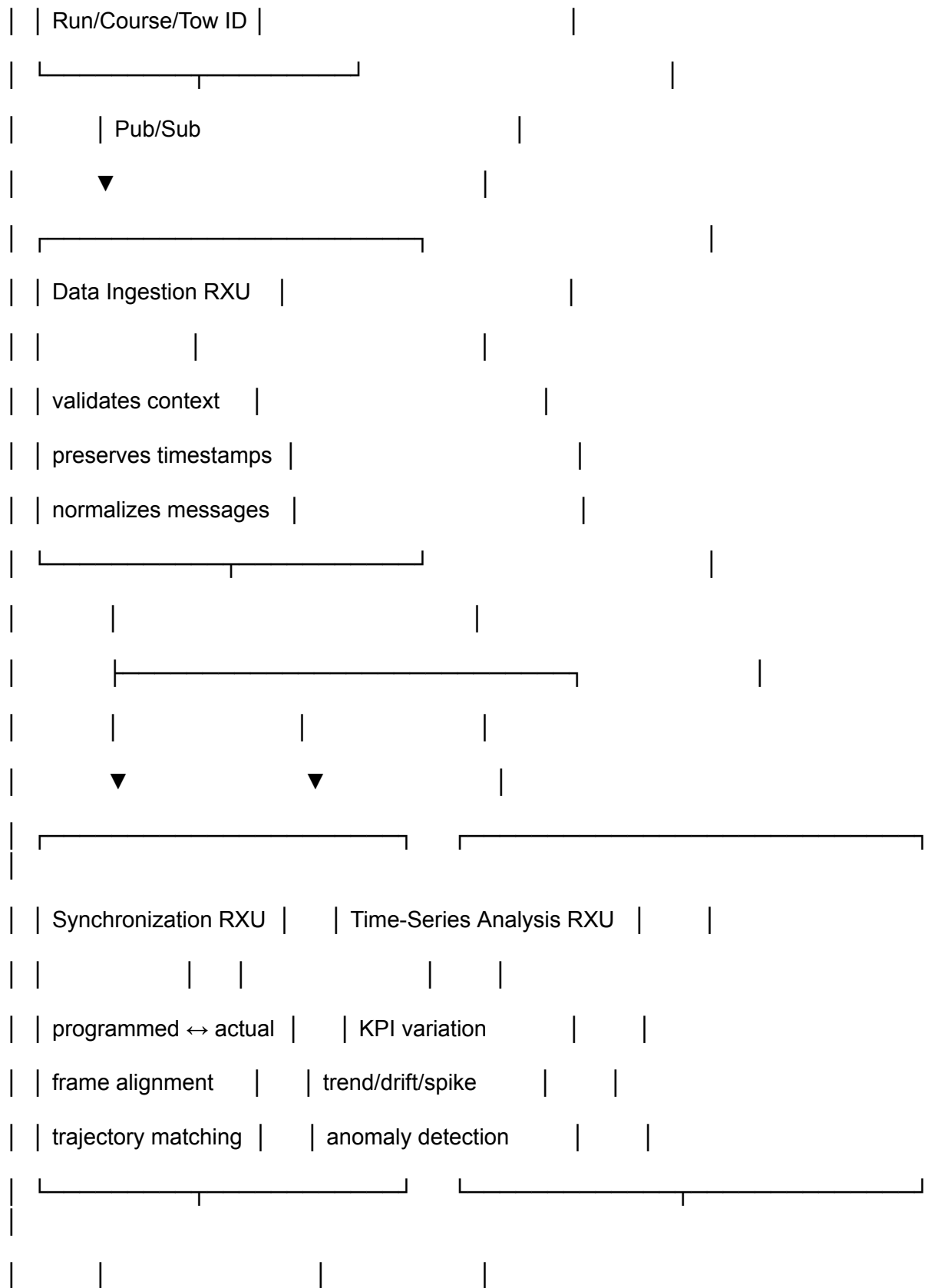
QuestDB / HMI / Engineering Analysis

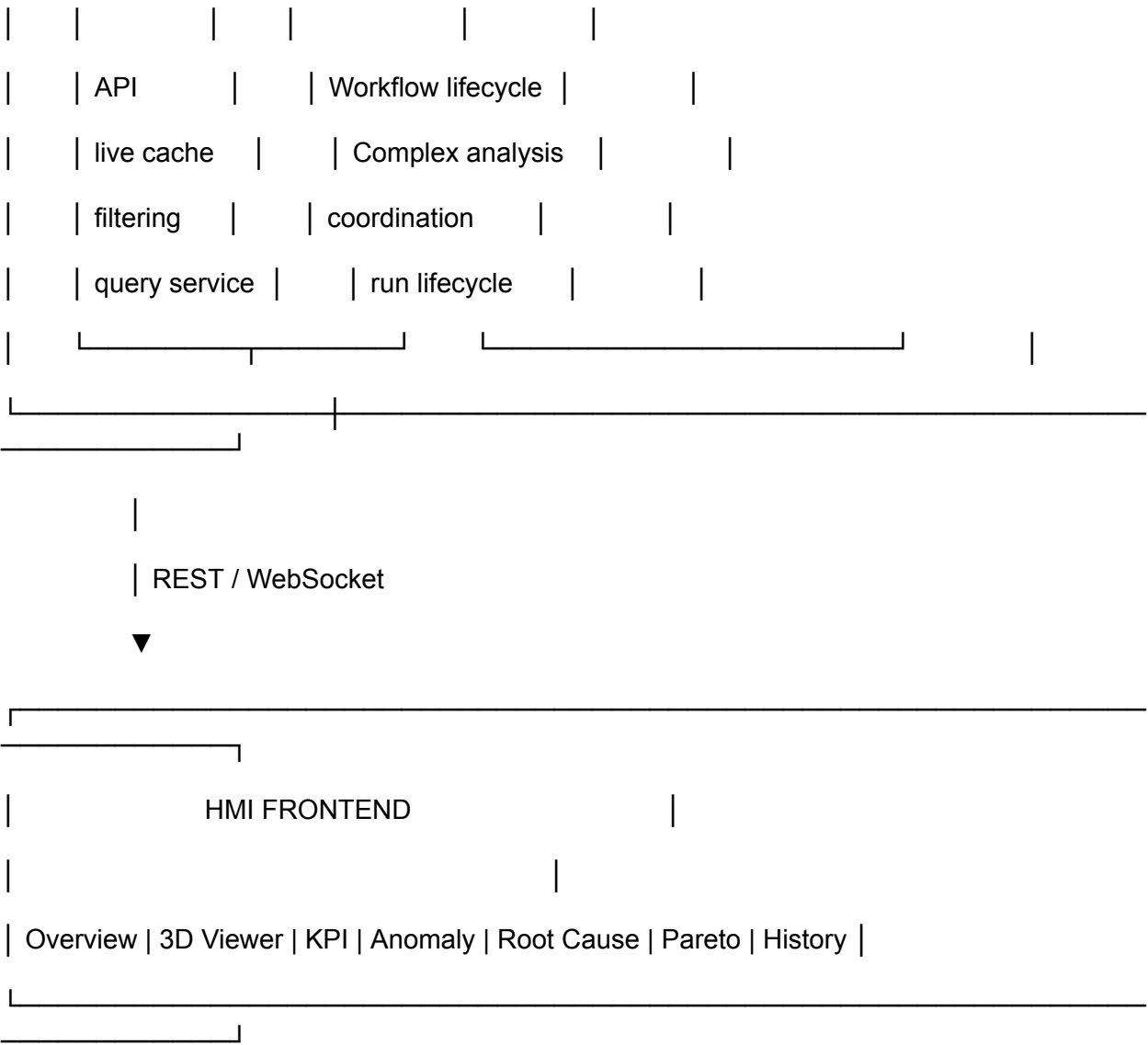
WP5.3 does **not** issue robot or AFP control commands.

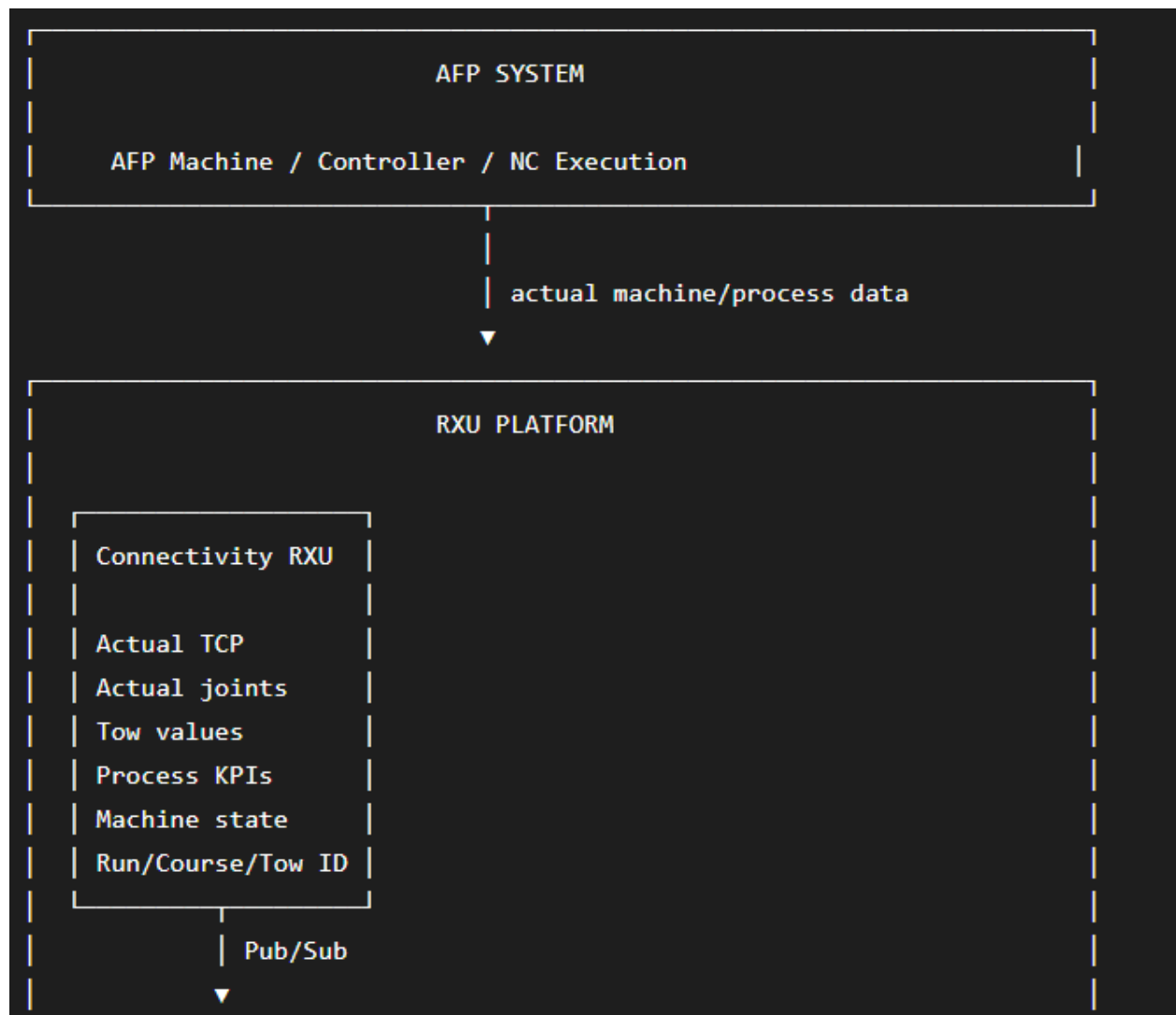
2. High-level logical architecture

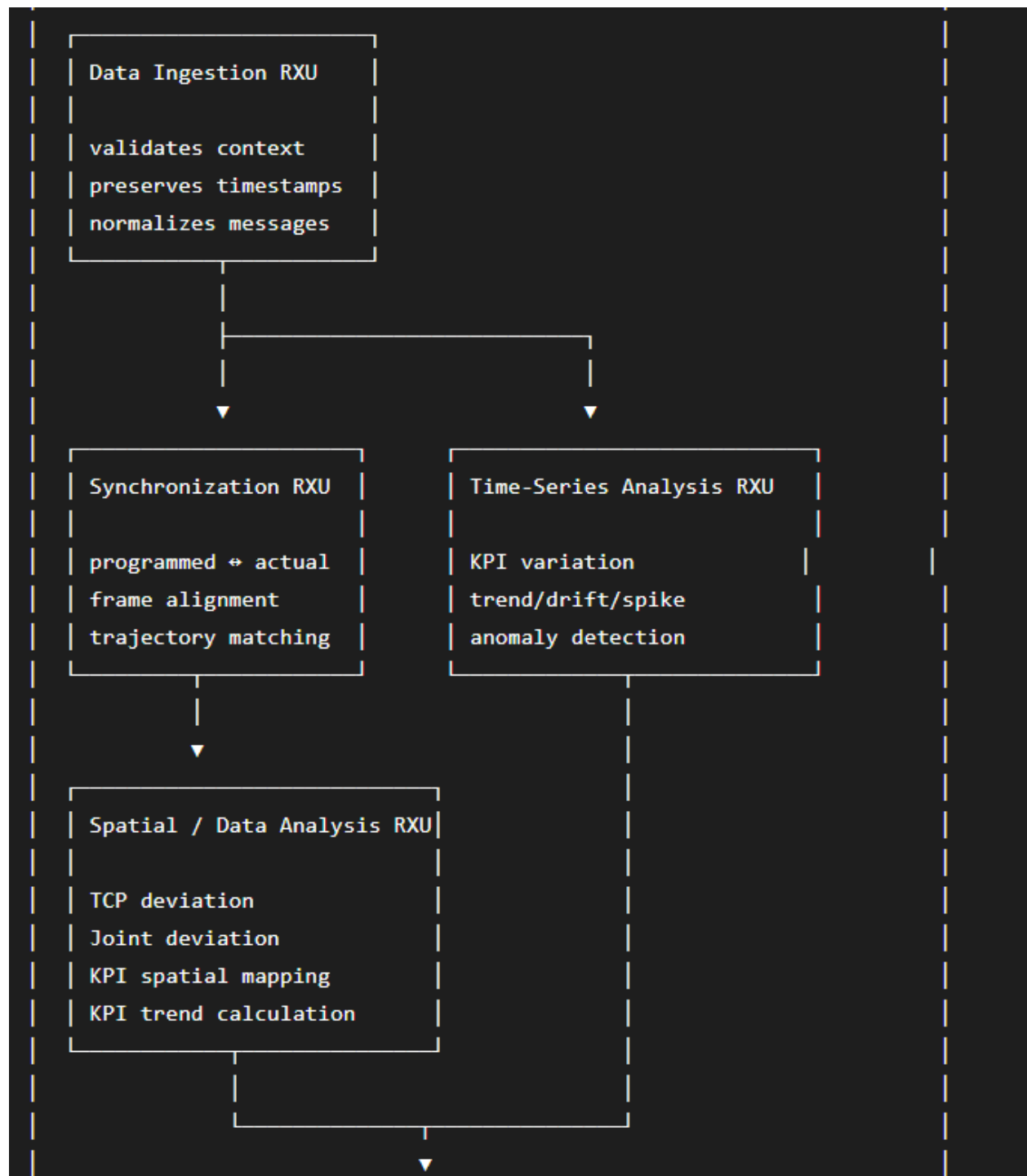
I would represent the architecture like this:

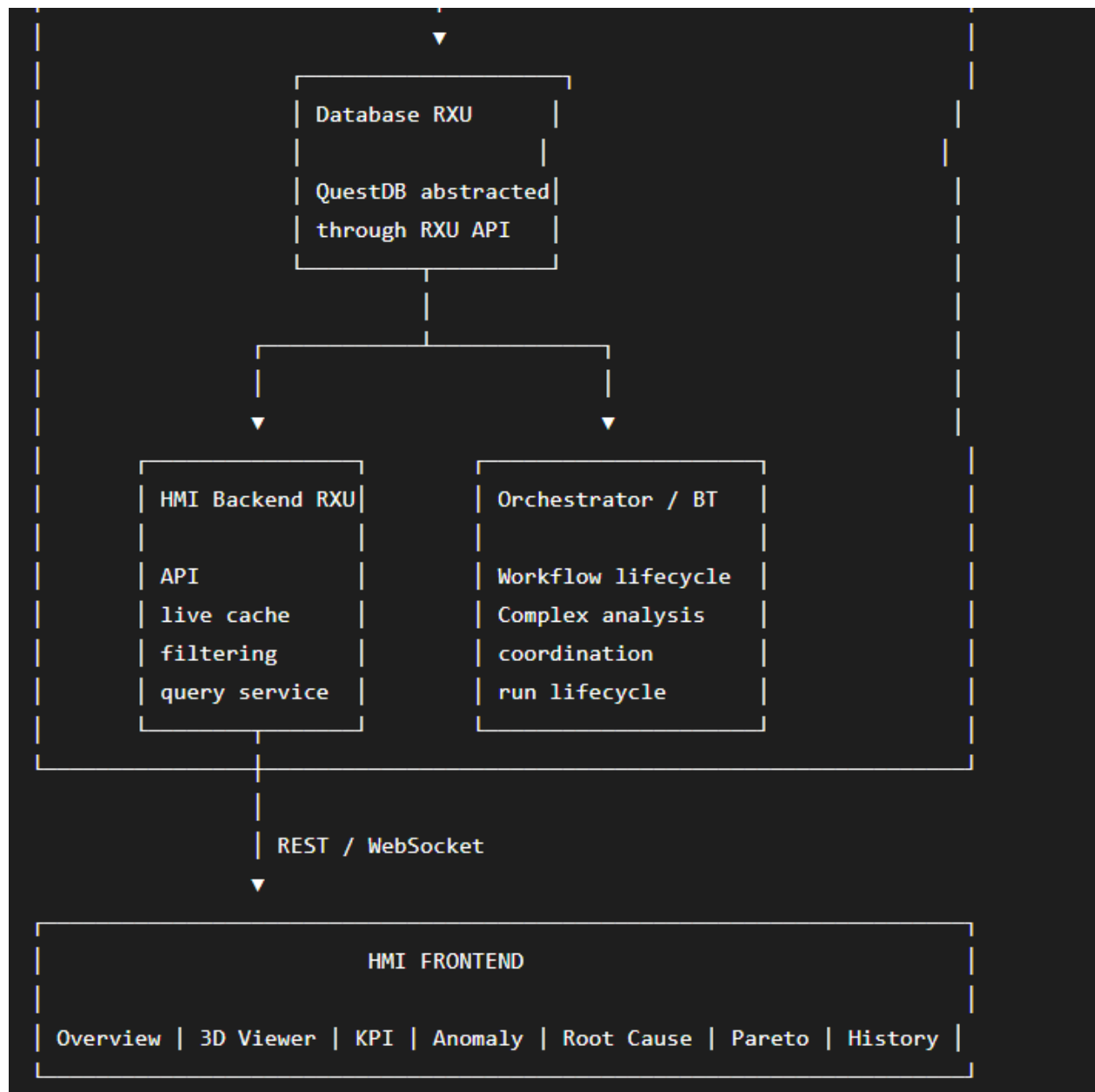












3. The three principal analysis paths

These should be treated as separate analysis capabilities even though they may reuse the same data.

3.1 TCP and joint deviation

This answers:

How much did the actual robot/machine motion differ from the programmed/reference trajectory?

Inputs:

Programmed TCP / joints

Actual TCP / joints

timestamps

run_id

course_id

possibly block_id

coordinate-frame information

The programmed information comes from the Database/NC RXU abstraction.

Actual information comes from Connectivity RXU.

The synchronization component establishes:

Actual sample



Corresponding programmed sample

Then:

TCP deviation:

$$dx = X_{\text{actual}} - X_{\text{programmed}}$$

$$dy = Y_{\text{actual}} - Y_{\text{programmed}}$$

$$dz = Z_{\text{actual}} - Z_{\text{programmed}}$$

position deviation =

$\sqrt{dx^2 + dy^2 + dz^2}$

Joint deviation:

J1 deviation = J1actual - J1reference

J2 deviation = J2actual - J2reference

...

Jn deviation = Jnactual - Jnreference

The result should contain not only the deviation magnitude but also the spatial context:

run_id

course_id

tow_id if relevant

timestamp

path_position

programmed_TCP

actual_TCP

dx/dy/dz

TCP deviation magnitude

joint deviations

This allows the result to be mapped back to the aircraft geometry.

4. TCP/joint deviation on 3D CAD

This corresponds closely to your requirements around:

TCP trajectory view

planned path view

planned-vs-actual view

3D geometry viewer

geometry layer control

KPI heatmap overlay

overlay selection

TCP point inspection

The frontend should support layers such as:

☒ CAD geometry

☒ Programmed TCP

☒ Actual TCP

☒ TCP deviation

☐ Joint deviation

☐ KPI overlay

☐ Anomaly markers

The user could therefore view:

Programmed only

Actual only

Programmed + actual

TCP deviation only

Joint deviation

TCP + KPI together

depending on the selected layer configuration.

For deviation visualization, I would use:

Green low deviation

Yellow increasing

Orange warning region

Red high deviation

but actual threshold values should come from configurable engineering limits rather than being fixed in the frontend.

5. KPI trend analysis — not anomaly detection

This distinction should remain very clear in the architecture.

KPI Trend Analysis answers questions such as:

How did tension vary along Course 05?

What was the compaction force along the geometry?

What was the temperature distribution over the panel?

This is **descriptive/process-data analysis**.

It does not say whether something is anomalous.

Example data:

timestamp	course	tow	tension
-----------	--------	-----	---------

10.001	C01	T01	42
--------	-----	-----	----

10.011	C01	T01	43
--------	-----	-----	----

10.021 C01 T01 44

...

The Data Analysis RXU calculates:

min

max

mean

variation

trend

moving average

distribution

and maps each sample to:

TCP/path_position

course_id

tow_id

This provides the link:

KPI value

|



physical position

|



3D CAD

6. KPI spatial visualization

Your frontend requirements already contain the necessary mechanism:

KPI heatmap overlay

KPI overlay selection

geometry layer control

The user could select:

Overlay:

- None
- Tow tension
- Compaction force
- Temperature
- Speed
- Pressure
- TCP deviation

or if the frontend supports multiple layers:

☒ TCP deviation

☒ Tow tension

This is where your requirement that **1 and 2 may be shown individually or together** should be implemented.

I would therefore define the HMI 3D Viewer conceptually as:

3D PROCESS VIEWER

Aircraft panel CAD	
programmed trajectory	
actual trajectory	
KPI heatmap	
deviation markers	

Layers



- [✓] CAD
- [✓] Programmed TCP
- [✓] Actual TCP

Overlay 1:

[TCP Deviation ▼]

Overlay 2:

[Tow Tension ▼]

Filters

Run [RUN_025 ▼]

Course [C07 ▼]

Tow [T12 ▼]

Time [...]

3D PROCESS VIEWER

Aircraft panel CAD

programmed trajectory

actual trajectory

KPI heatmap

deviation markers

Layers

[✓] CAD

[✓] Programmed TCP

[✓] Actual TCP

Overlay 1:

[TCP Deviation ▼]

Overlay 2:

[Tow Tension ▼]

Filters

Run [RUN_025 ▼]

Course [C07 ▼]

Tow [T12 ▼]

Time [....]

A design decision will eventually be needed about whether two heatmaps can be simultaneously legible.

For the first implementation, I would recommend:

Primary geometry heatmap = one selected quantity

+

secondary path/markers = another quantity

For example:

Panel color = Tow Tension

TCP path color = TCP deviation

This is easier to interpret than overlaying two independent color heatmaps on the same mesh.

7. Filtering architecture

Filtering should not just be a frontend cosmetic operation.

The frontend should send the selected context to the HMI Backend.

For example:

VisualizationRequest

run_id = RUN_025

course_id = COURSE_07

tow_id = TOW_12

metric = TENSION

start_time = ...

end_time = ...

HMI Backend then obtains only the necessary information.

Your main filters should include at least:

Run ID

Part ID

Program ID

Course ID

Tow ID

Time range

KPI type

Machine/process state

Not every filter is relevant for every view.

For example:

TCP deviation

Run ID

Course ID

time range

Tow ID is probably not intrinsically meaningful for TCP deviation unless the machine/process definition ties the TCP data directly to a particular tow or tow group.

KPI view

Run ID

Course ID

Tow ID

KPI

time range

Process anomaly

Run ID

Course ID

Tow ID

KPI

severity

time range

anomaly type

8. Process anomaly architecture

Your third case is different from KPI trend visualization.

The time-series analysis component continuously analyzes KPI behavior.

For example:

Tow tension

|

▼

Windowing

|

|— threshold violation

|— spike

|— sudden drop

|— drift

|— oscillation

└— statistical deviation



Anomaly Event

An anomaly message might contain:

anomaly_id

timestamp

run_id

course_id

tow_id

KPI

actual_value

expected/normal value

severity

duration

anomaly_type

spatial_position

That result is published continuously and persisted.

9. Grafana for process anomaly visualization

For the **time-series anomaly dashboard**, Grafana is actually a good option, particularly for engineering/diagnostic monitoring.

The architecture can be:

Time-Series Analysis RXU

|



Database RXU

|



QuestDB

|



Grafana

Grafana can show:

KPI time-series

threshold bands

anomaly markers

severity

course filters

tow filters

run filters

time window

For example:

Tow Tension

50 ————— threshold



45 ————— ● ————|

anomaly

40 ————— \ _____ / —————

35 ————— threshold

time →

My recommendation

Use **Grafana for engineering time-series/anomaly dashboards** if this is acceptable within the program architecture.

Use your **custom HMI 3D viewer for spatial context**.

That gives:

Custom HMI

- |
- |— 3D geometry
- |— TCP deviation
- |— KPI spatial mapping
- |— anomaly marker on geometry
- |— drill-down/navigation

Grafana

- |
- |— KPI time-series
- |— anomaly timeline
- |— thresholds

└— multiple KPI comparison

└— detailed engineering plots

You could then provide a link/drill-down from an anomaly in the HMI to the corresponding Grafana time window.

If you want a single Airbus-look-and-feel application without embedding another product, the same charts can eventually be built directly in the frontend. But for early WP5.3 engineering use, Grafana + QuestDB is a very reasonable combination.

10. Continuous processing versus user interaction

This is central to your sequence diagrams.

The following should **continue whether or not the user clicks anything**:

Connectivity

Data ingestion

data persistence

trajectory synchronization during active run

TCP deviation

joint deviation

KPI calculation/preparation

time-series anomaly detection

anomaly persistence

User interaction affects:

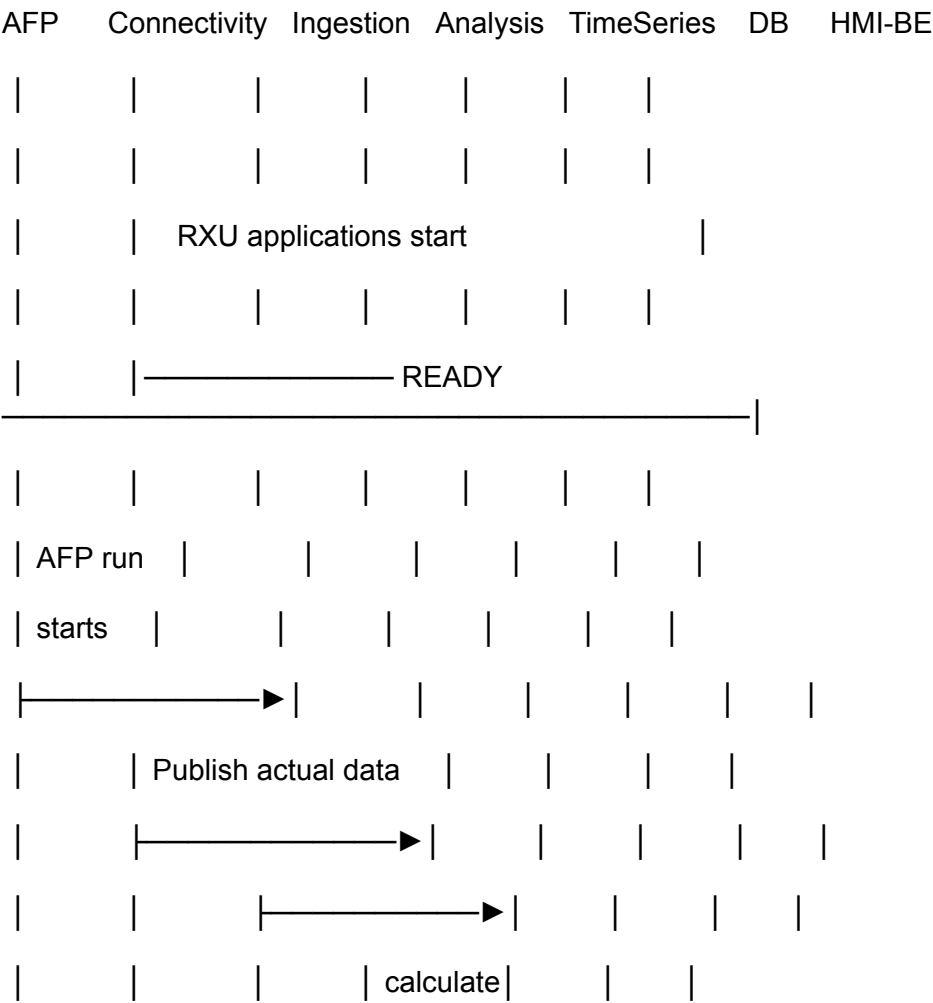
which dataset is requested

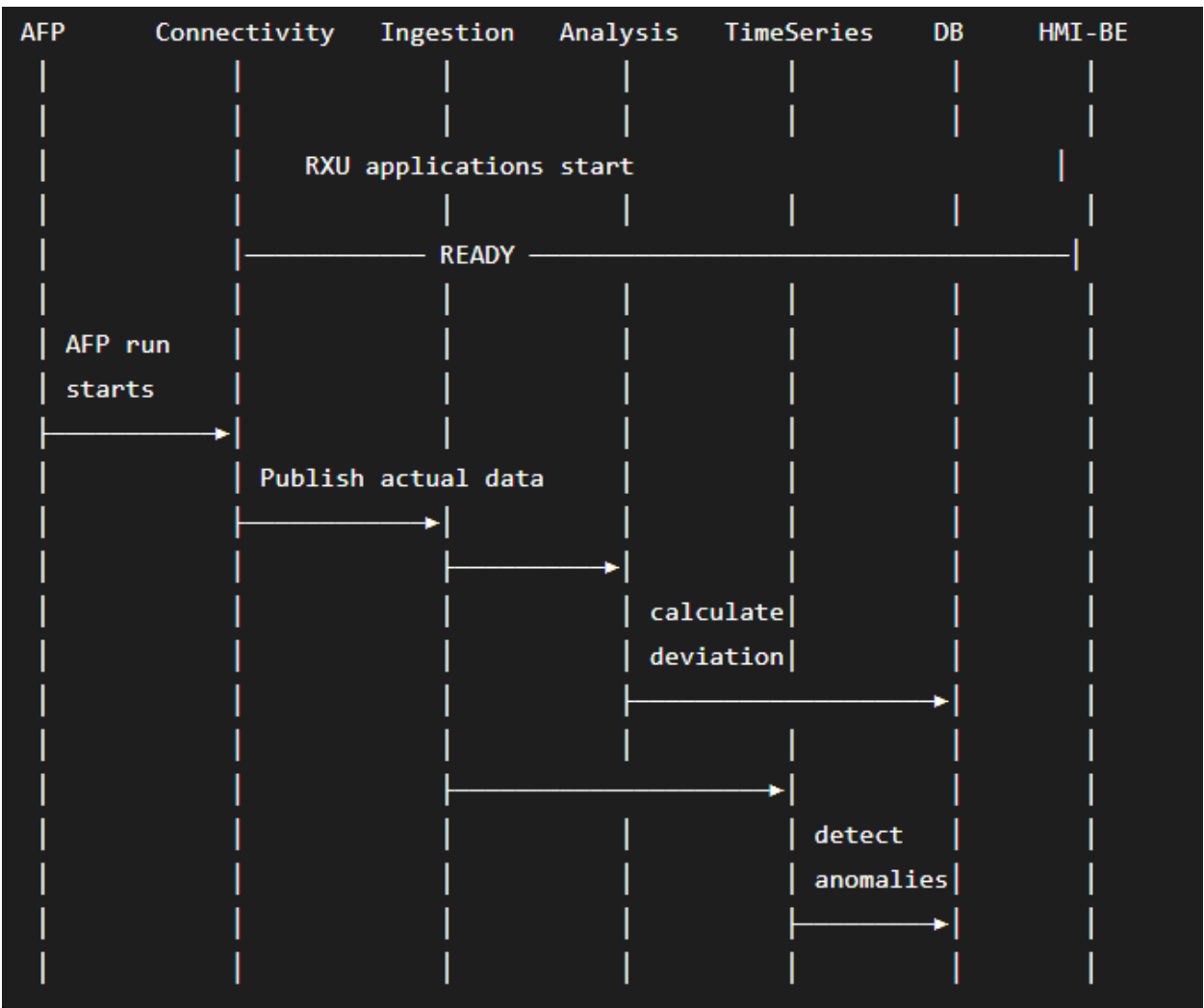
which run is displayed

which course is displayed

- which tow is displayed
 - which KPI is visualized
 - which layers appear on CAD
 - whether live data is pushed to browser
 - which historical period is retrieved
 - whether an investigation workflow is launched
-

11. Sequence A — IPC starts, no user logged in



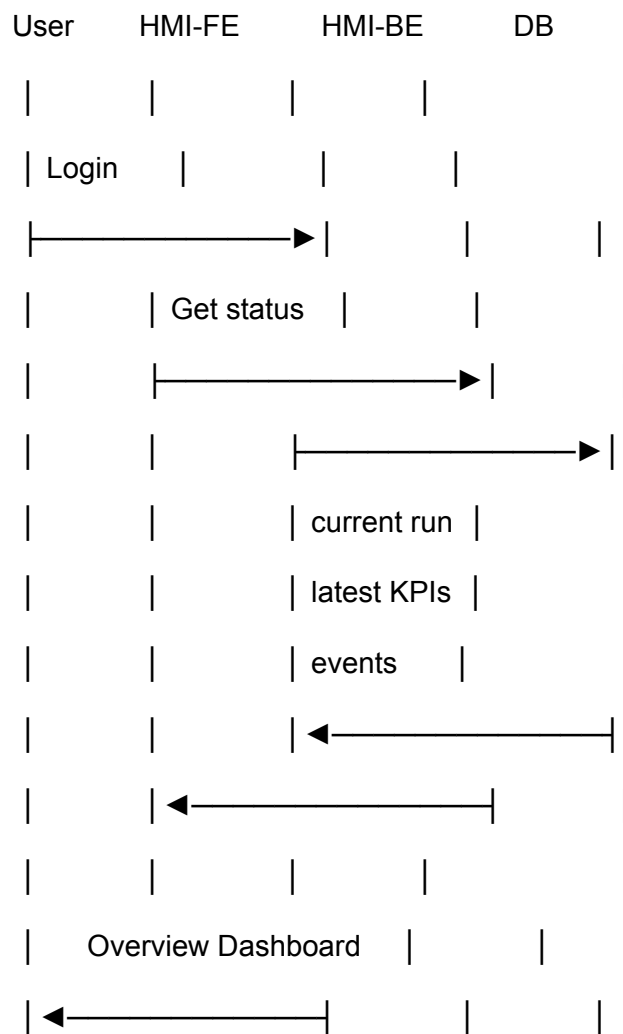


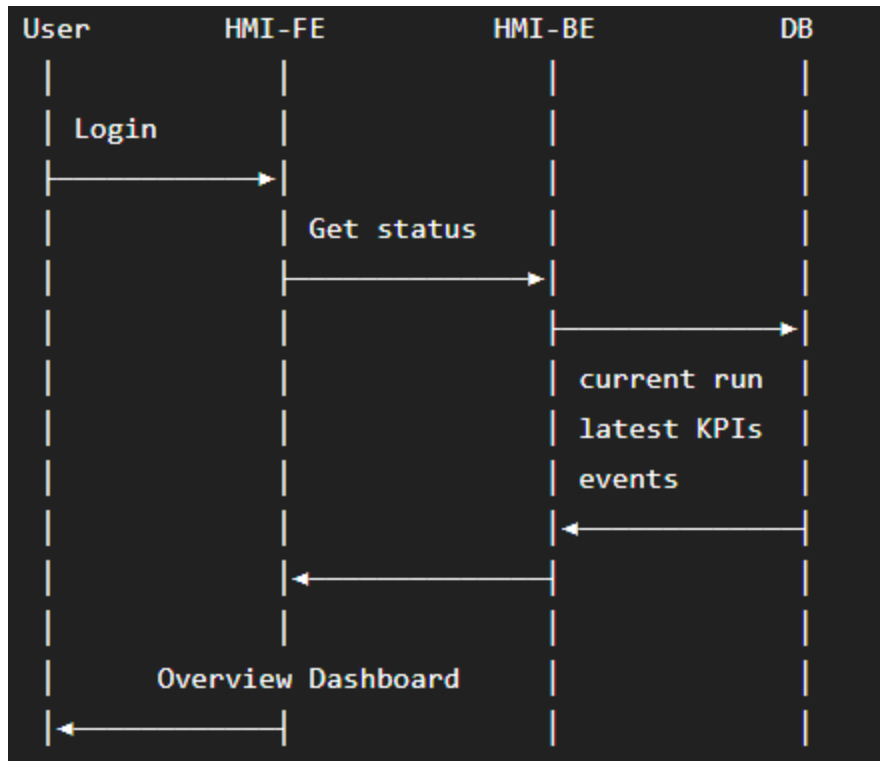
Nobody is logged in.

Analysis still happens.

That is intentional.

12. Sequence B — User logs in but does not click an analysis page





The HMI may show:

RUN_025

Program PANEL_A_17

Course 07

State LAYING

Latest KPIs

Latest event

Connection status

But no 3D deviation data needs to be streamed to the browser yet.

Meanwhile:

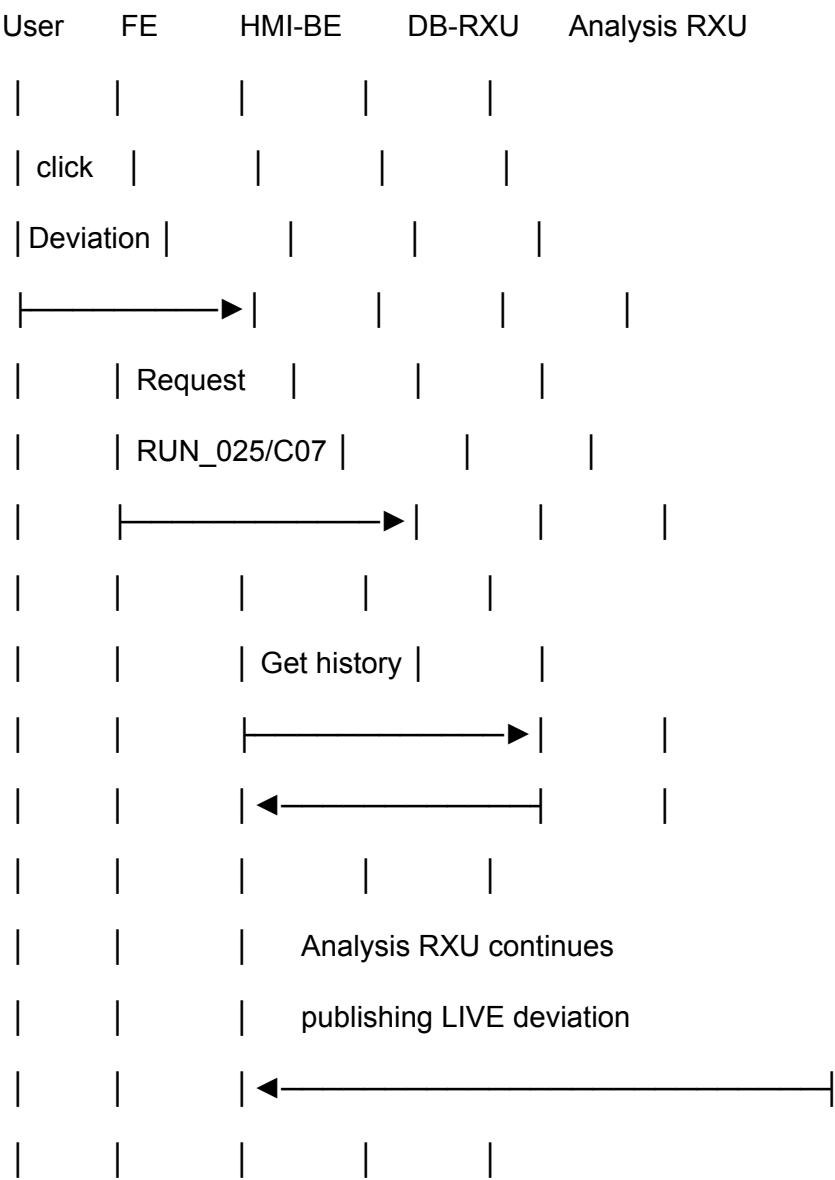
Analysis RXU

Time-Series Analysis RXU

continue running independently.

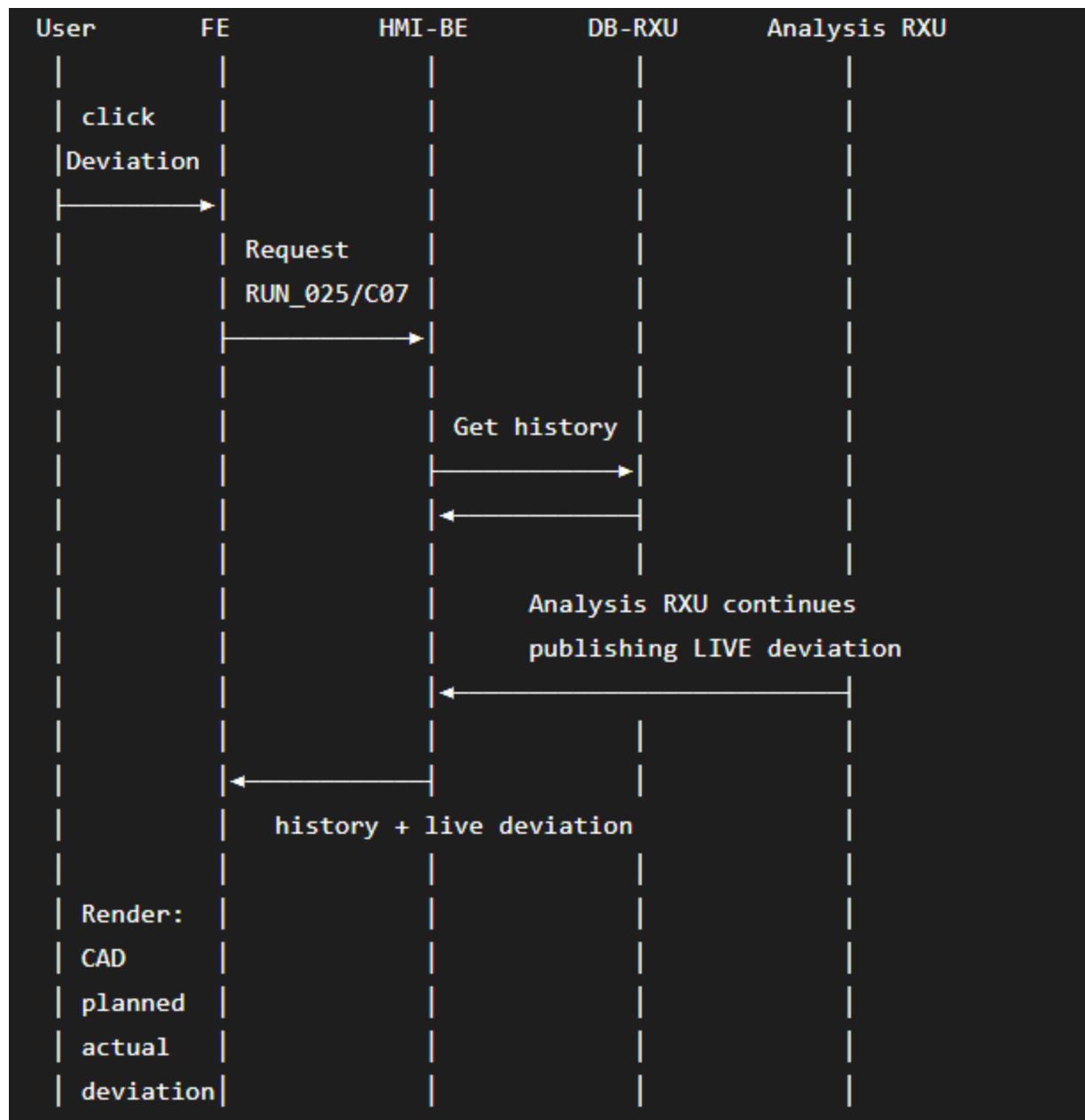
13. Sequence C — user clicks Planned vs Actual / deviation

This is one of your most important sequences.



		←		
		history + live deviation		
Render:				
CAD				
planned				
actual				
deviation				

No BT is needed simply to display an already-calculated deviation result.



14. Sequence D — user selects TCP versus joint deviation

Inside the same 3D viewer:

User selects:

Metric = TCP Deviation

Frontend sends:

metric = TCP_POSITION_DEVIATION

course = C07

Backend filters the available data and returns the corresponding spatial values.

If the user switches to:

Metric = JOINT_3_DEVIATION

the backend returns:

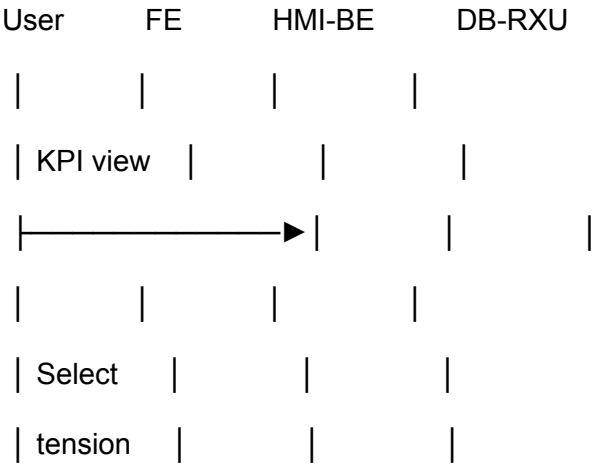
path position

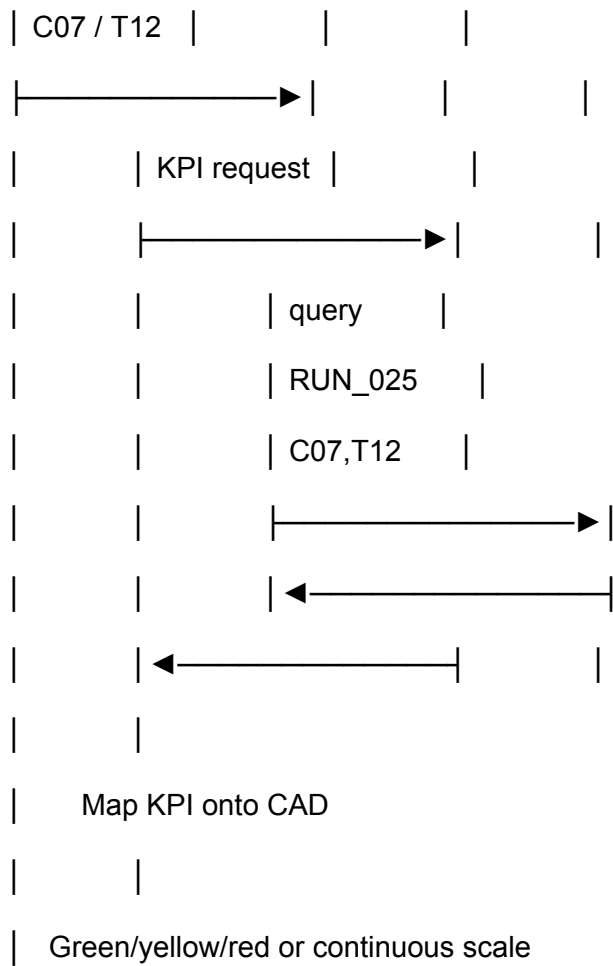
joint 3 deviation

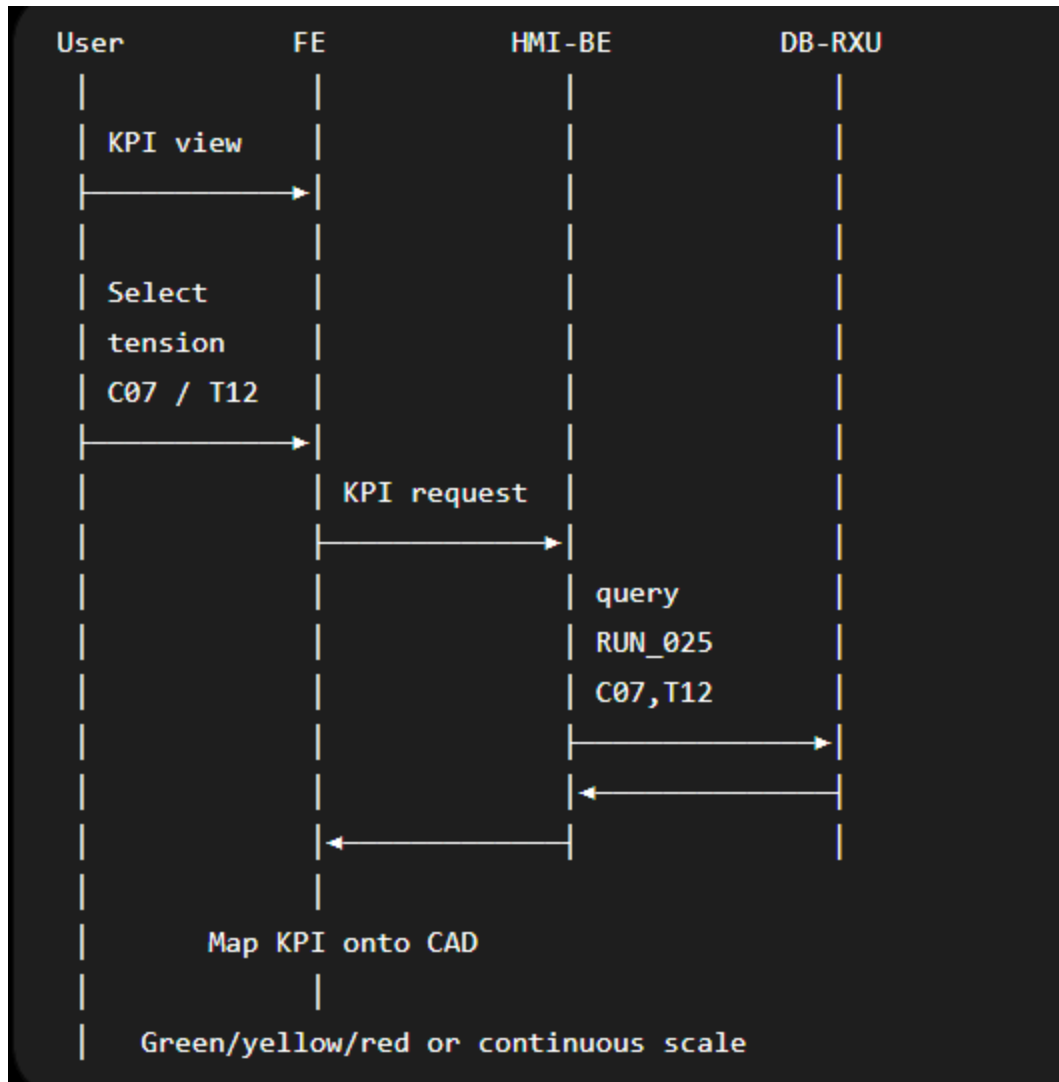
corresponding TCP location

The CAD coloring is therefore attached to the TCP spatial location even though the value being visualized represents joint deviation.

15. Sequence E — user clicks KPI spatial trend







Again, the Data Analysis RXU previously created or spatially associated the KPI values.

The HMI is **not performing the engineering computation**.

16. Sequence F — user wants deviation + KPI simultaneously

This is directly relevant to your requested UI.

User

|

| Enable:

| [✓] TCP Deviation

| [✓] Tow Tension



Frontend

|

| VisualizationRequest

|

| run=RUN_025

| course=C07

| tow=T12

| primary_metric=TENSION

| secondary_metric=TCP_DEVIATION



HMI Backend

|

|— retrieve/filter KPI data

|— retrieve/filter deviation

|— align using path position / spatial coordinate

|



Frontend

Then I would render:

CAD surface = tension heatmap

Actual TCP trajectory =

color/thickness/markers representing deviation

rather than trying to paint two full heatmaps on the same mesh.

This makes the visualization much easier to understand.

17. Sequence G — filter changes

Suppose the user changes:

Course C07 → C08

or:

Tow T12 → T17

There is **no reason to restart analysis**.

User

|

| Select Course C08

▼

Frontend

|

| FilterRequest

▼

HMI Backend

|

| query/filter



Database RXU



| result



HMI Backend



Frontend

The same applies to:

tow

time range

KPI

run

severity

BT is generally unnecessary.

18. Sequence H — process anomaly detection while nobody watches it

This is critical.

Connectivity



| KPI stream



Time-Series Analysis RXU

|

| analyze continuously

|

|— normal

|

|— normal

|

|— normal

|

|— anomaly detected

|



AnomalyEvent

|



Database RXU

At this point, the user may be:

logged out

looking at TCP page

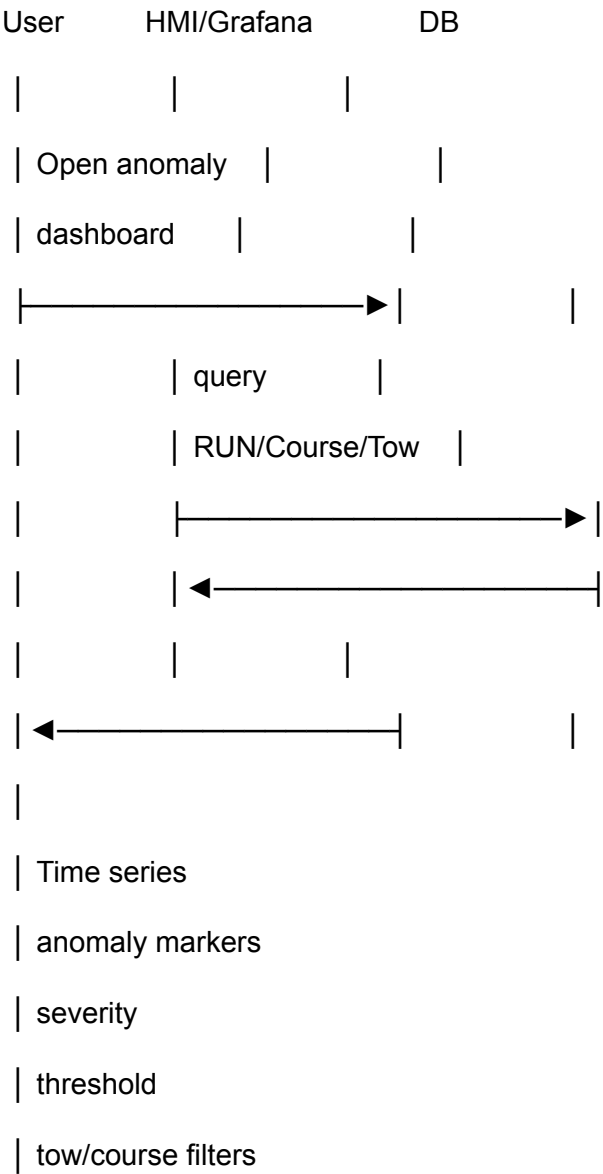
looking at KPI page

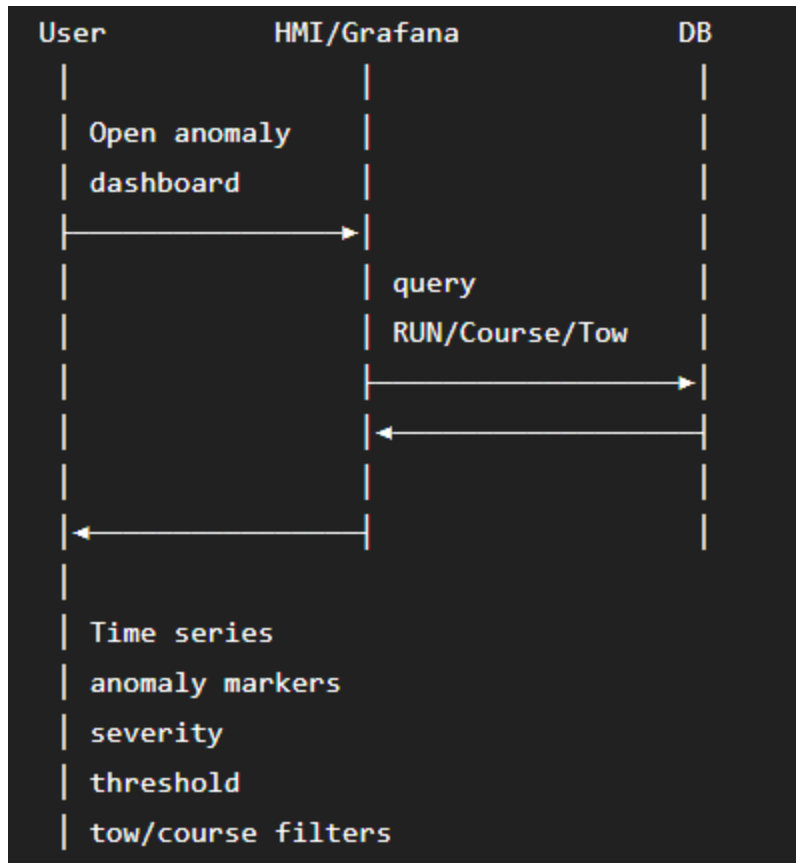
looking at overview

It makes no difference.

Anomaly detection continues.

19. Sequence I — user opens anomaly dashboard





For Grafana specifically:

QuestDB

|

▼

Grafana

|

▼

Dashboard

The Time-Series Analysis RXU remains the component detecting anomalies.

Grafana is only visualization/querying.

20. Anomaly drill-down into spatial HMI

A powerful future workflow would be:

Grafana / Anomaly HMI

Anomaly:

Run RUN_025

Course C07

Tow T12

Time 10:14:22

Tension 63 N

Severity Critical

|

| "Show on 3D"



HMI 3D Viewer

Panel

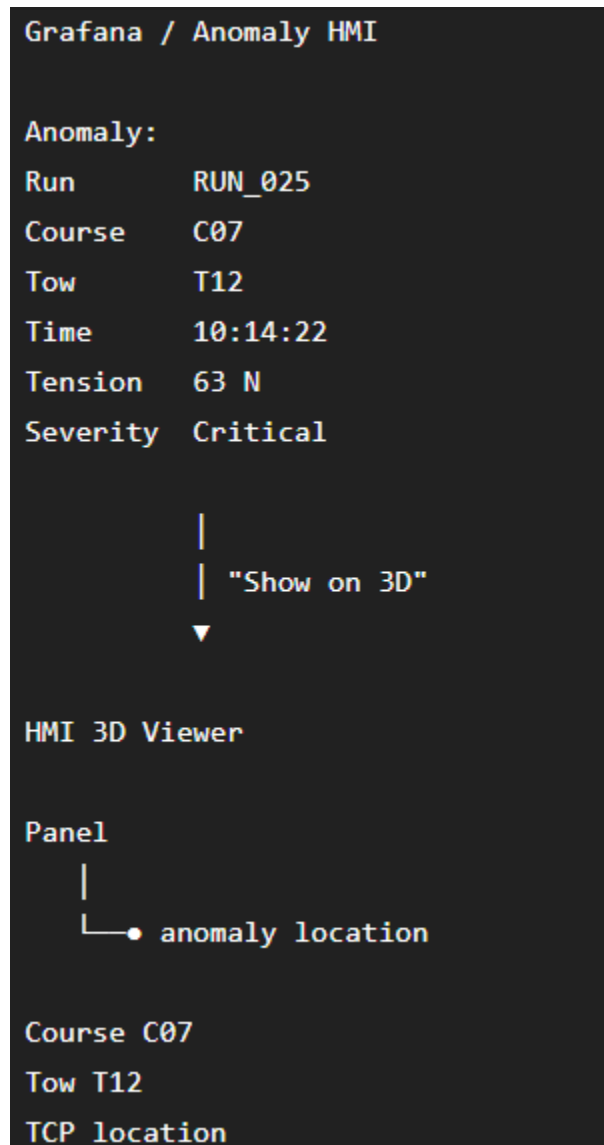
|

└─● anomaly location

Course C07

Tow T12

TCP location



This links temporal analysis and spatial analysis.

That would be extremely useful for your client.

21. Where BT should sit

I would not draw the BT between every frontend/backend interaction.

BT should handle things such as:

initialize run-monitoring workflow

finalize run

historical replay workflow

root-cause investigation

multi-stage model analysis

complex comparison

possibly generation of final report

It should **not** normally handle:

show TCP deviation

change course filter

change tow filter

change KPI

zoom CAD

toggle programmed path

open graph

retrieve already-calculated KPI data

Those are HMI/backend/data-service operations.

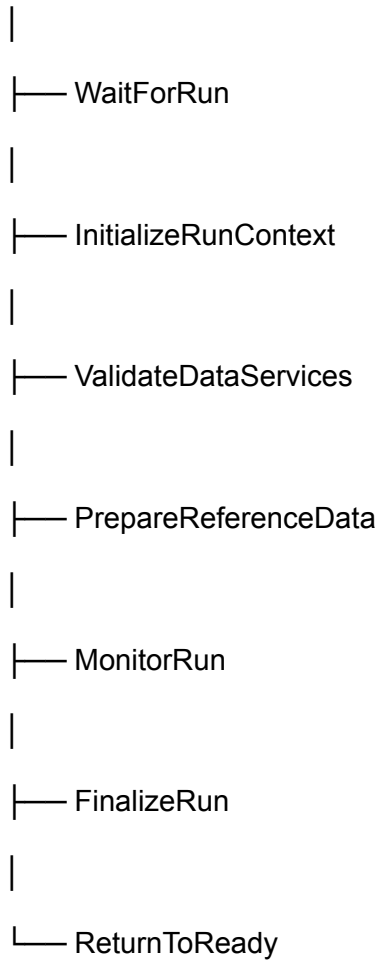
22. A good BT workflow for run lifecycle

ROOT

|



MonitorProcess



During **MonitorRun**, RXU pub/sub components independently process the high-frequency data.

The BT monitors their states rather than transporting their samples.

23. Final data-plane architecture

I would make this very prominent in your architecture documentation:

ACTUAL DATA

AFP

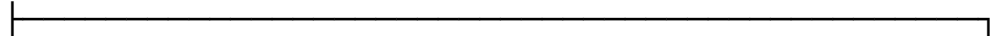
|



Connectivity RXU



Data Ingestion



Synchronization



Spatial/Data Analysis

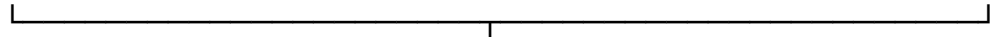


TCP deviation

Joint deviation

KPI trends

Spatial mapping



Time-Series Analysis



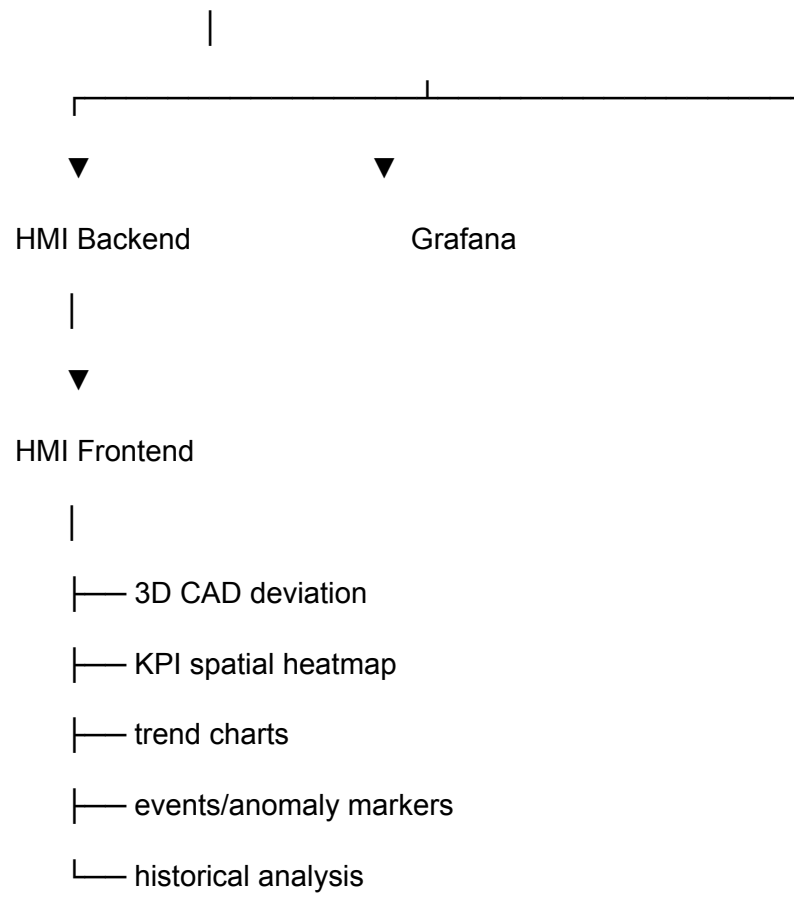
KPI anomaly



Database RXU



QuestDB



ACTUAL DATA

AFP



Connectivity RXU



Data Ingestion



Synchronization



Spatial/Data Analysis



TCP deviation



Joint deviation



KPI trends



Spatial mapping

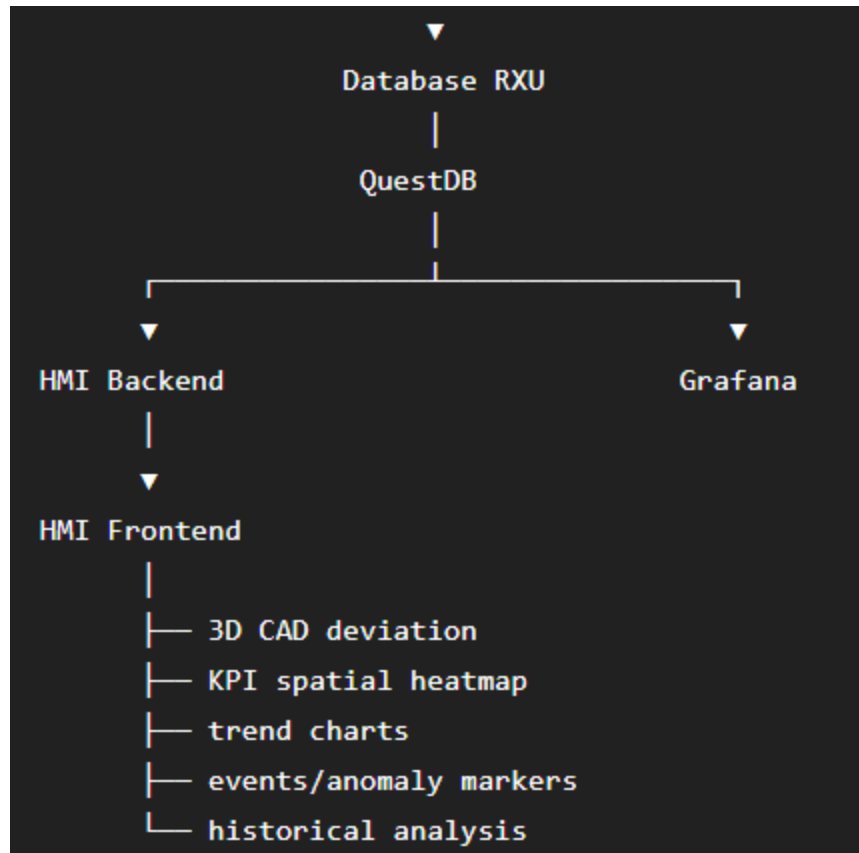


Time-Series Analysis



KPI anomaly





24. Recommended HMI information architecture

Based on the frontend requirements you showed, something approximately like this would make sense:

WP5.3 Process Twin HMI

Overview

|

|— Current Run

|— Current State

└─ KPI Summary

└─ Latest Events

Process Trends

|

└─ KPI trend charts

└─ KPI comparison

└─ Course filter

└─ Tow filter

└─ Time filter

3D Process View

|

└─ CAD Geometry

└─ Programmed TCP

└─ Actual TCP

└─ TCP Deviation

└─ Joint Deviation

└─ KPI Heatmap

└─ Anomaly Markers

└─ Layer / filter control

Process Anomalies

|

- |— Anomaly list
- |— Timeline
- |— Severity
- |— KPI
- |— Tow / Course
- └— Grafana/time-series drill-down

Expected vs Actual

- |
- |— Behaviour-model output
- |— Actual behaviour
- |— Residual/deviation
- └— Process envelope

Root Cause

- |
- |— Selected anomaly/event
- |— related KPIs
- |— correlations
- └— investigation support

Historical

- |
- |— run browser

- |— replay
- └— run comparison

Maintenance

- |
- └— Pareto dashboard

This is compatible with the requirements you supplied without committing to whether these become separate browser pages, tabs, or dashboard panels.

25. The key design decision I would put into the architecture document

I would explicitly state:

Analytical computation shall be decoupled from HMI navigation. Data ingestion, synchronization, deviation computation, KPI preparation and process anomaly detection shall execute according to the manufacturing run lifecycle and shall not depend on whether a corresponding HMI page is currently open. HMI user actions shall control data selection, filtering, retrieval, visualization and explicit investigative workflows.

That single architectural decision resolves most of the sequence questions we have been discussing.

And for the three functions you specifically identified, I would define the intended behaviour as:

Function	Calculation	Storage	Visualization	User click starts calculation?
----------	-------------	---------	---------------	--------------------------------

TCP & joint deviation	Continuous during active run	QuestDB through DB RXU	3D CAD	No
KPI trends/spatial mapping	Continuous/periodic	QuestDB through DB RXU	3D CAD + charts	No
KPI anomaly detection	Continuous time-series analysis	QuestDB through DB RXU	Grafana / HMI anomaly page	No
Filters / overlay choice	—	—	User controlled	Yes, affects only view/query
Root-cause/dee per investigation	On demand	Results may be persisted	Investigation page	Yes, BT may orchestrate

This is the architecture I would recommend taking forward into your formal WP5.3 architecture and the sequence diagrams.

now based on the front end requirement

can you give detailed documenttaion on how architecture looks like?

also a sequence diagram to explain how in each sequence is? whenever a user clicks or not clicks

i am looking for detailed one

also make sure that we are covering three important calculation

1. Tcp and joint deviation - Variation should be shown on 3D CAD geoemtry

2. KPI trends (not anomaly) - on 3D CAD geoemtry

3. Process anomaly (any variation in KPI using time series analysis) - show on grafana dashboard or any other better options

3. Health check RXU app (machine haelth) - pareto dashboard

4. Dummy rxu predtive maintainennace app (grfana ddashbaord)

all above 4 is RXU app

in 1st and 2nd case probably user should have an option to see both, or only one at a time (filter)

also filter based on IDs such as tow id, course id, part id, product id, layer id, strip ID, etc

also give detailed proto definition schema when communicating info between each components

explain whether its service -client or pub-sub

keep in mind that we use RXU platform

Yes. Based on the frontend requirements you shared and the RXU constraints you described, I would formalize the architecture around **continuous monitoring/analysis plus user-driven visualization**, rather than making calculations depend on HMI navigation.

One small clarification first: your message lists **five RXU analysis/monitoring functions** because the numbering repeats 3: TCP/joint deviation, KPI trend/spatial analysis, process anomaly analysis, machine health/Pareto, and dummy predictive maintenance. I will model them as five RXU applications.

WP5.3 Process Twin & HMI — Detailed Architecture

1. Architectural intent

WP5.3 is a **monitoring, analysis and visualization system**. It does not control the AFP machine, change NC execution, modify process parameters, or command machine states.

The AFP system continues operating independently:

AFP Machine / Controller

|

| executes NC program

| controls motion/process

▼

Manufacturing

WP5.3 observes that execution:

AFP Machine

|



Connectivity RXU



WP5.3 RXU Applications



└─ TCP / Joint Deviation

└─ KPI Spatial Analysis

└─ Time-Series Anomaly Analysis

└─ Machine Health

└─ Predictive Maintenance



Database RXU / QuestDB



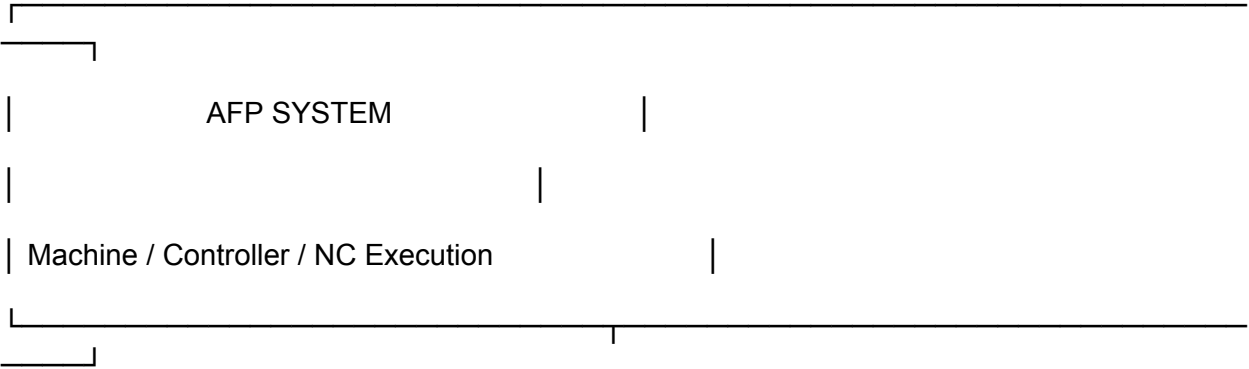
HMI / Grafana

The key architectural decision should be:

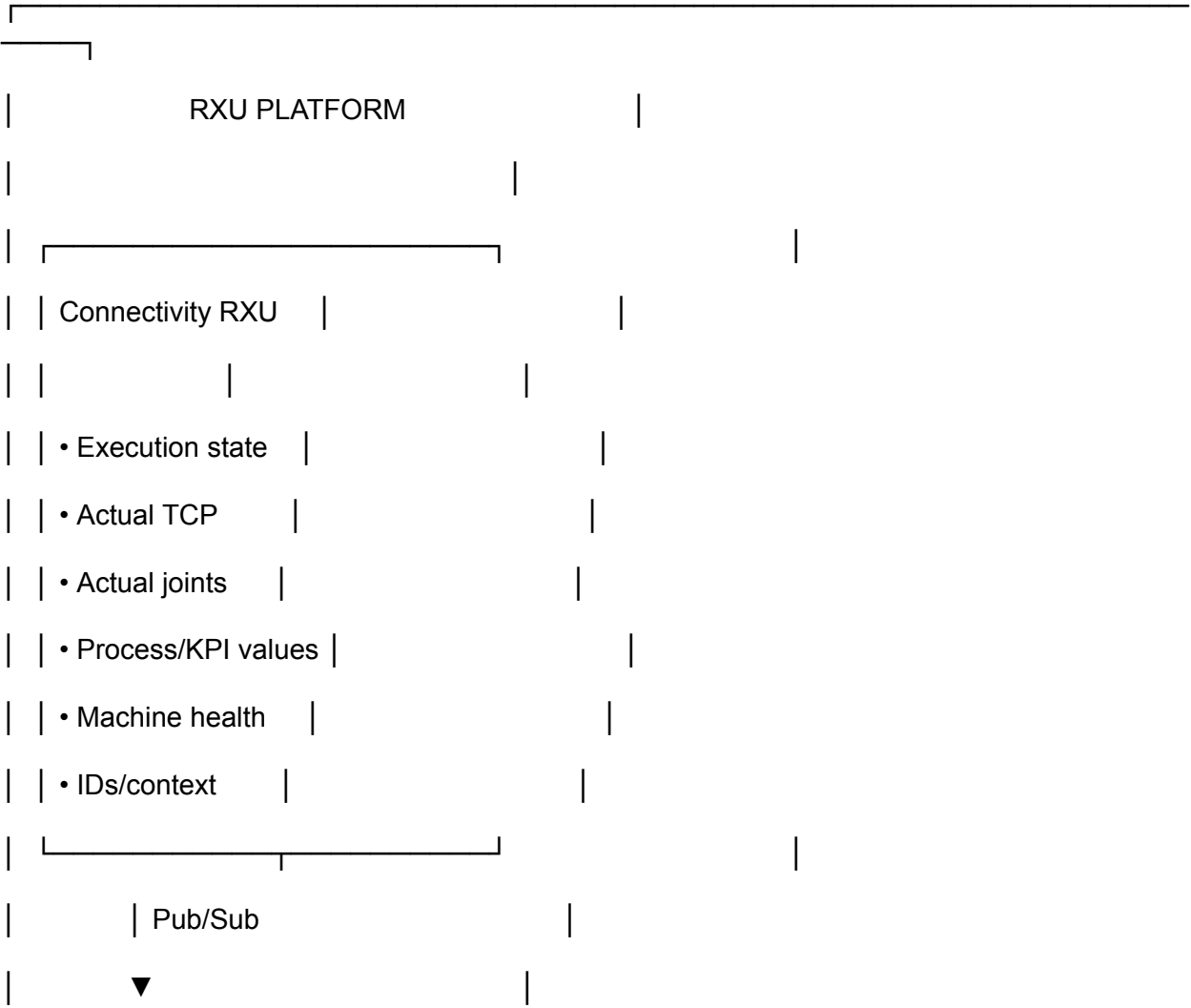
Analytical computation is independent of HMI navigation. The RXU applications calculate and publish their results during an active manufacturing run even if no user is logged into the HMI. User interaction determines which results are queried, filtered, streamed and rendered.

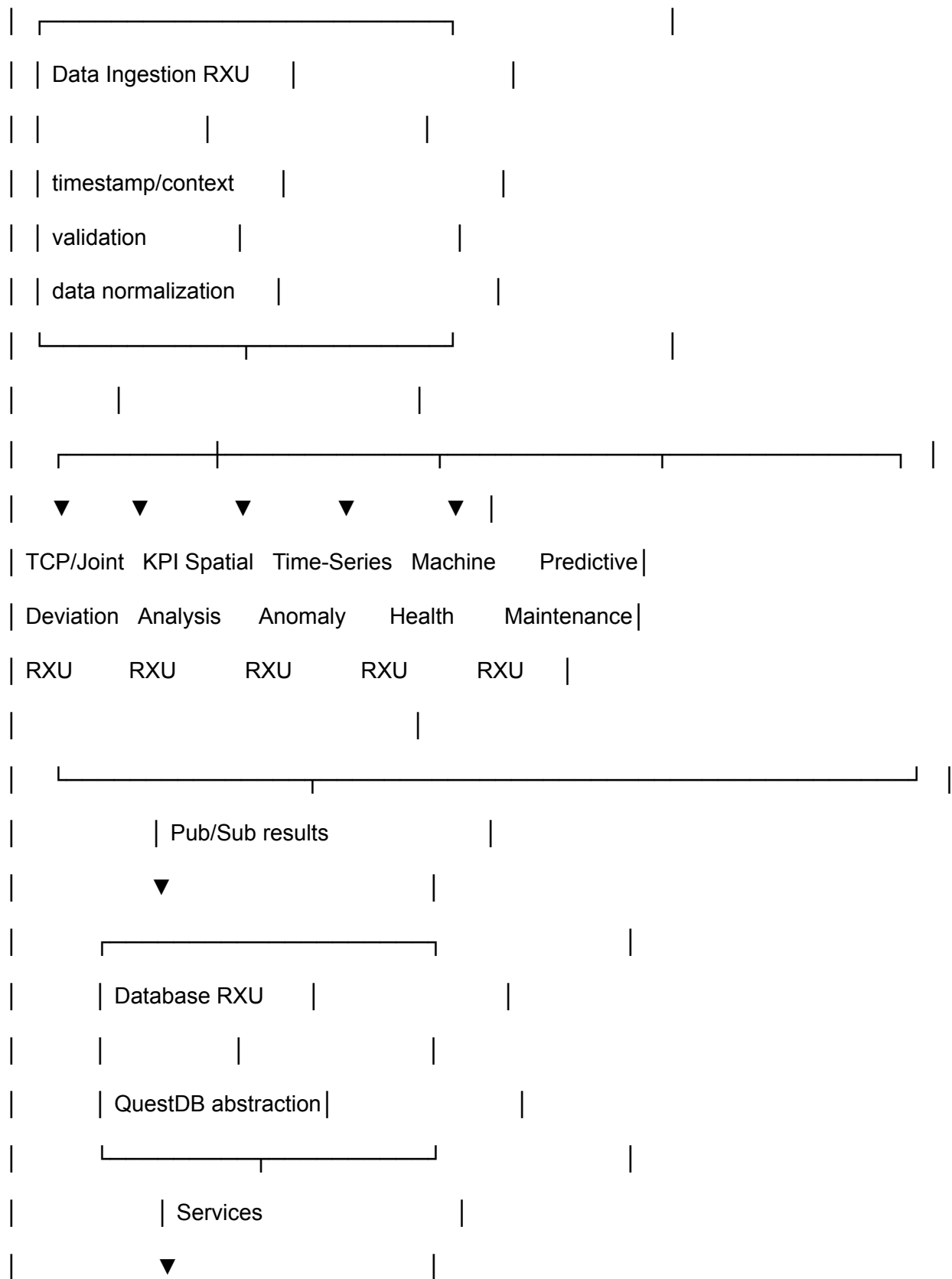
2. Main architectural layers

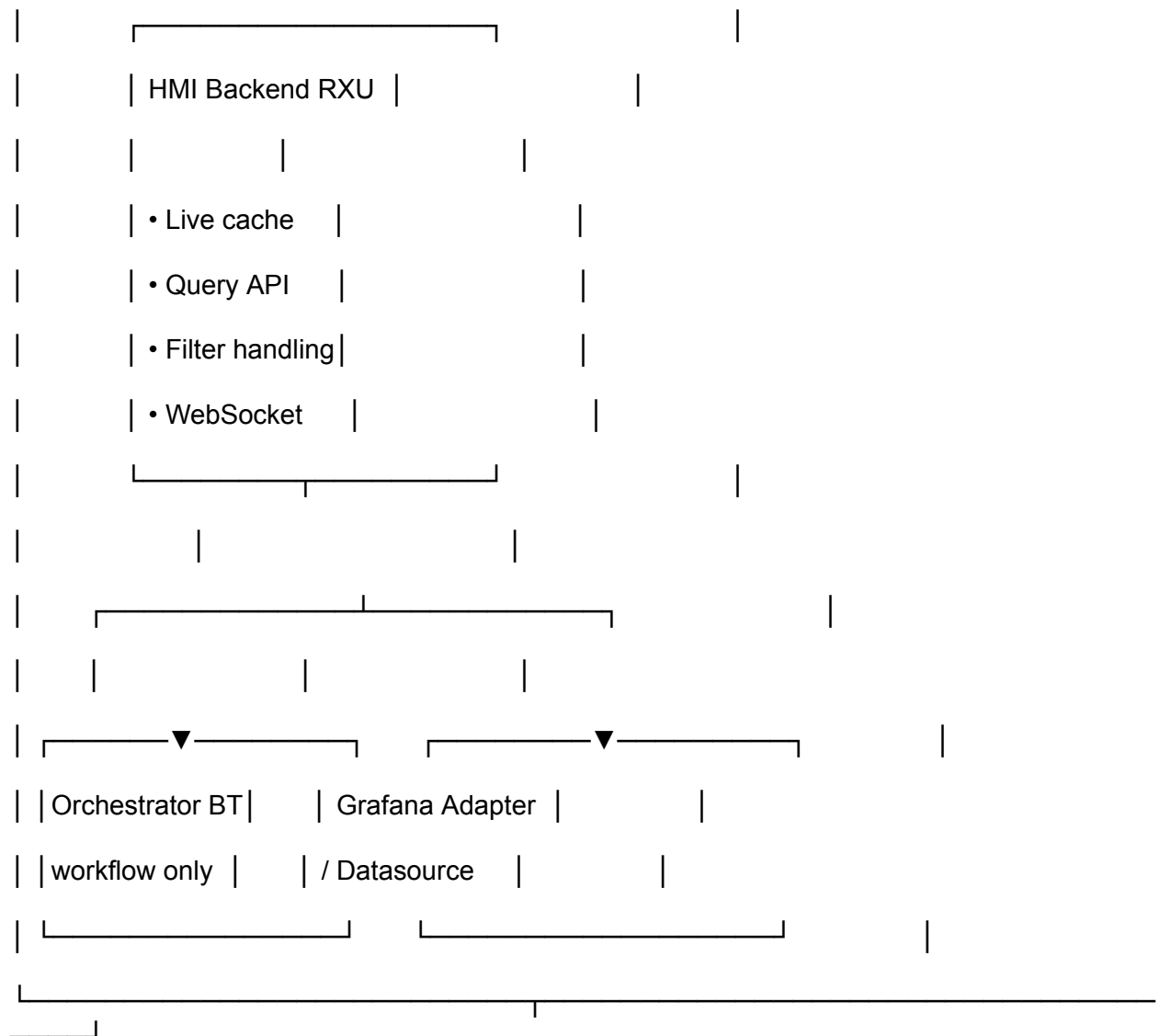
I would organize the system as follows.



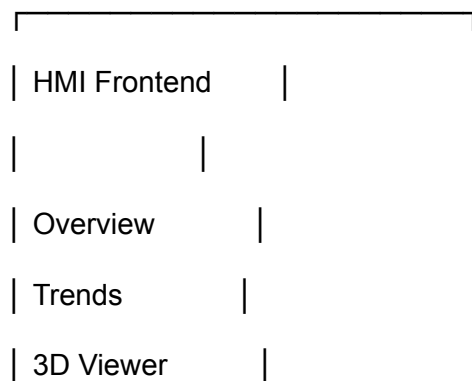
machine/process information

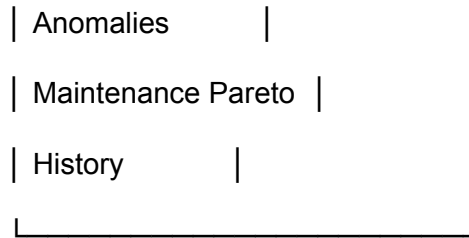






REST / WebSocket





The BT is deliberately **not in the high-frequency data path**.

3. Two communication planes

This distinction is important in RXU.

Data plane

High-frequency or continuously generated information uses **RXU Pub/Sub**.

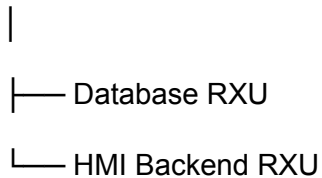
Connectivity



Analysis RXU



Result Topic



Examples are actual TCP samples, KPI samples, deviation results, anomaly events and machine-health measurements.

Control/query plane

Requests for bounded information or workflow actions use **RXU Service/Client**.

Client RXU

|

| Request



Service RXU

|

| Response



Client

Examples are:

GetProgrammedTrajectory

QueryHistoricalDeviation

QueryKPITrend

QueryAnomalies

QueryMachineHealth

QueryPredictiveMaintenance

This separation is important because you should not use service calls for every 10 ms TCP sample.

4. Common identity model

All WP5.3 information should carry a common manufacturing context.

This is particularly important for your filters:

Product ID

Part ID

Run ID

Program ID

Layer ID

Course ID

Strip ID

Tow ID

Machine ID

I would define one reusable protobuf object.

```
syntax = "proto3";
```

```
package wp53.common.v1;
```

```
import "google/protobuf/timestamp.proto";
```

```
message ManufacturingContext {
```

```
    string product_id = 1;
```

```
    string part_id = 2;
```

```
    string run_id = 3;
```

```
    string program_id = 4;
```

```
    string layer_id = 5;
```

```
    string course_id = 6;
```

```
    string strip_id = 7;
```

```
string tow_id = 8;
```

```
string machine_id = 9;
```

```
string process_id = 10;
```

```
}
```

```
message SampleHeader {
```

```
    google.protobuf.Timestamp source_timestamp = 1;
```

```
    google.protobuf.Timestamp rxu_timestamp = 2;
```

```
    uint64 sequence_id = 3;
```

```
    ManufacturingContext context = 4;
```

```
}
```

```
message Vector3 {
```

```
    double x = 1;
```

```
    double y = 2;
```

```
    double z = 3;
```

```
}
```

```
message Quaternion {
```

```
    double x = 1;
```

```
    double y = 2;
```

```
double z = 3;

double w = 4;

}
```

```
message Pose {

  Vector3 position = 1;

  Quaternion orientation = 2;

  string frame_id = 3;

}
```

The `source_timestamp` should preferably represent when the controller/sensor generated the measurement. `rxu_timestamp` can represent when RXU received it.

5. Connectivity RXU interface

Connectivity provides the raw operational state.

Execution state

Communication: Pub/Sub.

```
enum ExecutionState {

  EXECUTION_STATE_UNSPECIFIED = 0;

  IDLE = 1;

  PROGRAM_LOADED = 2;

  RUNNING = 3;

  PAUSED = 4;

  COMPLETED = 5;
```

```
ABORTED = 6;
}

message MachineExecutionState {
    SampleHeader header = 1;

    ExecutionState state = 2;

    uint32 nc_block_id = 3;

    double path_progress_m = 4;
}
```

Conceptual topic:

/rxu/machine/execution_state

This tells the analysis components:

which run

which NC program

which course/layer

whether execution is active

It is **not** a WP5.3 machine-control command.

6. Actual TCP and joint state

Communication: Pub/Sub.

```
message JointState {
```

```
repeated string joint_name = 1;
repeated double position_rad = 2;
repeated double velocity_rad_s = 3;
}
```

```
message ActualMachinePose {
  SampleHeader header = 1;

  Pose actual_tcp = 2;
  JointState actual_joints = 3;

  uint32 nc_block_id = 4;
  double path_progress_m = 5;
}
```

Conceptual topic:

/rxu/machine/actual_state

7. Programmed NC/reference data

Programmed data is different from actual machine streaming data.

Since you are not yet certain whether the NC information is stored as a file or structured in QuestDB, the implementation must remain hidden behind Database RXU.

I recommend **Service/Client**.

TCP Deviation RXU

|

| GetProgrammedTrajectory



Database RXU

|

|— maybe parse NC file

|

|— maybe query QuestDB

|



ProgrammedTrajectory

The caller sees exactly the same response in either case.

```
message ProgrammedTrajectoryRequest {
```

```
    ManufacturingContext context = 1;
```

```
    bool include_tcp = 2;
```

```
    bool include_joints = 3;
```

```
}
```

```
message ProgrammedTrajectoryPoint {
```

```
    uint64 sequence_id = 1;
```

```
    uint32 nc_block_id = 2;
```

```
    double path_progress_m = 3;
```

Pose programmed_tcp = 4;

JointState programmed_joints = 5;

double nominal_feed_rate_m_s = 6;

string course_id = 7;

string strip_id = 8;

string layer_id = 9;

}

message ProgrammedTrajectoryResponse {

ManufacturingContext context = 1;

repeated ProgrammedTrajectoryPoint points = 2;

string trajectory_frame_id = 3;

}

This interface survives both storage possibilities:

NC FILE

|

NC Parser

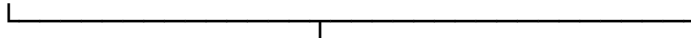
|

QUESTDB TABLE

|

DB query

|



ProgrammedTrajectoryResponse

8. RXU App 1 — TCP and joint deviation

This app runs continuously for an active run.

Its responsibility is:

actual TCP/joints

+

programmed TCP/joints

+

execution context



synchronization/alignment



deviation

It should retrieve the programmed trajectory **once when a new run/program becomes active**, then cache it.

It should not query the database once for every TCP sample.

Deviation output

message JointDeviation {

string joint_name = 1;

double programmed_rad = 2;

```
double actual_rad = 3;  
double deviation_rad = 4;  
}
```

```
message TcpDeviation {  
    Vector3 translation_error_m = 1;  
  
    double position_deviation_m = 2;  
    double orientation_deviation_rad = 3;  
  
    double surface_normal_deviation_m = 4;  
    double tangential_deviation_m = 5;  
}
```

```
message DeviationResult {  
    SampleHeader header = 1;  
  
    Pose programmed_tcp = 2;  
    Pose actual_tcp = 3;  
  
    TcpDeviation tcp_deviation = 4;  
  
    repeated JointDeviation joint_deviation = 5;
```

```
uint32 nc_block_id = 6;  
double path_progress_m = 7;  
}
```

Output communication: Pub/Sub.

/rxu/analysis/deviation

Database RXU subscribes and persists.

HMI Backend RXU can also subscribe and keep a short live cache.

9. Deviation on 3D CAD

The frontend should support both TCP and joint deviation.

For TCP:

CAD surface

+

programmed path

+

actual path

+

deviation color

For joint deviation, the value is a joint-space quantity but can still be displayed at the **TCP position where it occurred**.

For example:

Color mode:

- TCP position deviation

- TCP normal deviation
- Joint 1 deviation
- Joint 2 deviation
- Joint 3 deviation

...

The actual CAD mesh is not required in every protobuf sample. The message only needs:

position

path progress

context

deviation

The frontend/backend maps that information to the loaded CAD geometry.

10. RXU App 2 — KPI trend and spatial analysis

This is explicitly **not anomaly detection**.

It answers questions such as:

How did tow tension behave over Course 7?

Where on the panel was the compaction force high?

How did temperature vary over the layup path?

Connectivity publishes the process values.

```
enum KpiType {
```

```
    KPI_TYPE_UNSPECIFIED = 0;
```

```
TOW_TENSION = 1;
COMPACTION_FORCE = 2;
TEMPERATURE = 3;
PRESSURE = 4;
PROCESS_SPEED = 5;
}
```

```
message KpiSample {
    SampleHeader header = 1;

    KpiType kpi_type = 2;

    double value = 3;
    string unit = 4;

    Pose tcp_pose = 5;

    double path_progress_m = 6;
}
```

Communication from Connectivity: Pub/Sub.

/rxu/process/kpi

The KPI Spatial Analysis RXU subscribes and calculates descriptive information such as moving values, min/max/mean, range and spatial mapping.

```
message KpiSpatialPoint {
    SampleHeader header = 1;
```

```
KpiType kpi_type = 2;
```

```
double value = 3;
```

```
string unit = 4;
```

```
Pose spatial_pose = 5;
```

```
double path_progress_m = 6;
```

```
double moving_average = 7;
```

```
}
```

```
message KpiTrendSummary {
```

```
    ManufacturingContext context = 1;
```

```
    KpiType kpi_type = 2;
```

```
    double minimum = 3;
```

```
    double maximum = 4;
```

```
    double mean = 5;
```

```
    double standard_deviation = 6;
```

```
    uint64 sample_count = 7;
```

```
}
```

Output:

/rxu/analysis/kpi_spatial

/rxu/analysis/kpi_summary

Both use Pub/Sub.

11. Combined deviation and KPI visualization

This is directly relevant to your frontend requirements.

The user should have something conceptually equivalent to:

3D Process View

Geometry

[✓] CAD

Trajectory

[✓] Programmed path

[✓] Actual path

Spatial quantity

Primary overlay:

[Tow Tension ▼]

Secondary path overlay:

[TCP Deviation ▼]

Filters:

Product: [P01]

Part: [Panel-17]

Run: [RUN-025]

Layer: [L03]

Course: [C07]

Strip: [S04]

Tow: [T12]

The user should be able to select:

Deviation only

KPI only

Deviation + KPI

For the combined mode, I would avoid painting two independent heatmaps onto the same surface.

A clearer visualization is:

CAD surface color

=

selected KPI

Actual trajectory color / markers

=

TCP deviation

For example:

CAD heatmap → Tow Tension

TCP path color → TCP deviation

This gives the engineer two dimensions of information without visually confusing two color scales.

12. Filters

I recommend a common backend filtering schema.

```
message AnalysisFilter {
```

```
    string product_id = 1;
```

```
    string part_id = 2;
```

```
    string run_id = 3;
```

```
    string program_id = 4;
```

```
    repeated string layer_id = 5;
```

```
    repeated string course_id = 6;
```

```
    repeated string strip_id = 7;
```

```
    repeated string tow_id = 8;
```

```
    google.protobuf.Timestamp start_time = 9;
```

```
    google.protobuf.Timestamp end_time = 10;
```

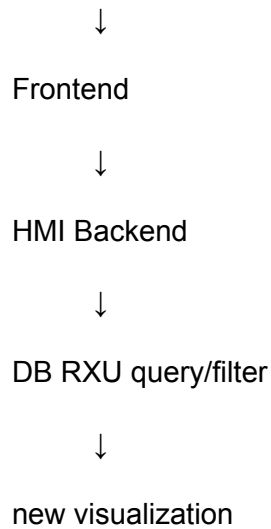
```
    repeated KpiType kpi_type = 11;
}
```

The frontend communicates the equivalent as JSON to HMI Backend.

HMI Backend then converts it to an RXU protobuf service request for Database RXU.

A filter change should **not restart the analysis apps**.

User changes Tow 12 → Tow 15



The continuous calculations continue unchanged.

13. RXU App 3 — Time-Series Process Anomaly

This is distinctly different from KPI Trend Analysis.

The KPI Spatial App asks:

What is happening to the KPI?

The Time-Series Anomaly App asks:

Is the KPI behaving abnormally?

It subscribes to:

/rxu/process/kpi

and performs time-series analysis such as trend/drift detection, spikes, threshold violations, oscillation, statistical outliers or other selected methods.

A suitable message is:

```
enum AnomalySeverity {  
  
    ANOMALY_SEVERITY_UNSPECIFIED = 0;  
  
    INFORMATION = 1;  
  
    WARNING = 2;  
  
    CRITICAL = 3;  
  
}
```

```
enum AnomalyType {  
  
    ANOMALY_TYPE_UNSPECIFIED = 0;  
  
  
  
    THRESHOLD_VIOLATION = 1;  
  
    SPIKE = 2;  
  
    DROP = 3;  
  
    DRIFT = 4;  
  
    OSCILLATION = 5;  
  
    STATISTICAL_OUTLIER = 6;  
  
}
```

```
message ProcessAnomaly {  
  
    SampleHeader header = 1;
```

string anomaly_id = 2;

KpiType kpi_type = 3;

double actual_value = 4;

double reference_value = 5;

AnomalyType anomaly_type = 6;

AnomalySeverity severity = 7;

google.protobuf.Timestamp start_time = 8;

google.protobuf.Timestamp end_time = 9;

Pose spatial_pose = 10;

string description = 11;

}

Output:

/rxu/analysis/process_anomaly

Communication: Pub/Sub.

Database RXU subscribes and persists the events.

14. Grafana for anomaly visualization

For the process-anomaly use case, I would use **Grafana for detailed time-series engineering analysis**.

The dashboard can show:

Tow tension against time

actual value

normal/threshold band

anomaly marker

severity

course ID

tow ID

run ID

You can retain the custom HMI anomaly list/timeline from your frontend requirements and use Grafana as the detailed engineering drill-down.

Conceptually:

HMI

|

| "Open detailed trend"

▼

Grafana

Run = RUN025

Course = C07

Tow = T12

Time = anomaly_time ± 30 sec

For integration there are two choices.

The simpler engineering implementation is:

QuestDB

↓

Grafana

with a read-only Grafana connection.

If your architectural rule requires **all QuestDB access to remain abstracted**, use:

Grafana

↓

Grafana datasource/API adapter

↓

Database RXU

↓

QuestDB

I prefer the second for strict architectural consistency, though it involves more implementation.

15. RXU App 4 — Machine Health

The Machine Health RXU App should consume machine/component-health information from Connectivity.

Example:

```
enum HealthState {  
  
    HEALTH_STATE_UNSPECIFIED = 0;  
  
    HEALTHY = 1;
```

```
    DEGRADED = 2;
    WARNING_STATE = 3;
    FAULT = 4;
}
```

```
message MachineHealthSample {
    SampleHeader header = 1;

    string component_id = 2;
    string component_type = 3;

    HealthState health_state = 4;

    string fault_code = 5;
    string fault_category = 6;

    double health_score = 7;
}
```

Connectivity publishes:

/rxu/machine/health

Machine Health App subscribes and produces structured health events and aggregation information.

```
message MachineHealthEvent {
    SampleHeader header = 1;
```

```
string event_id = 2;
string component_id = 3;

HealthState state = 4;

string failure_category = 5;
string fault_code = 6;

double duration_s = 7;
}
```

Output:

/rxu/analysis/machine_health

Database RXU stores it.

16. Maintenance Pareto Dashboard

This fits your frontend requirement for the **Maintenance Pareto Dashboard** very well.

The Pareto view is better calculated from the stored health events than generated from every incoming machine sample.

For example:

Failure category	Count
Motor alarm	43
Tow feeder	31
Temperature	18

Vacuum 9

Communication 5

with cumulative percentage:

Motor  41%

Tow Feeder  70%

Temp  87%

Vacuum  96%

Comm  100%

The user selects Maintenance/Pareto.

Frontend sends:

time range

machine

component

severity

HMI Backend queries Database RXU.

```
message HealthParetoRequest {
```

```
    AnalysisFilter filter = 1;
```

```
    repeated string component_id = 2;
```

```
    repeated HealthState health_state = 3;
```

```
}
```

```
message ParetoEntry {
```

```
    string category = 1;
```

```
uint64 count = 2;

double duration_s = 3;


double percentage = 4;

double cumulative_percentage = 5;
}

message HealthParetoResponse {
  repeated ParetoEntry entries = 1;
}
```

This is a **Service/Client** interface because Pareto is a bounded query based on user filters.

17. RXU App 5 — Dummy Predictive Maintenance

Your dummy predictive-maintenance RXU App can consume:

machine health

KPI history

fault history

operating time

and generate a synthetic or prototype prediction.

Example protobuf:

```
message PredictiveMaintenanceResult {
```

```
SampleHeader header = 1;
```

```
string component_id = 2;
```

```
double failure_probability = 3;
```

```
double health_score = 4;
```

```
double estimated_remaining_hours = 5;
```

```
string prediction_model_id = 6;
```

```
string recommendation = 7;
```

```
}
```

Output:

```
/rxu/analysis/predictive_maintenance
```

Communication: Pub/Sub if periodically updated.

For historical prediction retrieval:

```
message PredictiveMaintenanceQuery {
```

```
    AnalysisFilter filter = 1;
```

```
    repeated string component_id = 2;
```

```
}
```

Use Database RXU **Service/Client**.

Grafana can then provide a dashboard such as:

Health score

Failure probability

Estimated remaining useful life

Historical trend

Component comparison

18. HMI Backend responsibility

HMI Backend must **not perform the engineering calculations**.

The calculations remain in the RXU Apps.

Its responsibilities are:

receive live results

maintain short live cache

query historical results through DB RXU

apply user filters

convert protobuf → frontend JSON

expose REST APIs

expose WebSocket streams

handle frontend sessions

A useful conceptual cache is:

HMI Backend

Live Deviation Cache

Live KPI Cache

Latest Anomaly Cache

Machine Health Cache

Predictive Maintenance Cache

This is not persistent storage.

Persistent storage belongs to Database RXU/QuestDB.

19. Frontend communication

Because the frontend itself is not RXU, I would not expose RXU protobuf directly to the browser.

Use:

Frontend

|

|— REST

| historical/query/configuration

|

|— WebSocket

live updates

|

▼

HMI Backend RXU

For example:

GET /api/runs/{run}/deviation

GET /api/runs/{run}/kpi

GET /api/anomalies

GET /api/health/pareto

WS /live/deviation

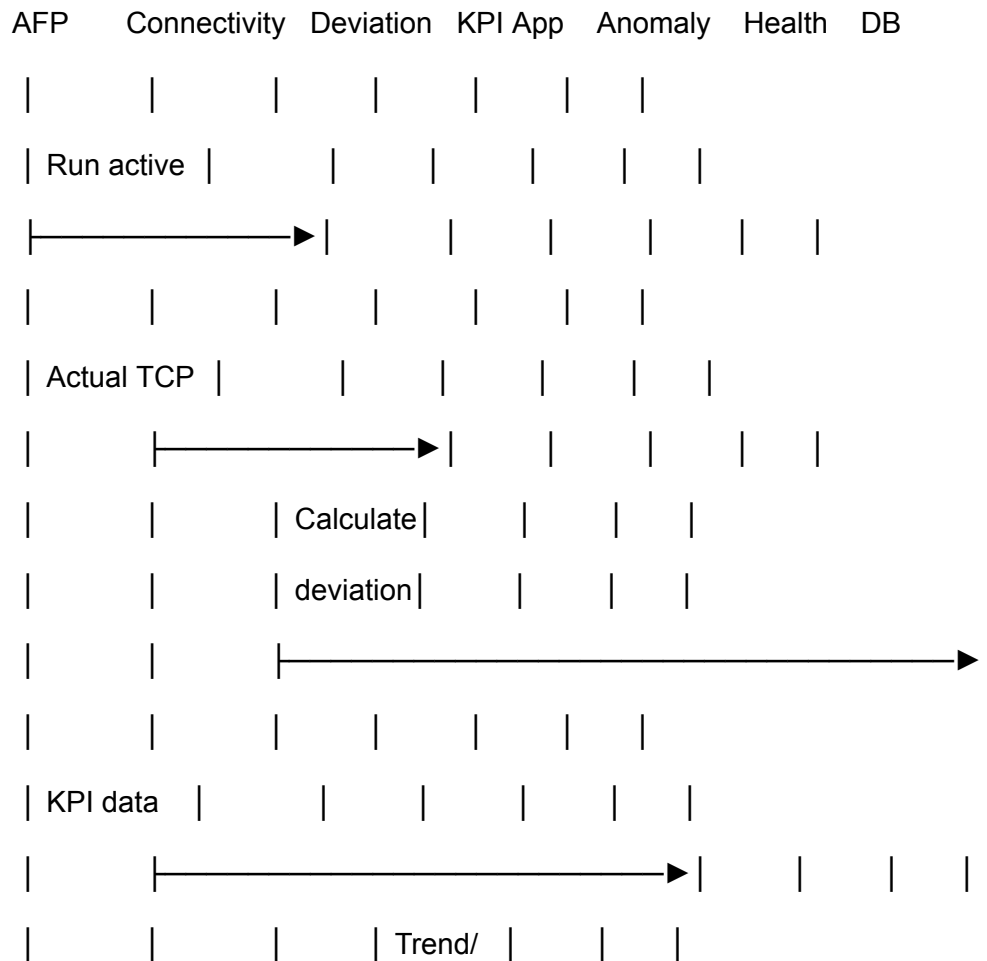
WS /live/kpi

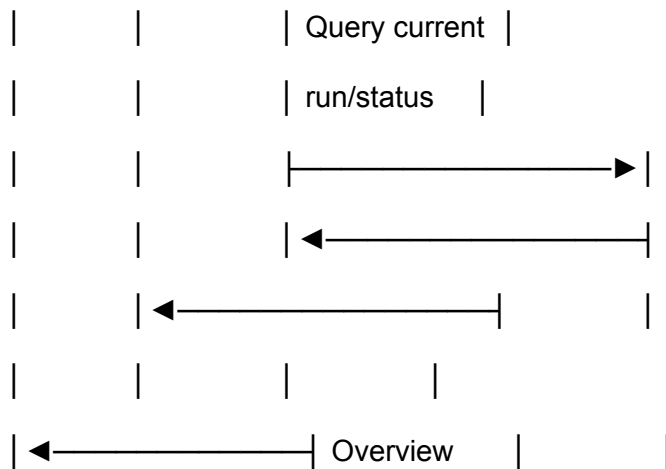
WS /live/anomaly

The HMI Backend performs the RXU ↔ web translation.

20. Sequence 1 — system running, user does NOT click anything

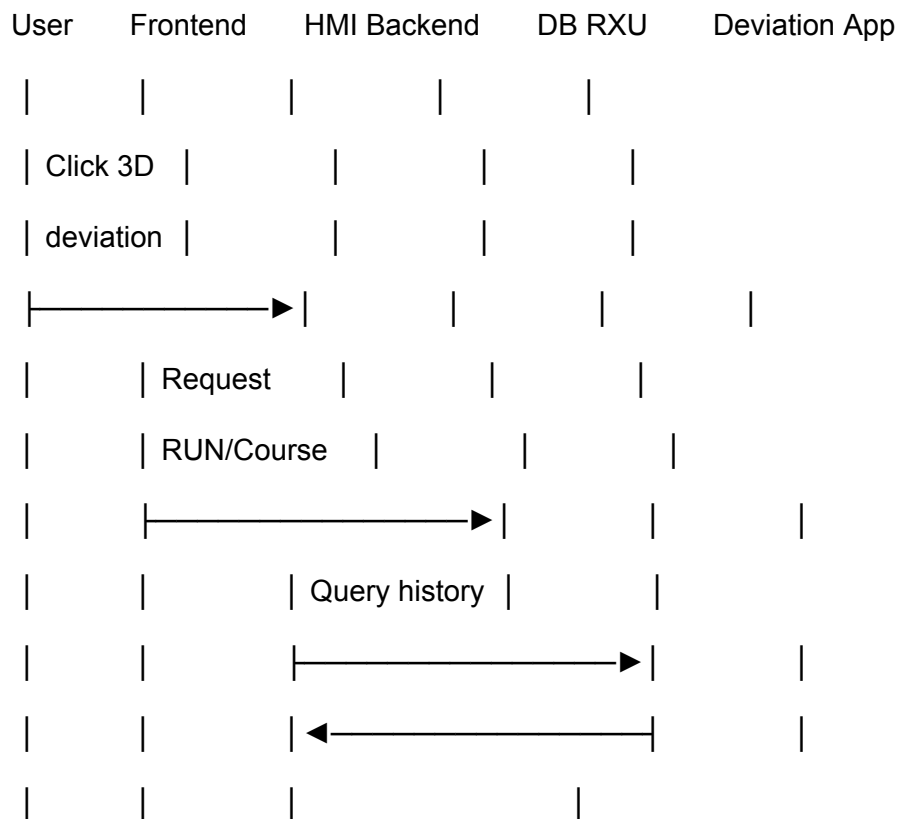
This sequence is crucial because it establishes that analysis continues independently.

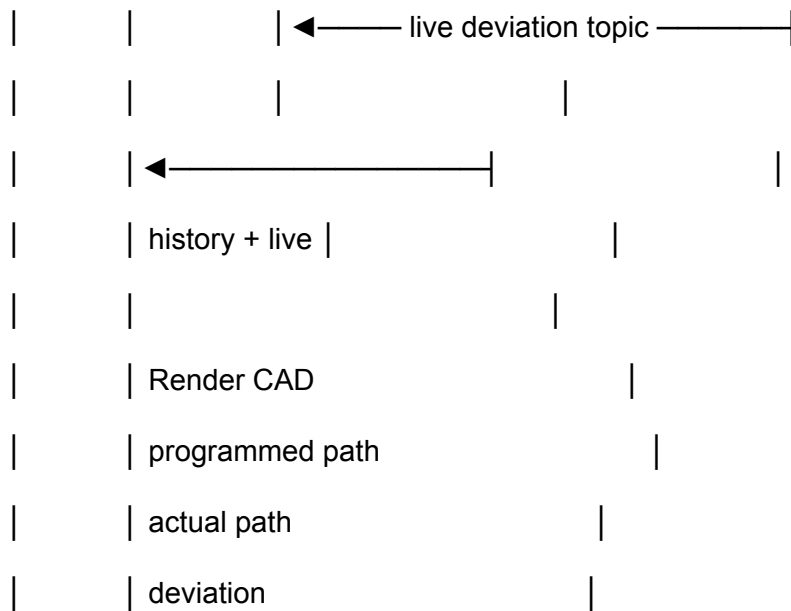




Meanwhile all RXU analyses continue in parallel.

22. Sequence 3 — user opens TCP/joint deviation 3D view





The important observation is:

The click did **not** start the Deviation RXU App.

The app was already calculating.

The click starts **visualization delivery**.

23. Sequence 4 — user selects KPI only

User

|
| 3D overlay = Tow Tension

▼

Frontend

|
| filter:
| run / layer / course / strip / tow



HMI Backend



| historical query



DB RXU



| KpiSpatialPoint[]



HMI Backend



| + live KPI stream



Frontend



CAD KPI heatmap

No BT required.

24. Sequence 5 — user selects deviation + KPI together

User



| Enable:

|

| [✓] KPI spatial overlay

| [✓] TCP deviation



Frontend

|

| VisualizationRequest



HMI Backend

|

|—————▶ DB RXU

| KPI history

| deviation history

|

|————— receives live KPI

|

|————— receives live deviation

|



Frontend

|

|—— CAD surface = selected KPI

|

└─ TCP path = deviation scale

The two datasets are correlated using:

Run ID

Course ID

Layer ID

Strip ID

time/path position

and Tow ID when applicable.

25. Sequence 6 — user changes Tow/Course/Layer

Changing filters should not start any RXU analysis.

User

|

| Course C07 → C08

▼

Frontend

|

| new filter

▼

HMI Backend

|

| Query



DB RXU



| filtered result



HMI Backend



Frontend

The same applies to:

Tow ID

Layer ID

Strip ID

Part ID

Product ID

Run ID

time range

26. Sequence 7 — anomaly detected while user is NOT on anomaly page

Connectivity



| KPI samples



Time-Series Anomaly RXU



| window analysis



| anomaly detected



ProcessAnomaly



|—————▶ Database RXU



|—————▶ HMI Backend

If the user is not looking at that dashboard, processing still continues.

Optionally, the overview can show:

Active anomalies: 3

Critical: 1

Warning: 2

without opening Grafana.

27. Sequence 8 — user opens anomaly dashboard

User



| Open Process Anomaly



Frontend



| selected run/course/tow/time



HMI Backend / Grafana Adapter



Database RXU



QuestDB



Anomaly time-series data



Grafana

Grafana renders:

KPI trend

normal/threshold range

anomaly timestamps

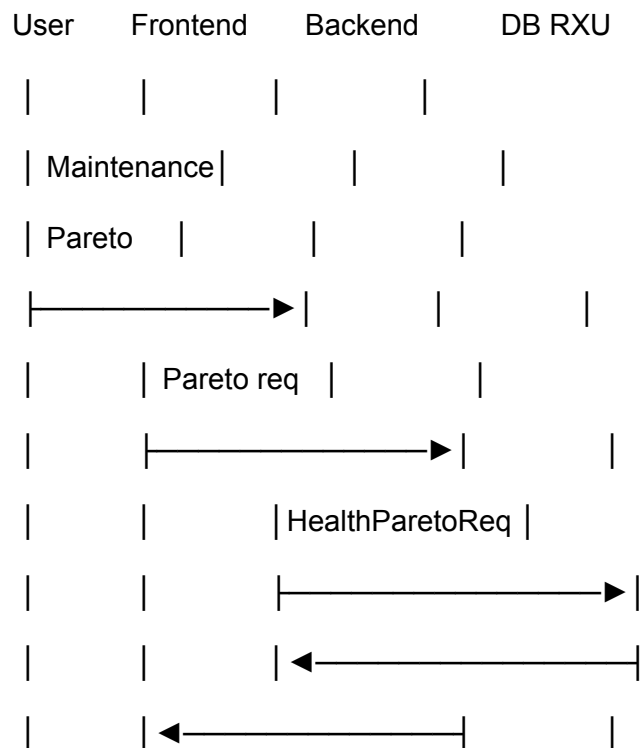
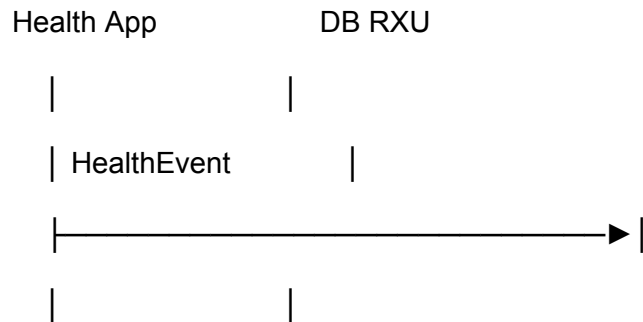
severity

tow/course filters

Anomaly computation remains in the Time-Series RXU App.

Grafana does **not** detect anomalies.

28. Sequence 9 — Machine Health / Pareto



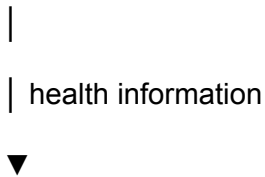


The frontend requirement **Maintenance Pareto Dashboard** fits this design directly.

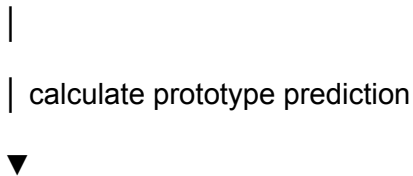
29. Sequence 10 — Dummy predictive maintenance

The dummy predictive-maintenance RXU App runs periodically or whenever new health data arrives.

Machine Health RXU



Predictive Maintenance RXU



PredictiveMaintenanceResult



Database RXU

Grafana can continuously display:

component health score

failure probability

remaining-hours estimate

prediction history

The user does not have to start prediction by opening Grafana.

30. Where Behaviour Tree belongs

I would make the distinction very explicit in your architecture document.

BT is **not**:

data router

topic bridge

high-frequency sample handler

HMI data proxy

BT is:

workflow coordinator

run lifecycle coordinator

multi-component analysis coordinator

error/retry/fallback coordinator

For example:

AFP run detected

↓

BT

|

├— validate services available

├— establish run context

├— ensure programmed reference available

- └─ monitor component health
- └─ finalize run

For a complex on-demand root-cause workflow:

User requests RCA



Backend



BT



- └─ retrieve selected event
- └─ retrieve surrounding KPIs
- └─ run behavior model
- └─ run correlation
- └─ prepare investigation result

That is a good use of the BT.

31. If your architecture rules require every HMI action to pass through BT

There is a compromise.

You can do:

User clicks Deviation



Frontend

↓

HMI Backend

↓

BT

```
"ActivateDeviationView(  
    user/session,  
    run,  
    filters  
)"
```

BT can validate:

run exists?

services healthy?

user selection valid?

and respond:

ViewReady

But after that:

Deviation RXU —————▶ HMI Backend —————▶ Frontend

The deviation stream still does **not** travel through BT.

Therefore:

BT = control plane

RXU topics = data plane

This is the most important separation to maintain.

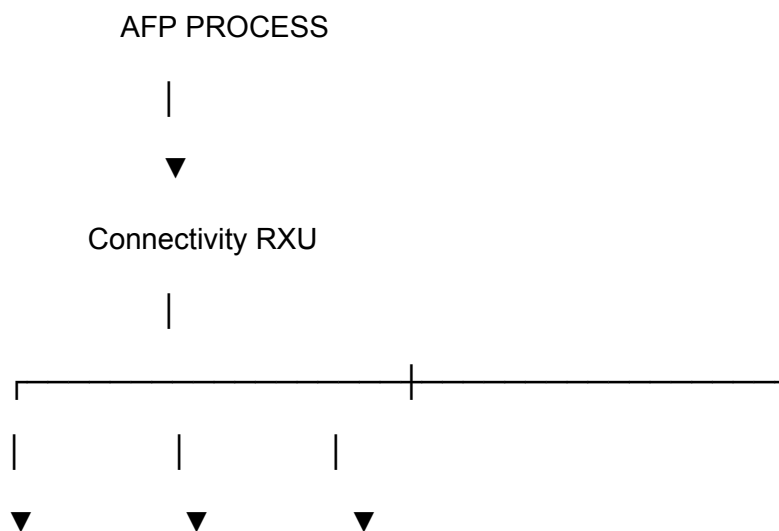
32. Interface summary

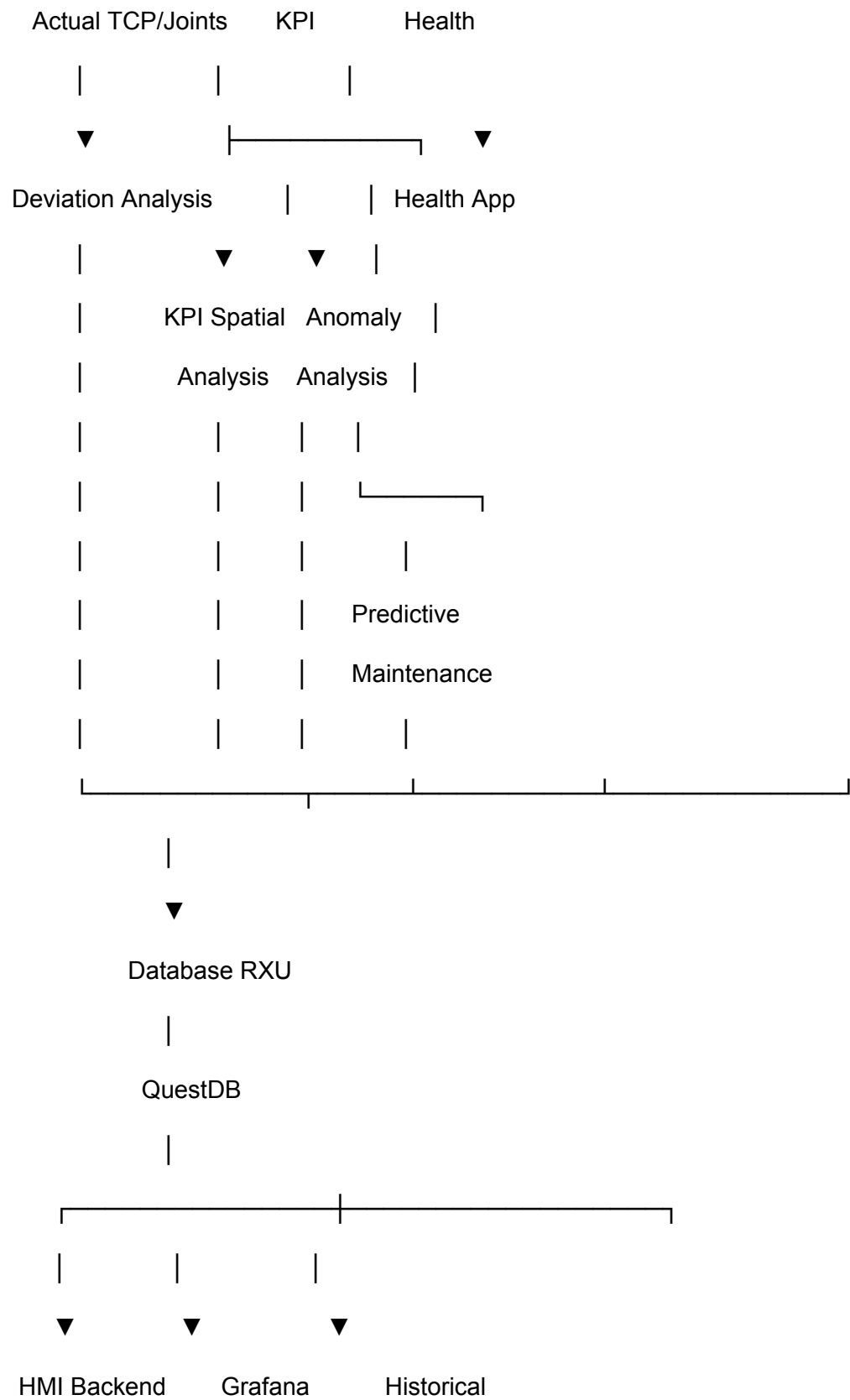
Source	Destination	Interface	Payload
Connectivity RXU	Deviation App	Pub/Sub	Actual TCP/joints
Connectivity RXU	KPI App	Pub/Sub	Process KPI samples
Connectivity RXU	Anomaly App	Pub/Sub	KPI samples
Connectivity RXU	Health App	Pub/Sub	Machine health
Connectivity RXU	BT	Pub/Sub	Execution state
Deviation App	Database RXU	Pub/Sub	DeviationResult
KPI App	Database RXU	Pub/Sub	KpiSpatialPoint
Anomaly App	Database RXU	Pub/Sub	ProcessAnomaly
Health App	Database RXU	Pub/Sub	MachineHealthEvent
Predictive App	Database RXU	Pub/Sub	PredictiveMaintenanceResult
Deviation App	Database RXU	Service/Client	GetProgrammedTrajectory

HMI Backend	Database RXU	Service/Client	Historical/filter queries
HMI Backend	Deviation/KPI/etc.	Pub/Sub subscription	Live result stream
Frontend	HMI Backend	REST	Historical/query/filter
HMI Backend	Frontend	WebSocket	Live updates
Grafana adapter	Database RXU	Service/Client	Time-series queries
HMI Backend	BT	Service/Client	Complex workflow requests

33. Recommended full runtime model

The complete runtime picture is therefore:





|



HMI Frontend

|

|— Overview

|— KPI Trend

|— 3D CAD

| |— programmed path

| |— actual path

| |— TCP/joint deviation

| |— KPI spatial overlay

|

|— Anomaly

|— Machine Health

|— Maintenance Pareto

|— Historical views

Final architectural recommendation

For your three main process-monitoring calculations plus the machine-health and predictive-maintenance apps, I would freeze the architecture around this rule:

RXU applications own continuous engineering computation. Database RXU owns persistent time-series/reference storage and abstracts QuestDB. HMI Backend owns filtering, live caching and browser delivery. HMI Frontend owns visualization. Grafana owns specialized time-series dashboards. Behaviour Tree owns workflow orchestration, not high-frequency data transport.

For the **3D view**, allow the user to independently switch on/off programmed TCP, actual TCP, TCP/joint deviation and KPI spatial information, with Product → Part → Run → Layer → Course → Strip → Tow → Time as hierarchical filters. For simultaneous KPI + deviation visualization, I

recommend **KPI as the CAD surface heatmap and deviation as a colored path/marker overlay**, because two competing surface heatmaps would be difficult to interpret.

This structure also aligns very well with the frontend requirements in your screenshots: the 3D geometry viewer, planned/actual views, layer controls, KPI overlay selection, anomaly list/timeline, historical replay, filtering and the Maintenance Pareto Dashboard all fit without coupling the calculations to whichever page the user happens to have open.

can you give the detailed sequence diagram between HMI front end, back end (RXU app), and BT (RXU app) when user clicks for anything (as per HMI front end requirements)

also usually when we turn i=on the system (IPC) , BT will initialize all nodes and user will login to the HMI and what is next , continously start monitoring things on HMI or waiting for some inputs from RXU connectivty, if yes what input ?

Yes. With the architecture you have described, I would make the startup and HMI behavior **state-driven** rather than “login-driven.”

The key point is:

IPC startup initializes the RXU system. HMI login initializes a user session. Neither of those means an AFP manufacturing run has started. Live process monitoring becomes active only when RXU Connectivity provides sufficient execution context indicating that a valid AFP run/program is active.

That gives a much cleaner Behaviour Tree and sequence diagram.

1. What happens when the IPC is switched on?

I would model the initial sequence as:

IPC ON

|



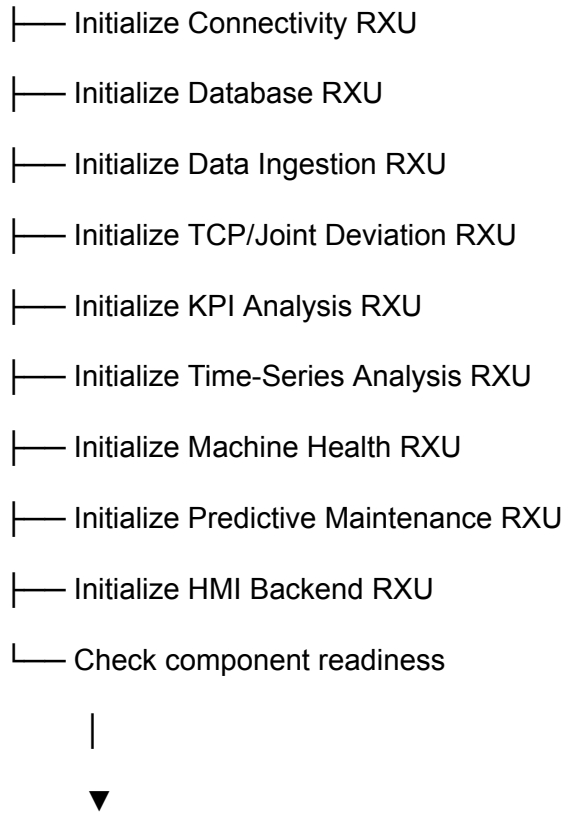
RXU Platform Starts

|



Behaviour Tree / Orchestrator Starts

|



SYSTEM_READY

At this stage the applications are **running**, but they may not yet be processing an AFP run.

For example:

Deviation App = READY, waiting for valid run data

KPI App = READY, waiting for KPI data

Anomaly App = READY, waiting for KPI stream

Health App = ACTIVE

Predictive App = READY/ACTIVE

HMI Backend = READY

Database RXU = READY

Connectivity RXU = READY

The distinction between **READY** and **ACTIVE_ANALYSIS** is useful.

2. What should HMI show immediately after login?

After login, I would **not show blank pages**, but I also would not pretend that an AFP run is active.

The HMI can immediately show system information such as:

System Status READY

Connectivity Connected

Database Connected

Analysis Apps Ready

Machine IDLE

Active Run None

Current Product -

Current Part -

Current Layer -

Current Course -

Current Strip -

Machine Health Available

Last Completed Run RUN_024

So the HMI itself is active immediately.

But live TCP deviation/KPI spatial analysis waits for an active manufacturing context.

3. What input are we waiting for from Connectivity RXU?

This is the critical input.

I would expect Connectivity RXU to publish a machine/execution context message containing at least:

Execution State

Run ID

Program ID

Product ID

Part ID

Machine ID

timestamp

and, when applicable:

Layer ID

Course ID

Strip ID

Tow ID

NC block / path progress

Conceptually:

```
message ManufacturingExecutionState {
```

```
    Timestamp timestamp;
```

string machine_id;

string product_id;

string part_id;

string run_id;

string program_id;

string layer_id;

string course_id;

string strip_id;

ExecutionState state;

uint32 active_nc_block;

double path_progress;

}

with:

IDLE

PROGRAM_LOADED

RUNNING

PAUSED

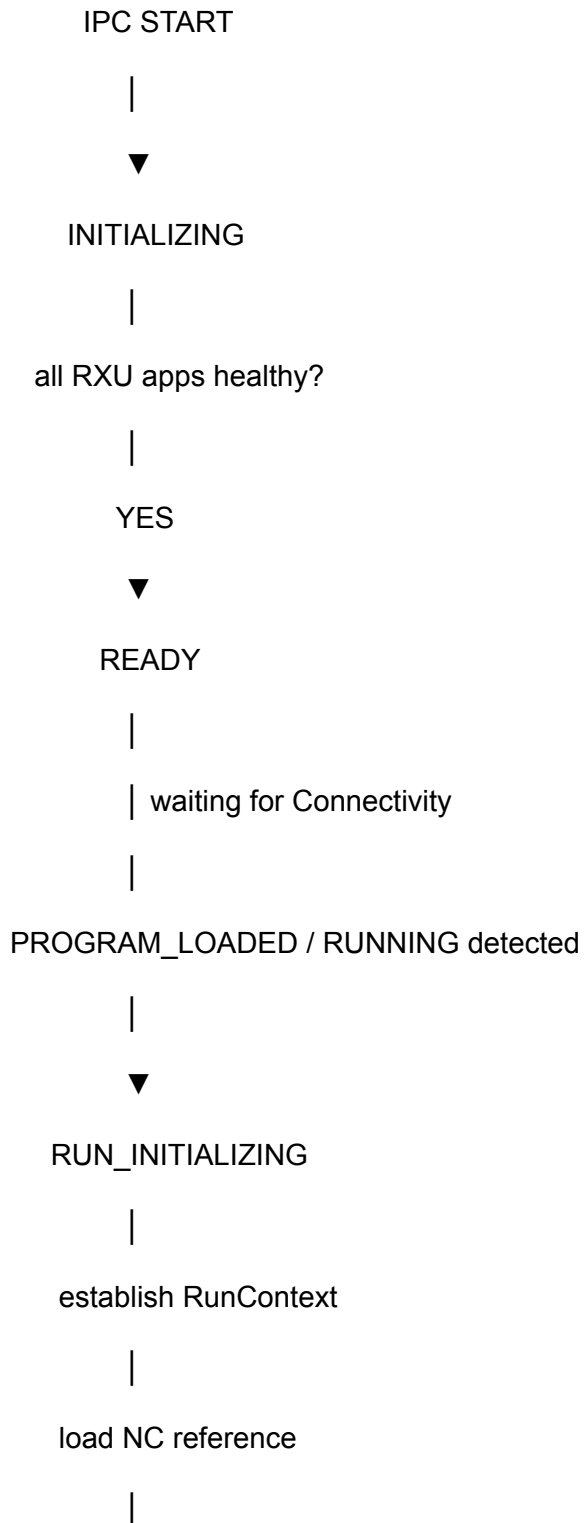
COMPLETED

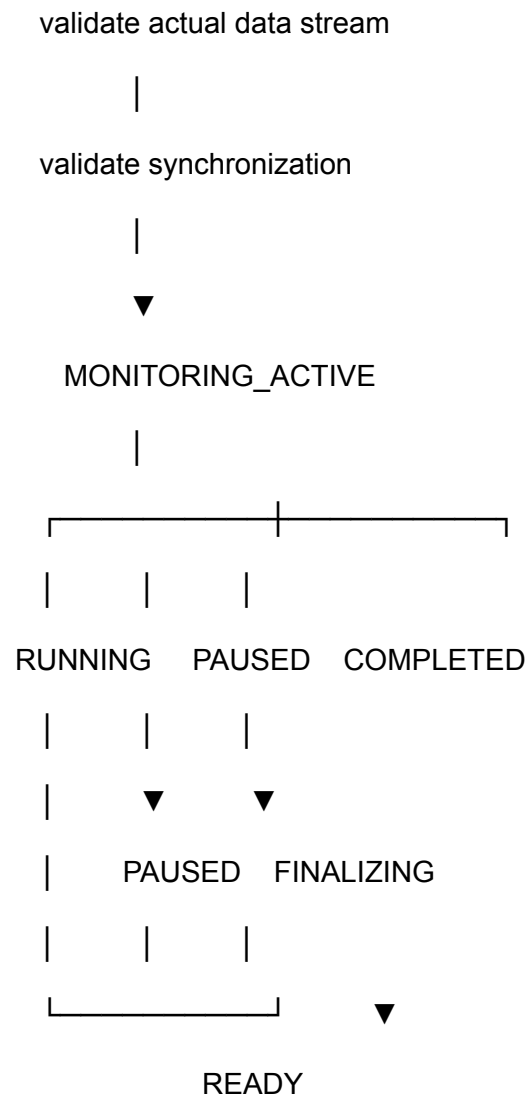
ABORTED

This message is the **main trigger for the BT manufacturing-monitoring lifecycle.**

4. Recommended startup state machine

I would give the BT approximately this lifecycle:

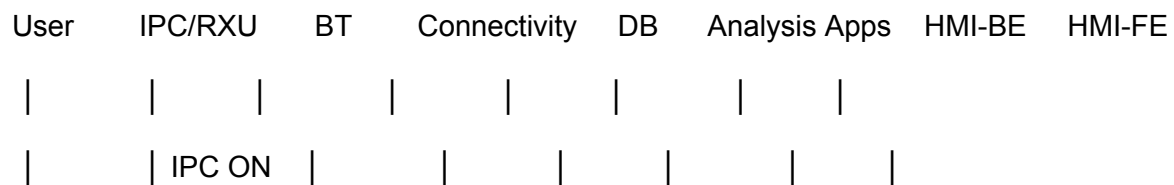




This lifecycle has nothing to do with which HMI page is currently open.

5. Full startup sequence

Here is the sequence I would put in the architecture documentation.



historical runs

previous anomalies

maintenance Pareto

predictive-maintenance history

even though there is no current AFP run.

6. Then Connectivity detects a new AFP execution

Suppose the operator starts manufacturing outside WP5.3.

Connectivity publishes:

state = RUNNING

run_id = RUN_025

program_id = PANEL_A_17

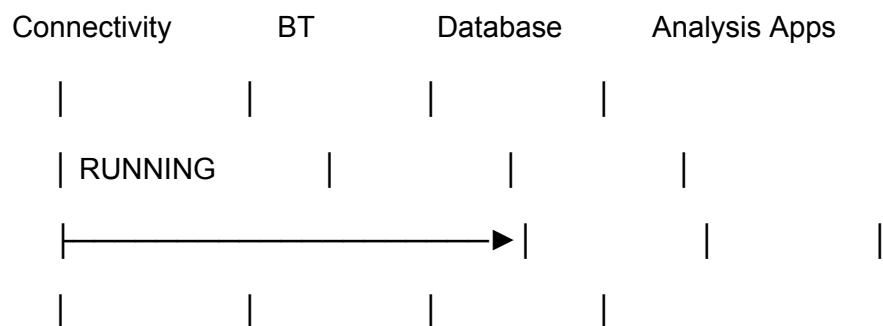
product_id = PRODUCT_04

part_id = PANEL_02

layer_id = L05

course_id = C07

Then:



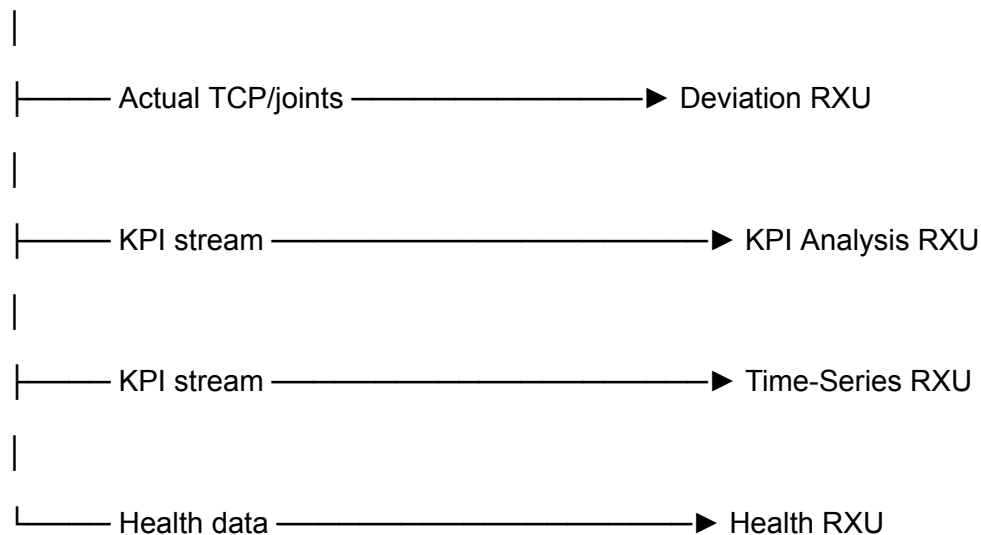
Machine state: RUNNING

The user does **not** need to press Start Monitoring.

7. What continuously happens even if the user clicks nothing?

Once **MONITORING_ACTIVE** exists, these flows happen independently of HMI navigation:

Connectivity



Then:

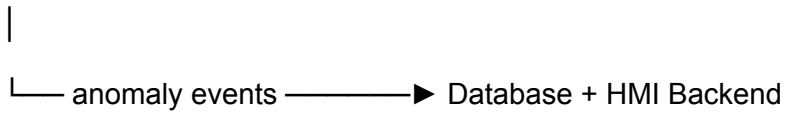
Deviation RXU



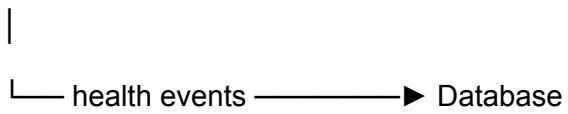
KPI Analysis RXU



Time-Series RXU



Health RXU



Predictive Maintenance RXU



This happens even when:

no user is logged in

or

user is only looking at Overview

or

user is looking at another page

8. Now the important part: what happens when a user clicks something?

I recommend dividing frontend actions into two types.

View/navigation operations

Examples:

Open TCP deviation

Open KPI trends

Open anomaly page

Change Tow ID

Change Course ID

Change Layer

Select KPI

Open Pareto

These normally **do not start the underlying calculation**.

Workflow operations

Examples:

Perform root-cause investigation

Compare Run A vs Run B

Generate a report

Run an offline model

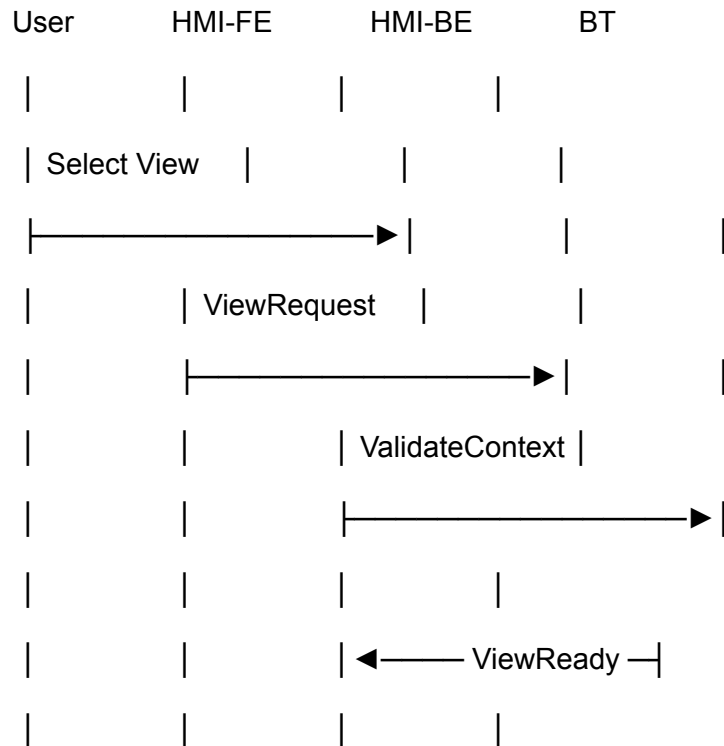
Perform detailed historical analysis

These may require BT orchestration.

This distinction is extremely important.

9. Generic sequence for a frontend click

If you want BT to remain aware of frontend application state, I would use this pattern:



BT can check:

valid run?

system state?

required application healthy?

required data available?

But after this check, **BT exits the data flow.**

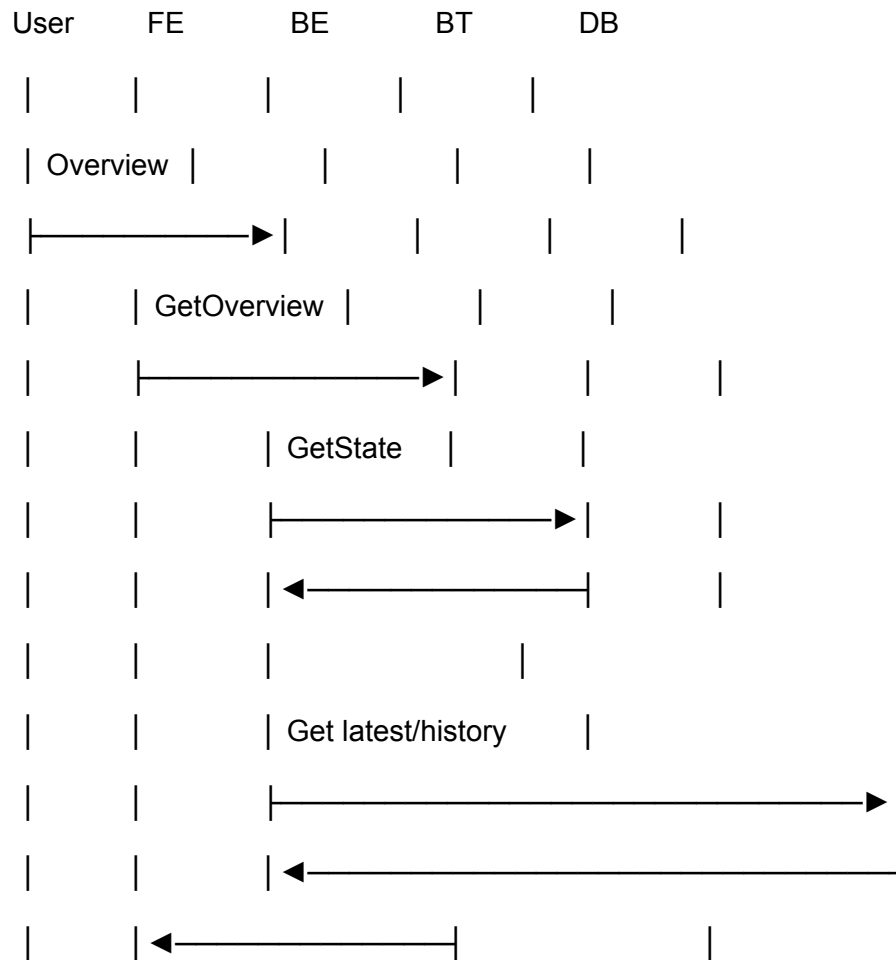
The data path becomes:

Analysis RXU —————> HMI Backend —————> Frontend

not:

10. User clicks Overview

This should be lightweight.



Overview might show:

machine state

run

current product/part

layer/course

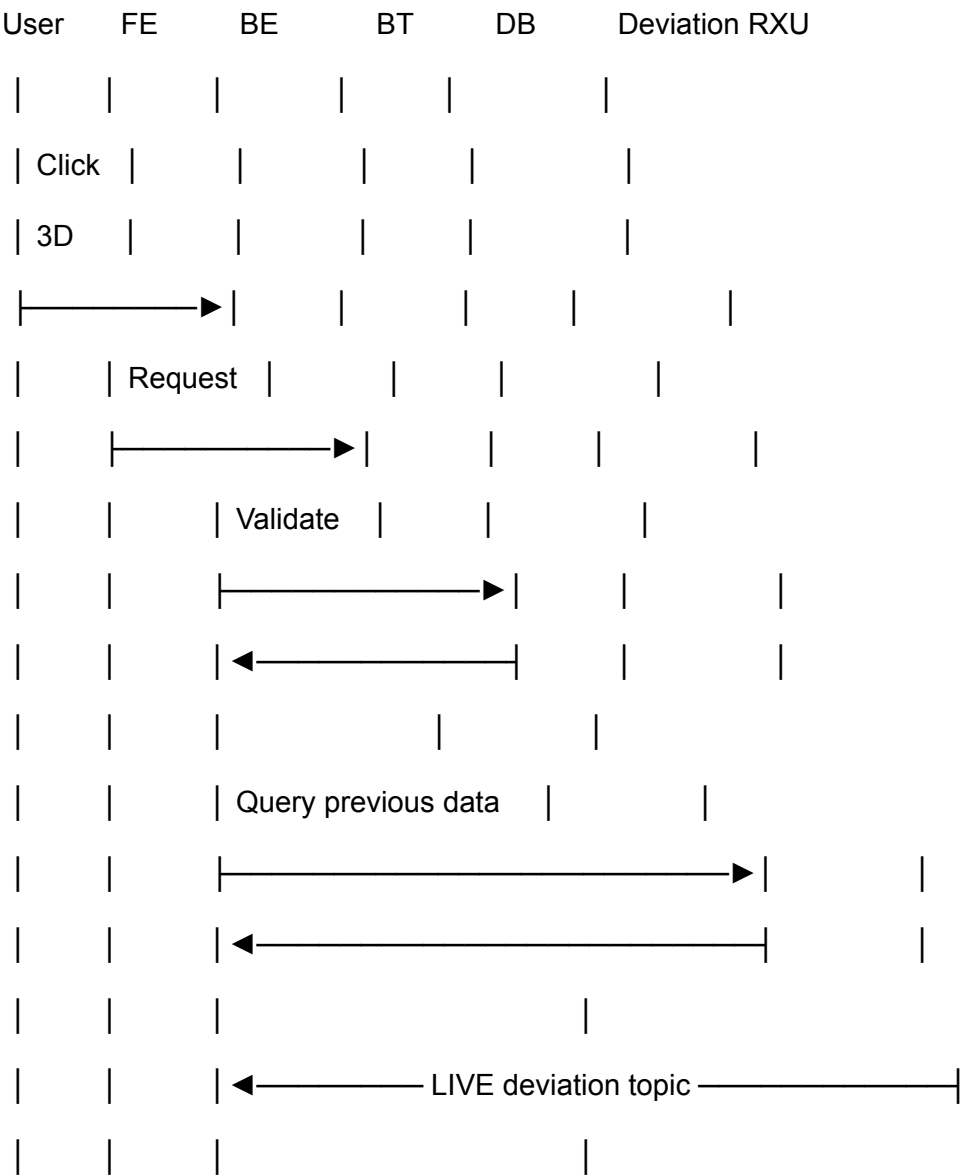
latest KPIs

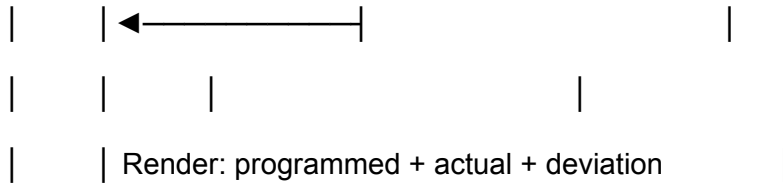
active anomaly count

machine-health summary

11. User clicks 3D Planned vs Actual

This is one of your key sequences.





Again:

The Deviation RXU App did not start because of the click.

It was already continuously calculating.

12. User selects TCP deviation

Inside the 3D page:

User

|
| Metric = TCP Position Deviation



FE

|
| FilterRequest



BE

|
| — select TCP deviation values
| — apply Run/Layer/Course/etc.
| — return geometry-associated values
|



FE

No BT is necessary unless your platform architecture explicitly requires it to validate every request.

13. User selects a joint deviation

For example:

Joint = J3

Sequence:

User → FE → BE

|

| filter:

| metric = JOINT_DEVIATION

| joint = J3

|



DB/live cache

|



result

|



FE

The HMI associates J3 deviation to the corresponding TCP spatial location.

14. User clicks KPI spatial view

Suppose the user wants:

Tow tension

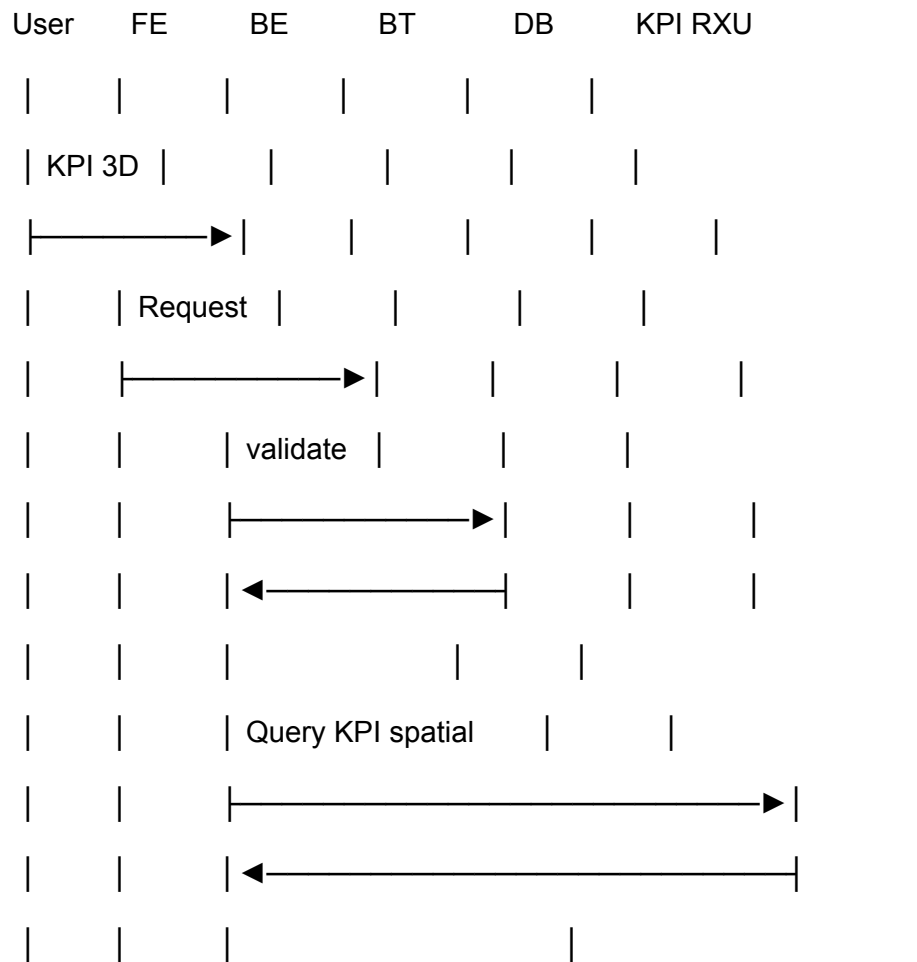
Run RUN_025

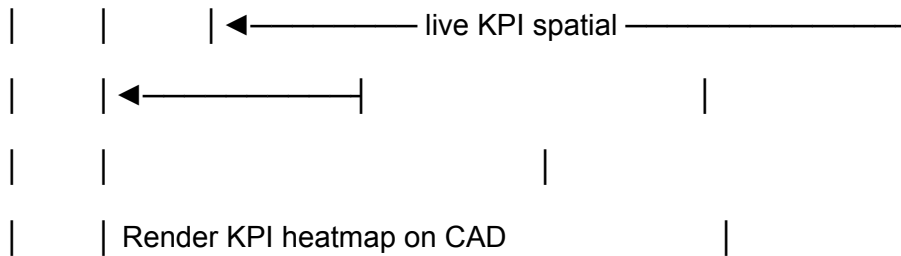
Layer L05

Course C07

Tow T12

Sequence:





15. User wants KPI + deviation simultaneously

This should be explicitly supported.

User selects:

[✓] KPI

[✓] TCP Deviation

and perhaps:

KPI = Tow Tension

Then:

User

|

▼

Frontend

|

| VisualizationRequest

|

| Run RUN_025

| Layer L05

| Course C07

| Tow T12

|

| Primary TOW_TENSION

| Secondary TCP_DEVIATION



Backend

|

|—— Query/receive KPI Spatial data

|

|—— Query/receive Deviation data

|

|—— Match by context/path position

|



Frontend

Recommended visualization:

CAD surface

=

KPI heatmap

TCP trajectory

=

deviation color

This makes the two quantities distinguishable.

16. User switches to KPI-only

User

|

| Disable TCP deviation

▼

FE

|

▼

BE

|

| stop sending deviation stream

|

| continue KPI stream

▼

FE

Importantly:

Deviation RXU continues calculating.

Only browser delivery changes.

17. User switches to deviation-only

Same concept:

FE → BE

KPI visualization OFF

Deviation visualization ON

Backend stops forwarding KPI samples to that particular browser session.

KPI RXU still continues processing.

18. User changes filter

You mentioned many important IDs:

Product ID

Part ID

Run ID

Program ID

Layer ID

Course ID

Strip ID

Tow ID

Changing any of these should normally be:

User

|

| Tow T12 → T17



FE

|

| FilterChanged

▼

BE

|

| query filtered data

▼

DB RXU

|

▼

result

|

▼

BE

|

▼

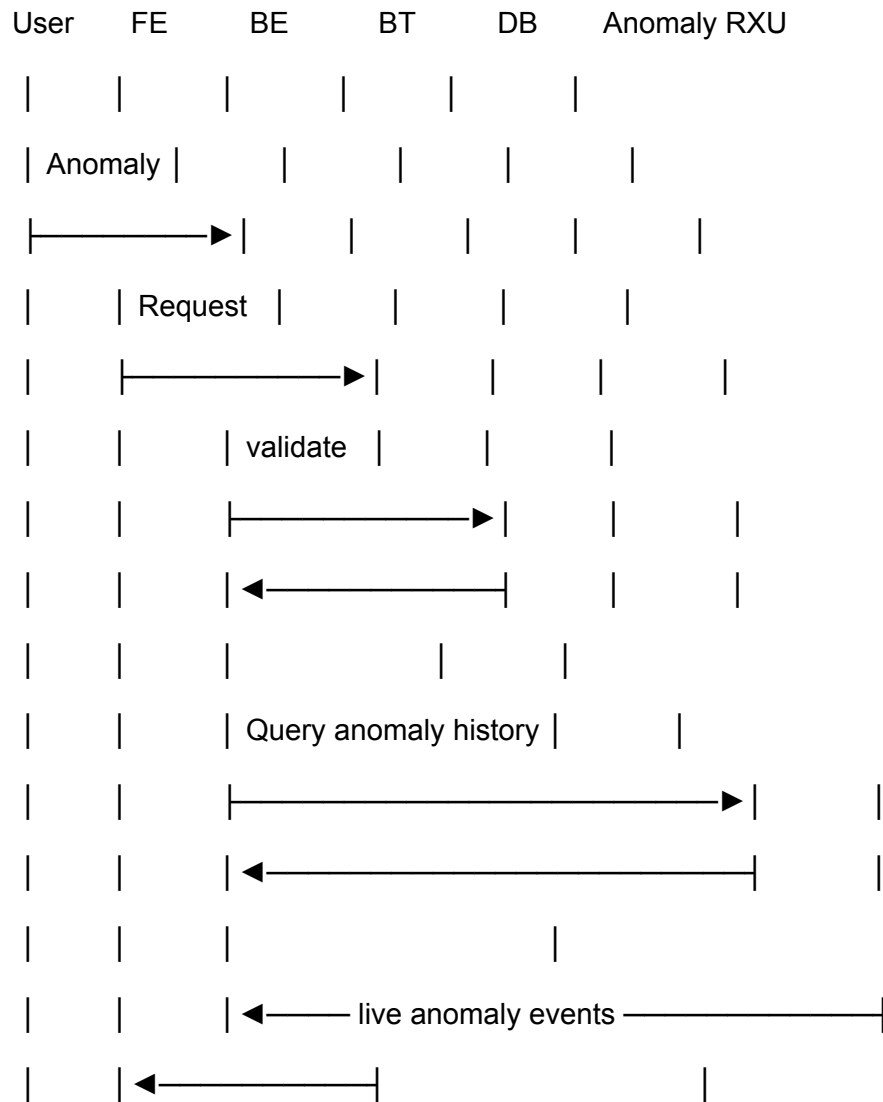
FE

No BT workflow is needed.

BT can optionally receive the selected context for session awareness, but it should not be executing the filter query.

19. User clicks Process Anomaly

The anomaly app has already been running.



The frontend can show:

anomaly count

severity

KPI

Tow

Course

time

description

Then the user can open detailed Grafana visualization.

20. User clicks "Open in Grafana"

Example:

User

|

| Click anomaly

▼

FE

|

| OpenDetailedTrend(

| run=RUN_025,

| course=C07,

| tow=T12,

| KPI=TENSION,

| time=10:22:18

|)

▼

BE

|

| construct dashboard context

▼

Grafana

Grafana then displays the corresponding time window.

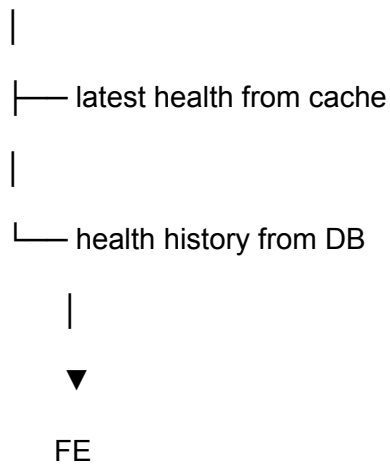
The Time-Series RXU App detected the anomaly.

Grafana only visualizes it.

21. User clicks Machine Health

Machine Health RXU is also continuously running.

User → FE → BE



Possible display:

Machine

Component

Health State

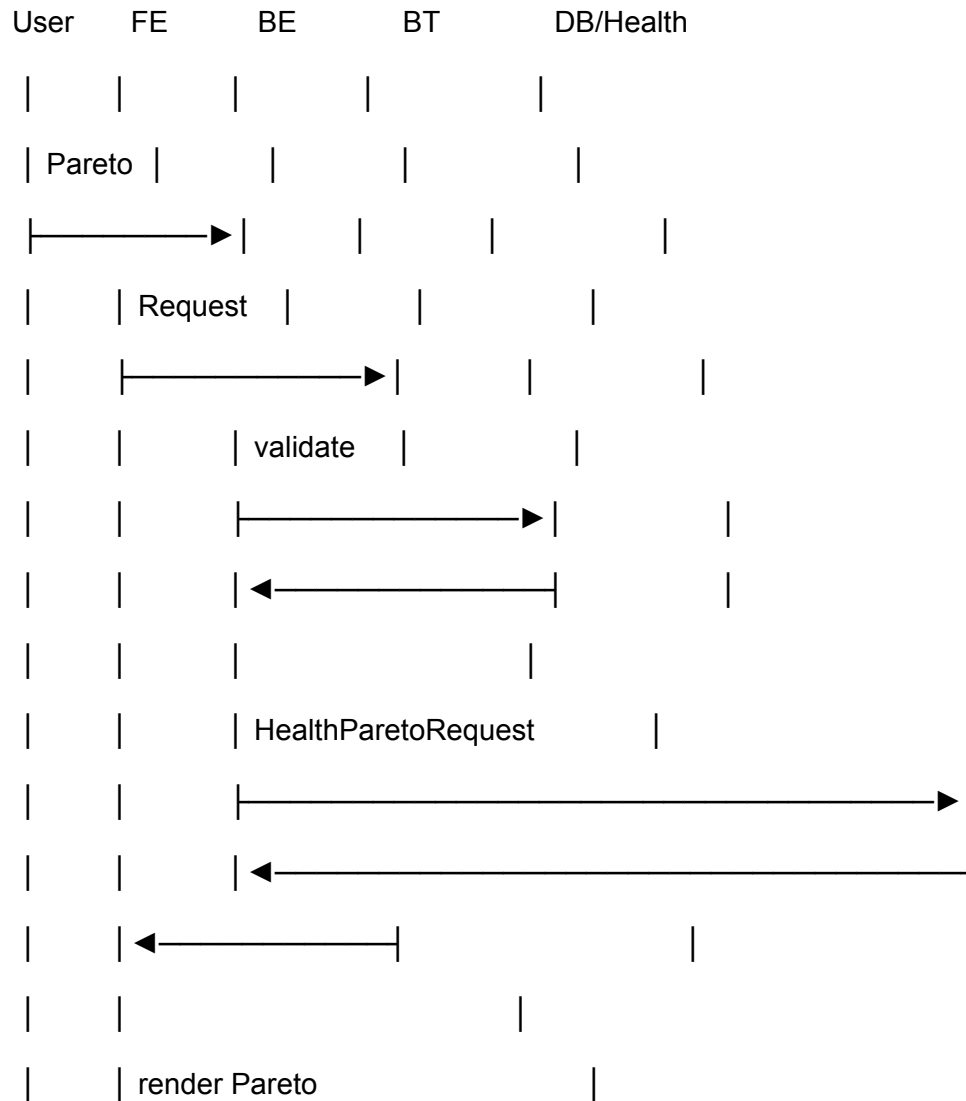
Fault Code

Duration

Latest Event

22. User clicks Maintenance Pareto

Here there is actual aggregation/filtering involved.



The user can change:

time range

severity

machine

component

fault category

and the Pareto is recomputed/query-aggregated.

23. User clicks Predictive Maintenance

The dummy predictive-maintenance RXU should preferably already be producing periodic predictions.

Therefore:

Predictive RXU

|

| periodically

▼

Database

When user opens the page:

User → FE → BE → DB

Get:

health score

failure probability

remaining hours

prediction trend

Grafana can visualize these.

Again, opening the page does not start the prediction model unless you intentionally design an on-demand model.

24. Where BT is really important when the user clicks

The important distinction is:

Simple visualization click

User → FE → BE → [optional BT validation] → data

Complex workflow click

For example:

Root Cause Investigation

This is where BT should genuinely orchestrate:

User

|

| Start Investigation

▼

FE

|

▼

BE

|

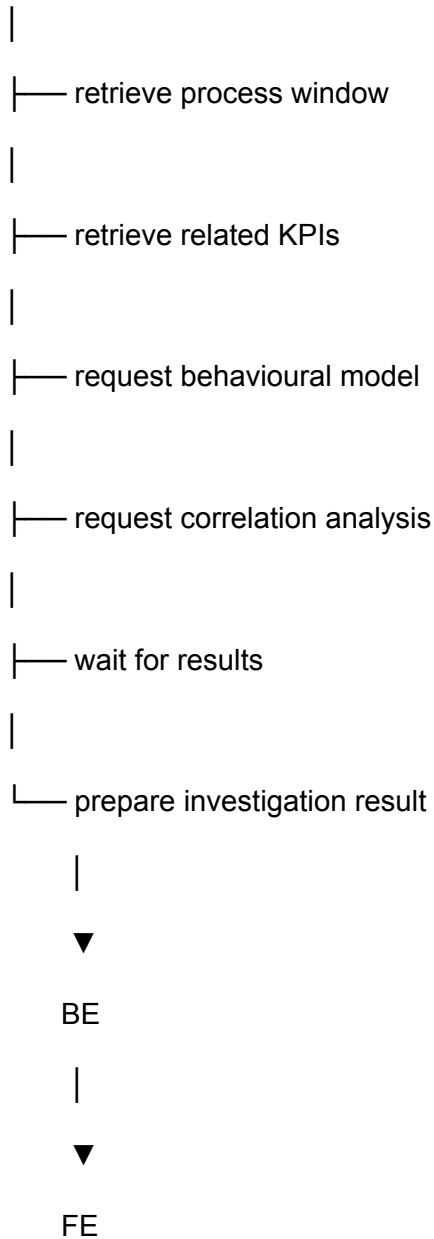
| InvestigationRequest

▼

BT

|

|— validate selected anomaly



That is substantially different from clicking a graph.

25. A generic frontend sequence covering all your pages

You can use this as the main HMI runtime sequence diagram:

User

|

| Login

▼

HMI Frontend

|

| GetSystemStatus

▼

HMI Backend

|

|

▼

BT

|

| Current system / run state

▼

HMI Backend

|

▼

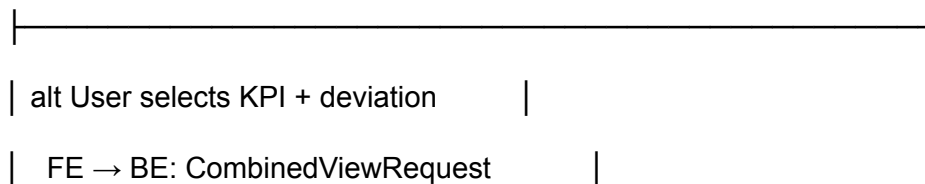
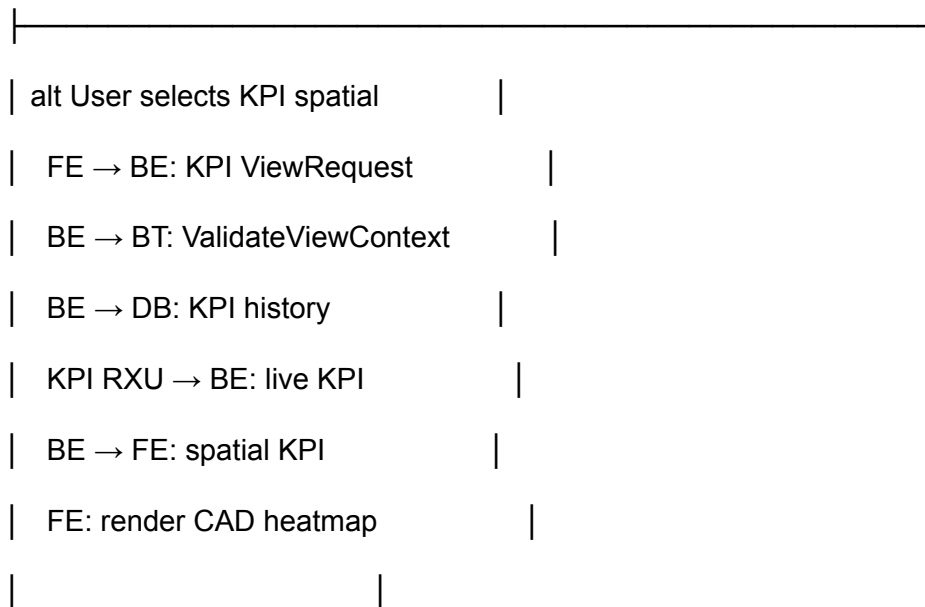
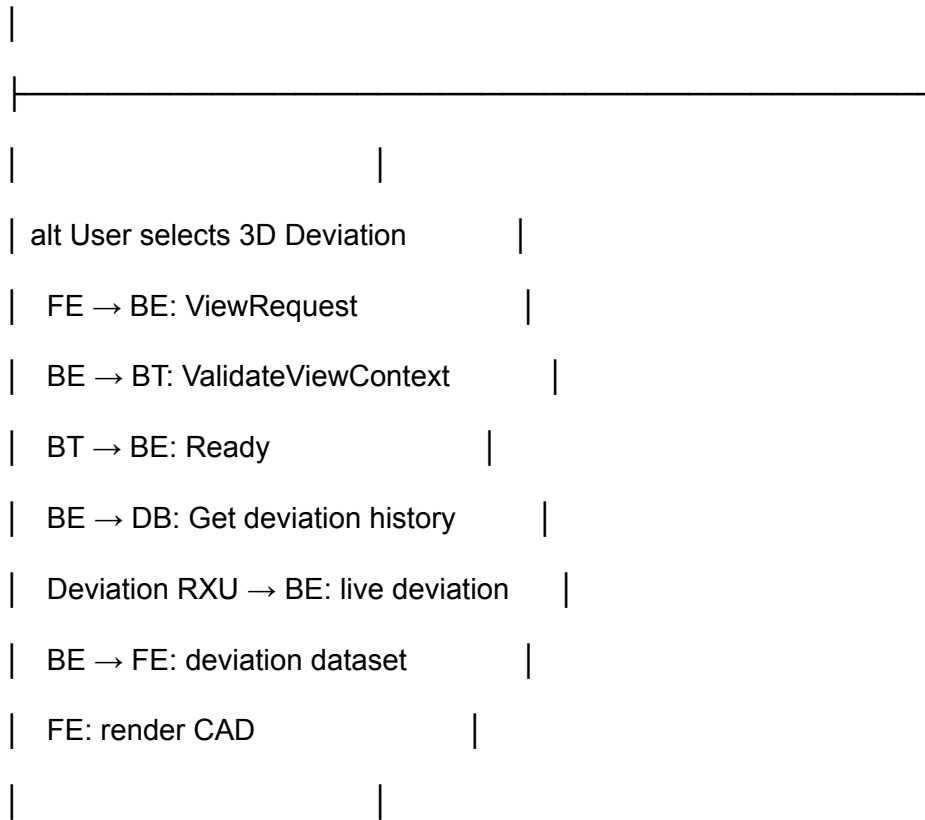
Frontend

|

|

| Display Overview

|



| BE: combine/filter both datasets |

| BE → FE: both datasets |

| FE: KPI surface + deviation path |

| |

|-----|

| alt User selects Process Anomaly |

| FE → BE |

| BE → DB: anomaly history |

| Anomaly RXU → BE: live anomalies |

| BE → FE |

| FE/Grafana: timeline/dashboard |

| |

|-----|

| alt User selects Machine Health |

| FE → BE |

| BE → DB |

| BE → FE: health information |

| |

|-----|

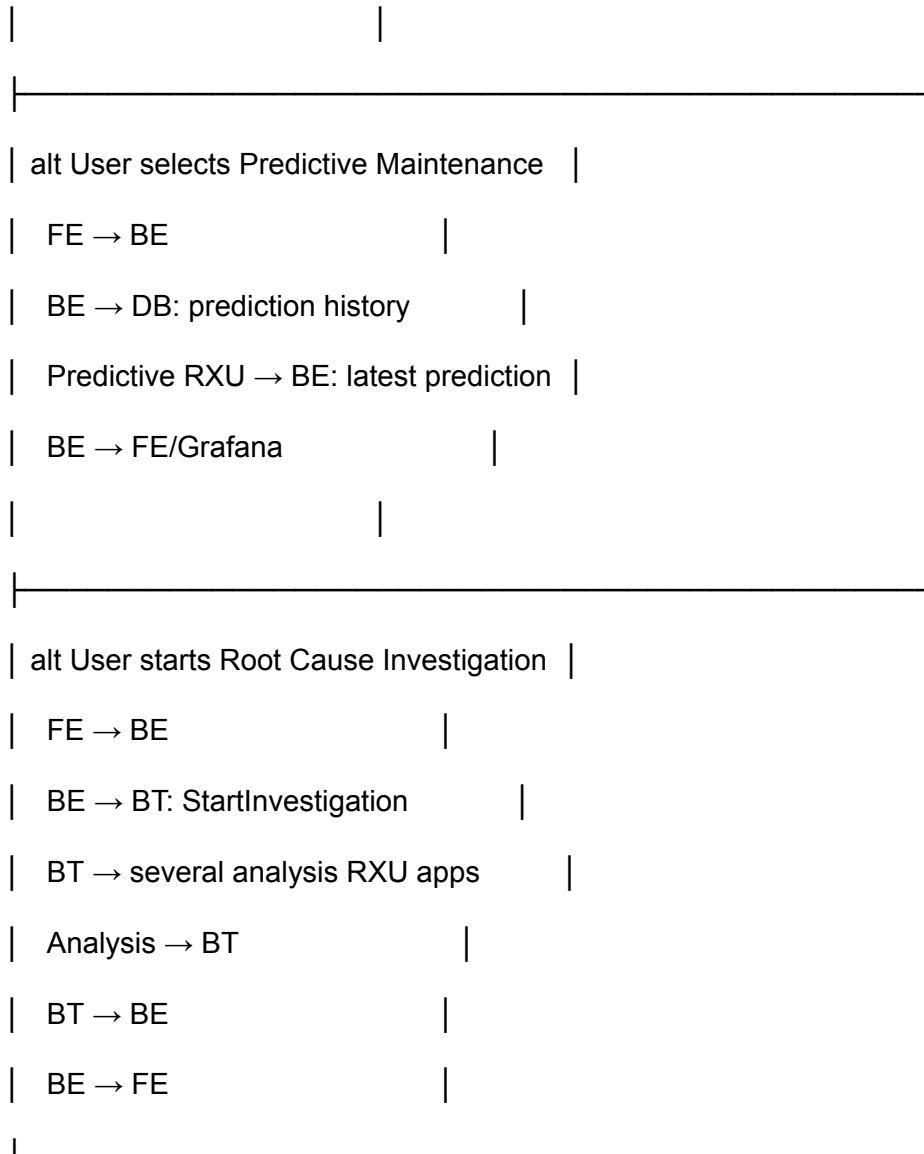
| alt User selects Pareto |

| FE → BE |

| BE → DB: aggregate health events |

| DB → BE: ParetoResult |

| BE → FE |



26. What if the user never logs in?

This is another important architecture test.

The system should still behave like:

IPC ON

↓

RXU apps READY



AFP RUN detected



monitoring active



TCP/joint deviation calculated

KPI analysis calculated

anomalies detected

machine health analyzed

predictive results generated



stored in QuestDB

Then the user logs in 30 minutes later.

The HMI can retrieve the previous 30 minutes from Database RXU and combine it with the current live stream.

This is much more robust than tying monitoring to login.

27. What if the user logs in before AFP starts?

Then:

HMI

System Status

RXU Platform	READY
--------------	-------

Connectivity	CONNECTED
--------------	-----------

Database	READY
----------	-------

Machine	IDLE
---------	------

Manufacturing

Active Run	None
------------	------

Program	None
---------	------

Product	-
---------	---

Part	-
------	---

Process Monitoring

Status	WAITING FOR RUN
--------	-----------------

The HMI Backend can be subscribed to Connectivity execution-state events.

When:

RUNNING

arrives, the HMI automatically transitions to:

Process Monitoring

Status	LIVE
--------	------

Run RUN_025

Product P02

Part Panel_A

Layer L05

Course C07

No user intervention should be required.

28. The most important trigger from Connectivity

If you are defining an interface requirement for Connectivity, I would specifically request that they provide an **execution-context/state message**.

Something conceptually like:

```
message ExecutionContext {  
    Timestamp source_timestamp;  
  
    ExecutionState state;  
  
    string machine_id;  
  
    string run_id;  
    string program_id;  
  
    string product_id;  
    string part_id;
```

```
string layer_id;  
  
string course_id;  
  
string strip_id;  
  
  
uint32 active_nc_block;  
  
double path_progress_m;  
  
}
```

I would not rely only on:

actual TCP

because an actual TCP tells you where the machine is, but not sufficiently:

which product?

which part?

which run?

which program?

which course?

which layer?

Those identifiers are crucial for synchronization, storage, filtering, historical replay and HMI visualization.

29. One subtle issue: Tow ID

Tow ID can be slightly different.

For a machine laying multiple tows simultaneously, one TCP sample may correspond to:

Course C07

Tow 01

Tow 02

Tow 03

...

Tow 48

So I would not necessarily put a single `tow_id` in every machine-state message.

Instead you may have:

```
message TowKpiSample {  
    SampleHeader header;  
  
    string tow_id;  
  
    KpiType type;  
    double value;  
}
```

Thus:

TCP data

identified mainly by:

run/course/layer/strip/path position

Tow process data

additionally identified by:

tow_id

That will make the frontend filtering cleaner.

30. Recommended division of responsibilities

I would document the roles like this:

Component	Main responsibility
BT / Orchestrator RXU	Initialize RXU applications, manage system/run lifecycle, validate readiness, coordinate complex workflows
Connectivity RXU	Interface to AFP/machine and publish actual state, process data and execution context
Database RXU	Abstract QuestDB/NC storage and provide persistence/query services
Deviation RXU	Synchronize programmed vs actual and continuously calculate TCP/joint deviation
KPI Analysis RXU	Continuously calculate descriptive KPI trends and spatial mapping
Time-Series RXU	Continuously detect abnormal KPI behavior
Machine Health RXU	Continuously evaluate machine/component health

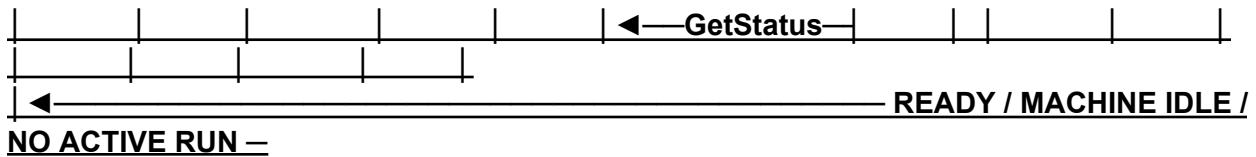
Predictive Maintenance RXU	Produce periodic/continuous maintenance predictions
HMI Backend RXU	RXU-to-web gateway, live cache, filtering, querying, frontend API
HMI Frontend	User interaction and visualization
Grafana	Specialized time-series/anomaly/predictive engineering visualization

The architecture principle I would put directly under this table is:

The BT controls lifecycle and workflows, while RXU Pub/Sub carries continuous process and analysis data. HMI user navigation shall not start or stop the underlying monitoring calculations.

That gives you a consistent answer for all the frontend requirements, regardless of whether the final UI uses separate pages, tabs, panels, or drill-down views.

Full startup sequence



i didn't understand this because RXU connectivity (only) can what is the status (that is the only one component communicating with the peripherals)

here you mentioned that get status from analysis app, how?

You are right. In your architecture, **R XU Connectivity is the authoritative source for machine/peripheral status** because it is the component connected to the AFP equipment.

So my previous startup sequence was misleading when it showed **GetStatus** effectively coming from an analysis app. The analysis apps can report **their own health/readiness**, but they cannot report machine state unless they consume it from Connectivity.

The clean separation should be:

Machine / peripherals status



Connectivity RXU



published execution / machine state

Component readiness



each RXU app



BT / health supervision

These are two different types of status.

For example, Connectivity RXU can publish:

Machine state = IDLE

Program loaded = false

Run ID = none

Course ID = none

Peripheral connectivity = OK

while the Deviation RXU can only report something like:

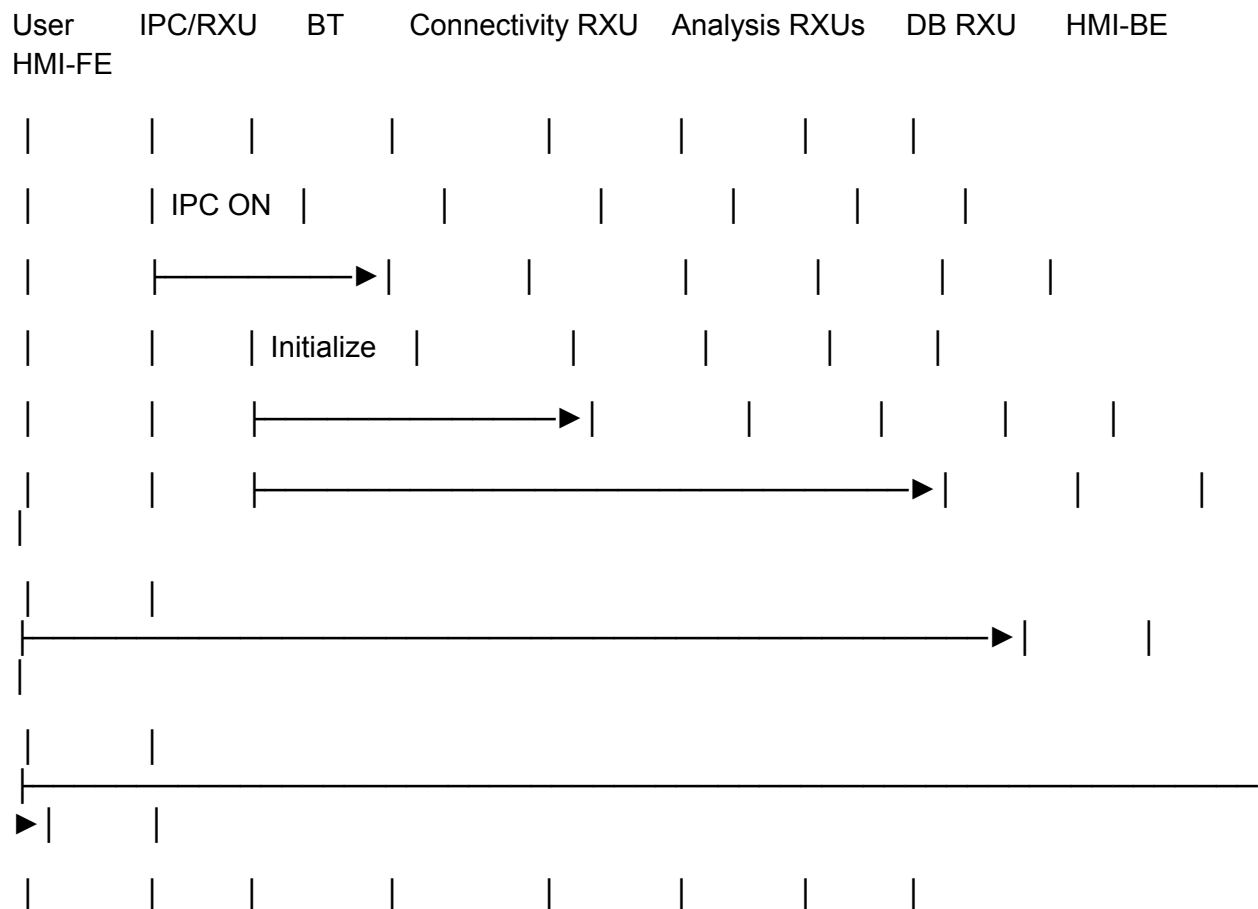
Component state = READY

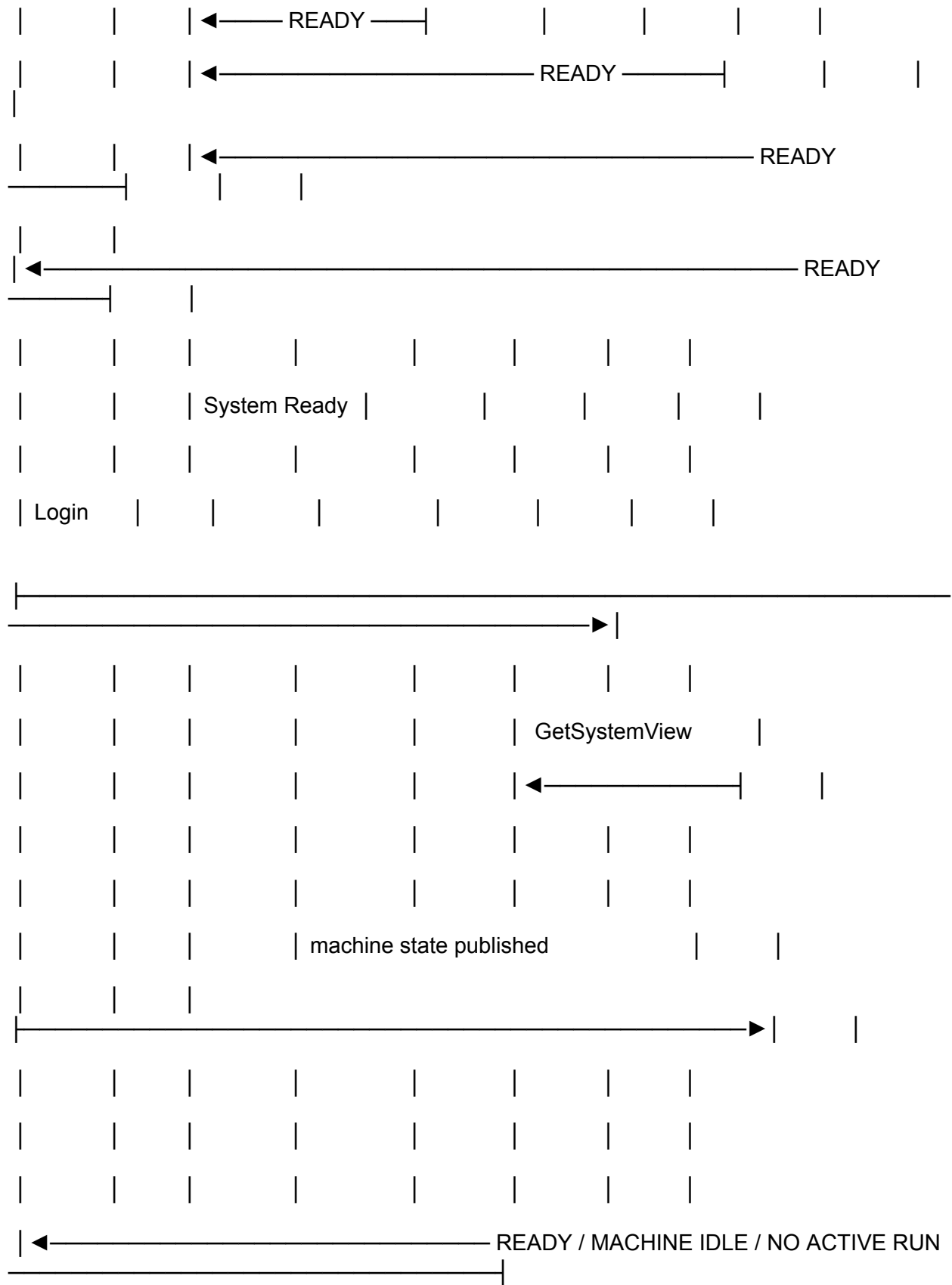
Program reference loaded = false

Waiting for active run

The Deviation RXU does **not** know independently whether the machine is IDLE. It only knows what it receives from Connectivity.

So I would correct the startup sequence like this:





The important part is that HMI Backend can know two categories of information:

System/component status

from BT / RXU health supervision

Machine/process status

from Connectivity RXU

So when the HMI shows:

System status READY

Connectivity READY

Database READY

Deviation App READY

KPI App READY

Machine state IDLE

Active run None

Program ID None

the first group comes from component readiness, while the second group comes from Connectivity.

A more precise runtime picture is:

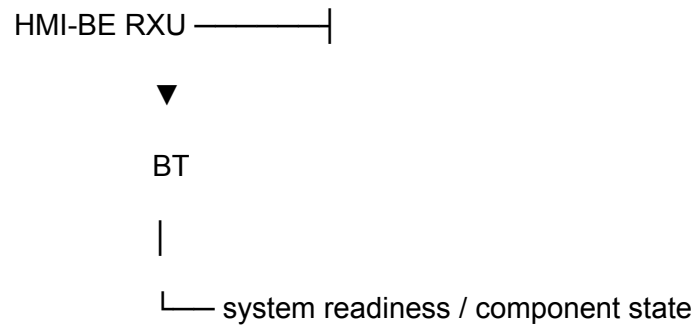
COMPONENT HEALTH

Deviation RXU ———┐

KPI RXU —————┐

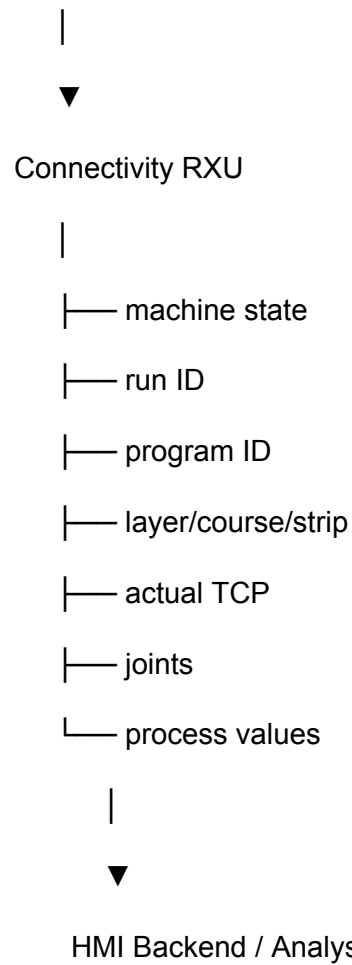
Anomaly RXU ———┐

DB RXU —————┐



MACHINE STATE

AFP machine



So, for the HMI startup, I would not make HMI Backend ask the Analysis RXUs:

“What is the machine status?”

It should instead use:

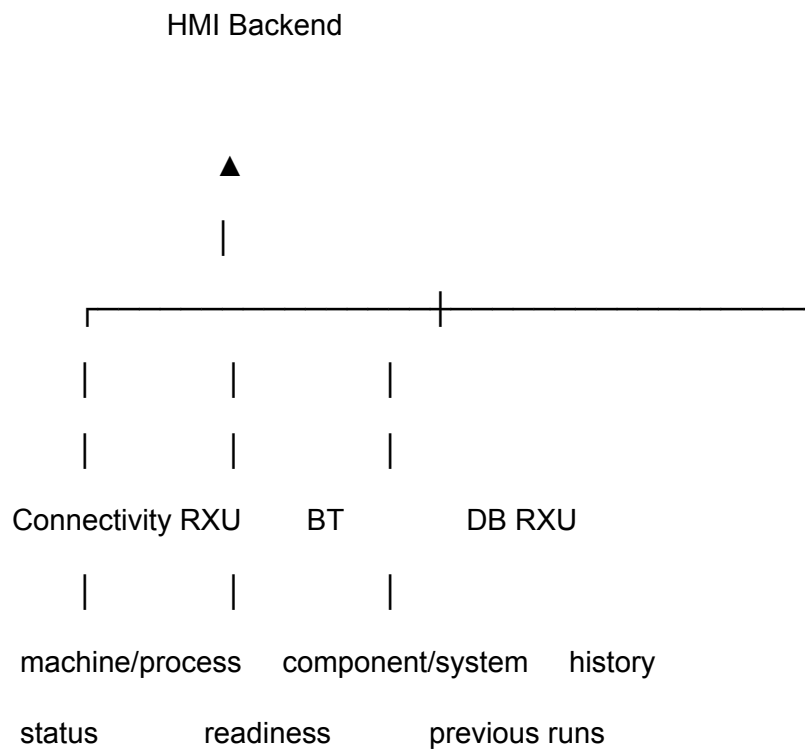
Connectivity RXU → machine/process state

BT → system/component readiness

DB RXU → historical/current stored data

This gives the HMI the complete picture without violating component responsibility.

The corrected high-level status aggregation is therefore:



This is the model I would use in your sequence diagram.

